

Vernetzte Systeme Zusammenfassung

Maximilian Wolf & Enes Karadeniz

6. April 2026

Inhaltsverzeichnis

1	A Einführung	9
1.1	Netzwerke	9
1.1.1	Begriffe	9
1.1.2	Heimnetzwerk	10
1.1.3	Network Core	10
1.1.4	Routing und Forwarding	10
1.1.5	Vernetzung globaler ISPs	10
1.2	Verzögerungen, Verlust und Durchsatz	11
1.2.1	Processing Delay (Verarbeitungsverzögerung)	11
1.2.2	Queueing Delay	11
1.2.3	Transmission Delay (Übertragungsverzögerung)	11
1.2.4	Queueing Delay	11
1.2.5	Propagation Delay (Ausbreitungsverzögerung)	12
1.2.6	Traceroute	12
1.3	Schichtenmodell	12
1.3.1	Internet Protocol Stack	12
1.3.2	ISO/OSI-Referenzmodell	13
1.3.3	Kapselung	13
1.4	Netzwerksicherheit	13
1.4.1	Malware	13
1.4.2	Beispiele für Angriffe auf Netzwerke	14
1.5	Historischer Überblick	14
1.5.1	Geschichte des Internets	14
2	B Netzwerkanwendungen	15
2.1	Erstellung	15
2.2	Architekturen	15
2.2.1	Client-Server	15
2.2.2	Peer-to-Peer	15
2.3	Anwendungsprotokolle	16
2.3.1	Spezifikation	16
2.3.2	Wichtige Aspekte	16
2.4	Sockets	17
2.4.1	Kommunikation zwischen Prozessen	17
2.4.2	Adressierung	17
2.4.3	Dienste der Transportschicht für Anwendungsschicht	17
2.4.4	Sicherheit bei TCP - TLS (früher SSL)	18
2.5	HTTP und Web	18
2.6	HTTP	18

2.6.1	Verbindungsablauf	19
2.6.2	Nachrichtenformat	19
2.6.3	HTTP Response	20
2.7	E-Mail	21
2.7.1	Simple Mail Transfer Protocol (SMTP)	21
2.7.2	POP3-Protokoll	22
2.7.3	IMAP-Protokoll	22
2.8	Domain Name System (DNS)	22
2.8.1	Dienste und Struktur	23
2.8.2	DNS Nameserver	23
2.8.3	DNS Namensauflösung	24
2.8.4	DNS Caching	25
2.8.5	DNS Records	25
2.8.6	DNS Protokoll	26
2.8.7	Angriffe auf DNS	26
2.9	Peer-To-Peer (P2P)	27
2.9.1	P2P-Architektur	27
2.9.2	Vergleich zu Client-Server	27
3	C Transportschicht	28
3.1	Dienste und Protokolle	28
3.1.1	Multiplexing und Demultiplexing	28
3.2	User Datagram Protocol (UDP)	28
3.2.1	Segmente	29
3.2.2	Checksummen	29
3.3	Zuverlässiger Datentransport	30
3.3.1	Reliable Data Transfer (RDT)	30
3.3.2	Pipelining	33
3.4	Transmission Control Protocol (TCP)	34
3.4.1	Segment-Struktur	35
3.4.2	Sequenznummer und ACKs	35
3.4.3	Round Trip Time und Timeouts	35
3.4.4	Zuverlässiger Datentransport	36
3.4.5	Flusskontrolle	36
3.4.6	Connection Management	36
3.5	Überlastkontrolle	38
3.5.1	Ansätze der Überlastkontrolle	38
3.5.2	Überlastkontrolle bei TCP	38
3.5.3	Multipath TCP	39
3.6	HTTP/2 und HTTP/3	40
3.6.1	Probleme bei HTTP/1.1	40

3.6.2	HTTP/2	41
3.6.3	HTTP/3 & QUIC	42
3.6.4	Entwicklung des HTTP-Protokoll-Stacks	43
4	D Netzwerkschicht	43
4.1	Aufgaben	43
4.1.1	Forwarding und Routing	43
4.1.2	Data Plane und Control Plane	44
4.1.3	Control Planes	44
4.1.4	Datagramm-basierte Netzwerke und Virtual Curcuits	44
4.1.5	Service-Modelle	45
4.2	Router	45
4.2.1	Router-Architektur	45
4.2.2	Eingangsports	46
4.2.3	Routing-Tabellen	46
4.2.4	Switching Fabrics	47
4.2.5	Queueing an Router-Ports	47
4.2.6	Output ports	48
4.2.7	Pufferspeicher	48
4.2.8	Scheduling-Strategien	48
4.3	Internet Protokoll IPv4	49
4.3.1	Fragmentierung und Reassemblierung	49
4.3.2	Adressierung	50
4.3.3	Subnetze	50
4.3.4	Classless InterDomain Routing (CIDR)	50
4.3.5	Dynamic Host Configuration Protocol (DHCP)	51
4.3.6	Hierarchische Adressierung	51
4.3.7	Adressblöcke	52
4.3.8	Network Address Translation (NAT)	53
4.3.9	NAT Traversal Problem	53
4.3.10	Internet Control Message Protocol (ICMP)	54
4.3.11	Adressknappheit	54
4.4	Internet Protokoll IPv6	54
4.4.1	Motivation und Verbreitung	54
4.4.2	Header	55
4.4.3	Adressformat	56
4.4.4	Transition von IPv4 zu IPv6	57
4.5	Routing Algorithmen	57
4.5.1	Abstraktion	57
4.5.2	Klassifikation	58
4.5.3	Dijkstra's Link-State Routing-Algorithmus	58

4.5.4	Distance-Vector-Routing	58
4.6	Routing im Internet	59
4.6.1	Intra-AS Routing	59
4.6.2	Inter-AS Routing	61
4.7	Software Defined Networking	63
4.7.1	Logisch zentralisierte Control Plane	63
4.7.2	Forwarding	63
4.7.3	Software Defined Networking (SDN)	64
4.7.4	OpenFlow Protokoll	65
4.7.5	Herausforderungen bei SDN	65
5	E Verbindungsschicht	66
5.1	Dienste der Verbindungsschicht	66
5.2	Technische Realisierung	67
5.3	Fehlererkennung	67
5.3.1	Paritätsprüfung (Parity Check)	67
5.3.2	Cyclic Redundancy Check (CRC)	68
5.4	Protokolle für Mehrfachzugriff	68
5.4.1	Link-Arten	68
5.4.2	Medium Access Control (MAC)	68
5.4.3	Kanalpartitionierung	69
5.4.4	Wahlfreier Zugriff (Random Access)	69
5.4.5	Carrier Sense Multiple Access (CSMA)	70
5.4.6	Taking-Turns-Mac-Protokolle	72
5.5	LANs	72
5.5.1	MAC-Adressen	72
5.5.2	Address Resolution Protocol (ARP)	73
5.5.3	Neighbor Discovery Protocol bei IPv6	73
5.5.4	IPv6: Stateless Address Autoconfiguration (SLAAC)	74
5.5.5	Ethernet	74
5.5.6	Spanning Tree Protocol	75
5.5.7	Virtual Local Area Network (VLAN)	76
6	F Physikalische Schicht	77
6.1	Signale	77
6.1.1	Periodisches Signal	77
6.1.2	Digitale Signale	77
6.1.3	Digitale Modulation	78
6.1.4	Bandbreitenbegrenzung	79
6.1.5	Nyquist-Rate	79
6.1.6	Shannon-Grenze	79

6.1.7	Signal-Noise-Ration (SNR)	79
6.2	Kabelgebundene Zugangsnetze	80
6.2.1	Asymmetric Digital Subscriber Line (ADSL)	80
6.2.2	VDSL & VDSL 2	81
6.2.3	Alternative Broadband Access Technologies	81
6.2.4	Glasfaser	82
6.2.5	Ethernet	83
6.3	Drahtlose Kommunikation	83
6.3.1	Bestandteile eines drahtlosen Netz	83
6.3.2	Herausforderungen drahtloser Links	84
6.4	IEEE 802.11 Wireless LAN (WLAN)	85
6.4.1	Architektur	85
6.4.2	Kanäle & Assoziierung	86
6.4.3	Mehrfachzugriff	87
6.4.4	Personal Area Networks (PANs): Bluetooth	88
7	G Wiederholende Zusammenfassung	89
7.1	Aufbau der Internetverbindung	89
7.2	ARP-Auflösung (noch vor DNS und HTTP)	90
7.3	DNS-Auflösung	91
7.4	TCP-Verbindungsaufbau	91
7.5	HTTP-Kommunikation	91
8	H Einführung Verteilte Systeme	92
8.1	H1 Motivation: Warum verteilte Systeme?	92
8.1.1	Definition verteilter Systeme	92
8.1.2	Verteilungsbegriffe	93
8.1.3	Anwendungsszenarien verteilter Systeme	94
8.2	Definitionen und Grundlagen	95
8.2.1	Flynn'sche Taxonomie	95
8.2.2	Nebenläufigkeit, Parallelität & Verteilung	96
8.2.3	Zentralisierte vs. dezentralisierte Topologien	97
8.2.4	Enge kopplung vs. lose Kopplung von verteilten An- wendungen	98
8.3	H.3 Design-Ziele und Merkmale verteilter Systeme	99
8.3.1	Gemeinsame Nutzung von Ressourcen	99
8.3.2	Offenheit	99
8.3.3	Heterogenität	100
8.3.4	Verteilungstransparenz	100
8.3.5	Auswahl an Verteilungstransparenz	101
8.3.6	Sicherheit	103

8.3.7	Skalierbarkeit	104
8.3.8	Fehlertoleranz	104
8.4	H.4 Fallstricke verteilter Systeme & Anwendungen	104
8.4.1	Fallstricke verteilter Systeme	104
8.4.2	Fallcies of Distributes Computing	106
8.4.3	The Twelve Networking Truths	106
9	I Konzepte und Paradigmen verteilter Systeme	107
9.1	Konzepte und Paradigmen	107
9.1.1	Betriebssysteme für verteilte Anwendungen	107
9.1.2	Lokale vs. globale Perspektive	108
9.1.3	Zeit in verteilten Systemen	108
9.2	Kommunikationsparadigmen	108
9.2.1	Nachrichtenbasierte Kommunikation	108
9.2.2	Remote Procedure Call (RPC)	110
9.3	Architekturstile verteilter Systeme	111
9.3.1	Client/Server-Architekturen	111
9.3.2	Mehrschichtige bzw. Multi-Tier Architekturen	111
9.3.3	Service-orientiere Architekturen	112
9.3.4	Publish/Subscribe-basierte Architekturen	113
9.3.5	Peer-to-Peer-Architekturen	114
9.3.6	Hybride Architekturen	115
9.3.7	Monolithische vs. modulare Architekturen	116
9.4	Koordinierung	117
9.4.1	Ansätze zur Zeitsynchronisation	117
9.4.2	Logische Uhren	118
9.4.3	Snapshots	119
9.4.4	Gegenseitiger Ausschluss (Mutex)	120
9.4.5	Einigung in verteilten Systemen	120
9.5	Replikation und Konsistenz	121
9.5.1	Konsistenzmodelle	121
9.6	Fehlertoleranz	122
9.6.1	Fehlermodelle	122
9.6.2	Umgänge mit Fehlern	122
10	J Anwendungen Verteiler Systeme	122
10.1	J.1 Verteilte Berechnung mit MapReduce	122
10.1.1	Motivation	123
10.1.2	Problemstellung: Wie solche Berechnungen durchführen?	123
10.1.3	Map & Reduce als verteiltes Programmiermodell	124

10.1.4	MapReduce-Paradigma als datenparalleles Berechnungsmodell	125
10.1.5	MapReduce: Simplified Data Processing on Large Clusters	126
10.1.6	Google MapReduce: Übersicht der Original-Architektur	127
10.1.7	Google MapReduce: Fehlertoleranz	127
10.1.8	Einige Optimierungen aus dem MapReduce Paper . . .	127
10.1.9	Ergebnisse der Performance-Evaluierung (aus Originalpaper von 2004)	128
10.1.10	Ausblick: Apache Hadoop	129
10.1.11	Berechnungen mit MapReduce	129
10.2	J.2 Bitcoin als Distributed Ledger Technology	130
10.2.1	Der Traum vom Digitalen Geld	130
10.2.2	Bitcoin: A Technical Perspective	130
10.2.3	Bitcoin verwaltet Transaktionen in einem Transaktionsgraph	131
10.2.4	Blockchain und Proof-of-Work	131
10.2.5	Die Blockchain	132
10.2.6	Was benötigen wir für Bitcoin (oder andere Kryptowährungen)?	132
10.2.7	Identität in Bitcoin	132
10.2.8	Kategorien von Kryptographie	133
10.2.9	Bitcoin Identitäten	134
10.2.10	Transaktionen	134
10.2.11	Digitale Signaturen	134
10.2.12	Schlüssel und Adressen in Bitcoin	135
10.2.13	Kryptografische Hashfunktionen	135
10.2.14	UTXO Modell (Unspent Transaction Output)	137
10.2.15	Verteilte, unveränderliche Datenbank (append-only) für Transaktionen	137
10.2.16	Block-ID	138
10.2.17	Konsens bei Bitcoin	139
10.2.18	Konsens	139
10.2.19	Blockchain Mining	140
10.2.20	Mining Intuition	140
10.2.21	Bitcoin Script	140

1 A Einführung

1.1 Netzwerke

Definition 1.1: Internet

"Netz von Netzen", Vernetzung von Internet Service Providern (ISPs)

1.1.1 Begriffe

- **Endsystem** (Host): Systeme auf denen Netzwerkanwendungen ausgeführt werden
- **Zugangsnetz** (Access Network): geben Endsysteme Zugriff auf (weitere) Netzwerke
- **Verbindung** (Link): Anbindung zwischen Sender und Empfänger
 - Physikalischer Link: gerichteter / drahtgebundenes Medium z.B. Kupferkabel
 - Ungerichtetes Medium z.B. Elektromagnetische Wellen
- **Router / Switches**: Packet Switching, leiten Datenpakete weiter
 - **Internet Service Provider** (ISP): weltweite Kommunikation zwischen Netzwerken
- **Network Edge**: Schnittstelle zwischen Endsystemen und Core-Netzwerk → Datenverarbeitung
- **Network Core**: Rückgrat des Netzwerks, Weiterleitung und Routing von Daten
- **Internet Standards**
 - **RFC**: Request For Comments: Offizielle Dokumente, die technische Standards des Internet definieren
 - **IETF** (Internet Engineering Task Force): internationale Gemeinschaft, die Standards entwickelt

Definition 1.2: Protokoll

Protokolle legen Formaten und die Reihenfolge der Nachrichten, die Netzwerkgeräte senden und empfangen, fest. Beispiele: TCP, IP, HTTP, 802.11

1.1.2 Heimnetzwerk

Definition 1.3: Digital Subscriber Line (DSL)

Breitbandanschluss an bestehenden Telefonleitungen von DSLAM (DSL Access Multiplexer) an Vermittlungsstelle

- **DSL Router**
 - Kabel / DSL-Modem
 - Router, Firewall, NAT
 - Wireless Access Point (WLAN)

1.1.3 Network Core

Definition 1.4: Packet-Switching

Zerlegung von Anwendungsdaten in kleinere Pakete, die jeweils einen Link exklusiv belegen

1.1.4 Routing und Forwarding

- **Routing:** bestimmt für Paket mit (Quell- und) Zieladresse den jeweiligen Ausgangsport → globale Operation im Überblick über alle Router
- **Forwarding:** Weiterleitung eines Pakets vom Eingangs-/ Input-Port zum Ausgangs-/ Output-Port → lokale Operation für Router

1.1.5 Vernetzung globaler ISPs

Problem: Verbindung jedes Netz mit jedem skaliert nicht bzw. benötigt enormen technologischen Aufwand $\mathcal{O}(n^2)$

- **Globale bzw. Tier 1 ISPs** werden über **Internet Exchange Points** (IXP) bzw. über **Peering Links** (bilaterale Verbindungen) vernetzt
- **Regionale ISPs** verbinden sich mit ein oder mehreren Globalen ISPs
- **Content Provider Networks** stellen kommerziell Inhalte für Endnutzer zur Verfügung und sind direkt angebundene Datenzentren großer Unternehmen (z.B. Google, YouTube)

1.2 Verzögerungen, Verlust und Durchsatz

Definition 1.5: Store-and-Forward

- Gesamtes Paket muss vor Weiterleitung beim Router angekommen sein
- **Ende-zu-Ende-Verzögerung:** $N \cdot \frac{L}{R}$ bei N Links bzw. N-1 Hops (Routern)

- Das Delay je Hop berechnet sich durch die Summe der Delays:

$$d_{hop} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

- **Durchsatz:** wird bestimmt durch **Bottleneck Link** (langsamster Link auf End-zu-End Pfad)

1.2.1 Processing Delay (Verarbeitungsverzögerung)

- Fehlerprüfung
- Ausgangslink bestimmen
- typischerweise $< \text{msec}$

1.2.2 Queueing Delay

- Wartezeit bis Output Link frei für Übertragung
- **Packet Loss:** ist Buffer voll (Congestion), muss das Paket gedroppt werden

1.2.3 Transmission Delay (Übertragungsverzögerung)

- Zeit, um Paket auf Link zu senden
- $d_{trans} := \frac{L}{R}$ wobei L: Länge des Pakets und R: Datenrate über Zugangnetzwerk

1.2.4 Queueing Delay

d_{queue} : Jeder Packet Switch hat mehrer Links mit jeweils einem Output Buffer bzw. Queue, um Pakete zwischenspeichern

- $\frac{L \cdot a}{R}$ wobei L: Paketlänge, R: Link Bandbreite und a: durchschnittliche Paketankunftsrate
- ≈ 0 : Queueing Delay klein
- $=1$: Queueing Delay groß
- >1 : mehr Pakete als verarbeitet werden können: Paketverlust

1.2.5 Propagation Delay (Ausbreitungsverzögerung)

- $d_{proc} = \frac{d}{s}$
- d: Länge des Links
- s: Ausbreitungsgeschwindigkeit im Medium

1.2.6 Traceroute

- Messung von Paketverzögerungen zwischen Sender und Empfänger über die dabei verwendeten Hops
- Für jeden Hop werden typischerweise 3 Probe Paket geschickt und empfangen und die **Round-Trip-Time** (RTT) gemessen

1.3 Schichtenmodell

- Internet hat hohe Komplexität: Hosts, Router, Links, Anwendungen, Protokolle, Hardware, Software
- **Schichtenmodell**: Strukturisierung, Modularisierung und Abstraktion auf verschiedenen Ebenen
- Höhere Schichten nutzen untere, letztere verbergen Komplexität

1.3.1 Internet Protocol Stack

- **Anwendungsschicht** (Application Layer): Netzerkanwendungen z.B. FTP, SMTP, HTTP
- **Transportschicht** (Transport Layer): Ende-zu-Ende Datentransfer zwischen Prozessen z.B. TCP, UDP
- **Netzwerkschicht** (Network Layer): Routing von Paketen z.B. IP, Routingprotokolle

- **Verbindungsschicht** (Link Layer): Datentransfer zwischen benachbarten Netzwerkelementen z.B. Ethernet, IEEE 802.11 (WiFi), DSL
- **Physikalische Schicht** (Physical Layer): Kodierung der Bits auf einem Medium

1.3.2 ISO/OSI-Referenzmodell

- ISO Open Systems Interconnection Modell, fügt zusätzliche Schichten hinzu:
- **Präsentationsschicht** (Presentation Layer): Datenrepräsentation z.B. Verschlüsselung, Komprimierung
- **Sitzungsschicht** (Session Layer): Synchronisation, Checkpoints, Sitzungswiederaufnahme
- Bei Internet Stack müssen diese auf Anwendungsschicht implementiert werden

1.3.3 Kapselung

- **Anwendungsebene:** Message $\boxed{M}$
- **Transportebene:** Segment $\boxed{H_t \mid M}$
- **Netzworkebene:** Datagram $\boxed{H_n \mid H_t \mid M}$
- **Verbindungsschicht:** Frame $\boxed{H_l \mid H_n \mid H_t \mid M}$

1.4 Netzwerksicherheit

1.4.1 Malware

- **Infektion** von Hosts / Routern
 - **Virus:** Infektion und (passive) Weiterverbreitung durch infiziertes Trägermedium z.B. Email, Datenträger, Dokumente
 - **Wurm:** Infektion und (aktive) Weiterverbreitung durch Netzwerk
- **Spyware:** Spionage eines PCs, speichert z.B. Tastendrucke, Browsing History

- **Ransomware:** Verschlüsselung eines Rechners und Freigabe durch Lösegeld
- **Botnetze:** infizierte Rechner verteilen Spam, führen DDOS-Attacken aus

1.4.2 Beispiele für Angriffe auf Netzwerke

- **Denial of Service (DOS)** Angreifer überlastet gezielt Rechnernetz bzw. Server, s.d. der Dienst für Nutzer nicht mehr zur Verfügung steht
- **Packet Sniffing** (Eavesdropping)
 - Broadcast-Medium erlaubt Zugriff durch Dritte z.B. Ethernet-Bus, WLAN
 - Interface empfängt Pakete und zeichnet diese auf z.B. Wireshark
 - **Lösung:** Verschlüsselung des Datenverkehrs
- **IP-Spoofing:** senden von Paketen mit gefälschten Absendeadressen

1.5 Historischer Überblick

1.5.1 Geschichte des Internets

- **1967 - 1972:** ARPAnet
 - **1967:** Advanced Research Projects Agency konzipiert ARPAnet
 - **1969:** erster Knoten des ARPAnet online
 - **1972:** öffentliche ARPAnet Demo, Network Control Protocol (NCP) als erstes Host-zu-Host Protokoll mit 15 Knoten, erstes E-Mail-Programm
- **1970:** ALOHAnet in Hawaii als erstes Funk-Rechnernetz
- **1976:** Ethernet (Xerox PARC)
- **1982:** Spezifikation SMTP-Protokoll
- **1983:** Einsatz von TCP/IP, DNS-Protokoll
- **1985:** FTP-Protokoll

- **Frühe 1990er:** Stilllegung des ARPAnet, Aufhebung der Beschränkung kommerzieller Anwendungen (NSFnet), HTTP bzw. HTML, erste Browser (Mosaic, Netscape)
- **Späte 1990er:** WWW, Instant-Messaging, P2P

2 B Netzwerkanwendungen

2.1 Erstellung

- Unterschiedliche Endsysteme
- Kommunikation typischerweise über **Application Programming Interface** (API) oder **Service Access Point** (SAP) z.B. Kommunikation von Webbrowser über TCP Sockets mit Webserver
- Architekturen: **Client-Server** / **Peer-to-Peer** (P2P)
- Komponenten im Kernnetz sind unabhängig von Netzwerkanwendungen

2.2 Architekturen

2.2.1 Client-Server

- **Server:** bietet Dienst an
 - dauerhaft erreichbar
 - permanente IP-Adresse / DNS-Name
 - Rechenzentren / Data Center
- **Client:** nutzen Dienst
 - Verbindung zum Server
 - Häufig dynamische, wechselnde IP-Adresse
 - Keine direkte Kommunikation

2.2.2 Peer-to-Peer

- Kein ständig verfügbarer Server
- Direkte Kommunikation zwischen **Peers**

- Nutzen Dienste anderer Peers bzw. bieten Dienst an
- Nur zeitweise verbunden (wechselnde IP-Adressen)
- **Sebstskalierbarkeit**: Steigerung der Leistungsfähigkeit eines Dienstes mit jedem weiteren Peer

2.3 Anwendungsprotokolle

2.3.1 Spezifikation

- **Nachrichtentyp**: Request / Response
- **Nachrichtensyntax**: Felder und Begrenzungen
- **Nachrichtensemantik**: Bedeutung der Informationen in Feldern
- **Verhalten der Beteiligten**: Wann senden Prozesse Nachrichten bzw. wie reagieren sie darauf?
- **Offene Protokolle**
 - Offene Spezifikation z.B. Internet RFCs
 - Sicherstellung von Interoperabilität
- **Proprietäre Protokolle**
 - Nicht-öffentliche Spezifikation
 - Häufig kommerzielle Anwendungen im Internet

2.3.2 Wichtige Aspekte

- ASCII-basiert vs. binäre Protokolle
- Control- vs. (Nutz-)Daten-Nachrichten
- Zentralisiert vs. dezentralisiert
- Zustandslos vs. zustandsbehaftet
- **Transport**: zuverlässig vs. unzuverlässig
- Komplexität im Edge-Netzwerk
- Core-Netzwerk: **KISS**: keep it simple and stupid

- Request/Reply-Nachrichtenaustausch
- **Nachrichtenformate**
 - **Header:** Informationen über Daten im Body
 - **Body:** eigentliche Daten

2.4 Sockets

Definition 2.1: Socket

Ein Socket ist die Schnittstelle zum Betriebssystem, mit der Prozesse Nachrichten senden und empfangen können.

2.4.1 Kommunikation zwischen Prozessen

- **Inter-Prozess Kommunikation:** selber Host z.B. shared memory
- **Austausch von Nachrichten** auf unterschiedlichen Hosts

2.4.2 Adressierung

- Host-Interface hat eindeutige 32-bit / 128-bit IP-Adresse
- Diese reicht jedoch einzig nicht aus, um einen Prozess zu adressieren
→ **Port**

2.4.3 Dienste der Transportschicht für Anwendungsschicht

- **Datenintegrität:** Zuverlässigkeit des Datentransports - Tolerierung von Paketveränderungen / Paketverlust
- **Quality of Service:** Latenz zwischen Endpunkten beim Transport von Anwendungsdaten - Notwendigkeit niedriger Latenzen z.B. Telephonie
- **Durchsatz:** Durchsatz bei Übertragung von Anwendungsdaten - Notwendigkeit eines Minstdurchsatzes
- **Security:** Vertraulichkeit und Unveränderlichkeit (Integrität) von Anwendungsdaten

TCP

- **Zuverlässiger Transport** zwischen sendendem und empfangendem Prozess

- **Flusskontrolle** (flow control): Sender überlastet Empfänger nicht
- **Lastkontrolle** (congestion control): Sender wird gedrosselt, wenn Netzwerk überlastet
- **Verbindungsorientiert**: expliziter Verbindungsaufbau zwischen Client und Server
- **Fehlend**: QoS, Delay, minimaler Durchsatz, Security

UDP

- **Unzuverlässiger Datentransport** zwischen sendendem und empfangendem Prozess
- **Verbindungslos**: Client und Server tauschen Nachrichten aus
- **Fehlend**: Zuverlässigkeit, Flusskontrolle, Lastkontrolle, Durchsatz, Security

2.4.4 Sicherheit bei TCP - TLS (früher SSL)

- **Verschlüsselte** und **integritätsgeschützte** TCP-Verbindungen
- **Authentisierung** der Kommunikationsendpunkte
- Realisierung im **Session Layer** oder **Application Layer**
- **TLS Socket API**: Daten, die an TLS Socket gesendet werden, werden verschlüsselt und übertragen

2.5 HTTP und Web

- Webseiten bestehen aus mehreren Ressourcen, darunter häufig HTML- und CSS-, Javascript- und Multimediadateien, Bildern usw.
- Eindeutige Identifikation durch **Uniform Resource Locator** (URL)

2.6 HTTP

- Anwendungsprotokoll des WWW
- Client-Server-Modell
- **Client** (meist Webbrowser)

- Senden von HTTP-Requests, Empfangen von Ressourcen
- **Browser**: häufig auch Rendern von Ressourcen
- **Server** (Webserver)
 - beantwortet HTTP-Requests mit HTTP-Responses
 - Bereitstellung von statischen (Dateien) und dynamischen Daten (Datenbank)
- **Zustandslos** (Stateless): Server behält keine Informationen über frühere Clients

2.6.1 Verbindungsablauf

- **Nicht-persistente Verbindung**: maximal eine Ressource pro TCP-Verbindung
- **Persistente Verbindung**: Verwendung einer TCP-Verbindung für die sequentielle Übertragung mehrerer Ressourcen

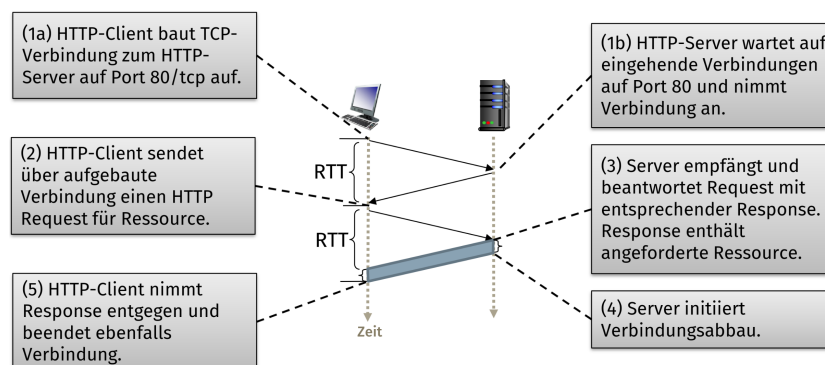


Abbildung 1: Verbindungsablauf HTTP

2.6.2 Nachrichtenformat

Methoden

- **GET**: (idempotenter) Abruf
- **POST**: (nicht idempotenter) Abruf, Client-Daten im Body
- **HEAD**: wie GET, jedoch ohne Body

- **PUT** (ab HTTP/1.1): Upload einer Ressource an spezifizierte URL
- **DELETE** (ab HTTP/1.1): Löschen einer Ressourcen auf dem Server
- **OPTIONS**

Beispiel HTTP 1.1 Requests für URL: `http://moodle.uni-ulm.de/my/`
`GET /my/ HTTP/1.1 Host: moodle.uni-ulm.de`

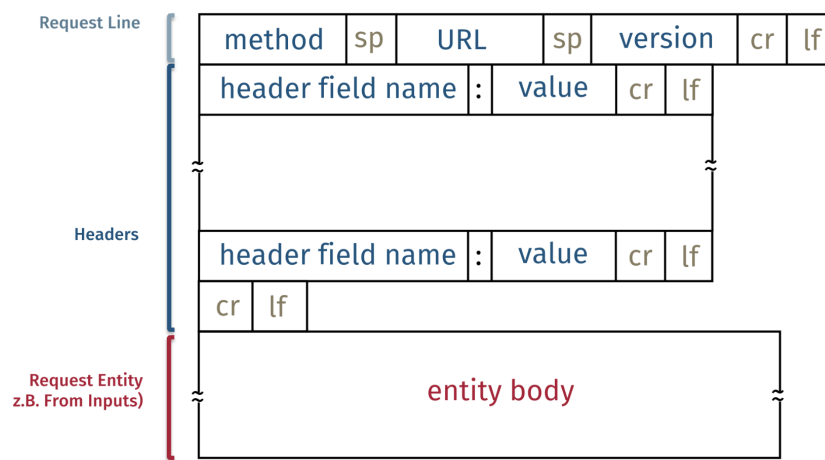


Abbildung 2: HTTP Request Nachrichtenformat

2.6.3 HTTP Response

- **1xx:** Informational
- **2xx:** Success
- **3xx:** Redirection
- **4xx:** Client Error
- **5xx:** Server Error

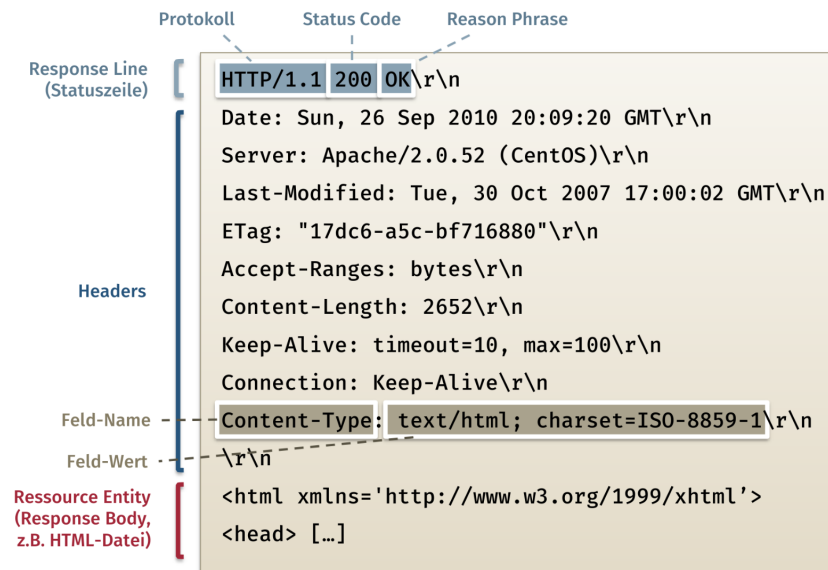


Abbildung 3: HTTP Response Nachrichtenformat

2.7 E-Mail

- **User-Agent:** Mail-Client
 - Erstellen und lesen von E-Mails
 - z.B. Outlook, Thunderbird
- **Mail-Server**
 - Mailbox beinhaltet eingehende Mail Nachrichten der User
 - **Simple Mail Transfer Protocol (SMTP)** zum Austausch von E-Mails zwischen Servern

2.7.1 Simple Mail Transfer Protocol (SMTP)

- TCP-basiert, well-known Port: tcp/25
- **Protokollphasen**
 1. Handshake (Hello)
 2. Nachrichtentransfer
 3. Beenden der Verbindung
- **Command/Response**-Protokoll, ASCII-basiert, MIME-Encoding

- Persistent: Übertragung mehrerer E-Mails in einer TCP-Verbindung möglich
- **Push-Modell**, erlaubt durch MIME auch multipart Nachrichten (mehrere Ressourcen)
- **RFC 822**: Aufbau der Nachrichten

2.7.2 POP3-Protokoll

Zustandslos, da keine Zustände über die einzelnen Emails gespeichert werden

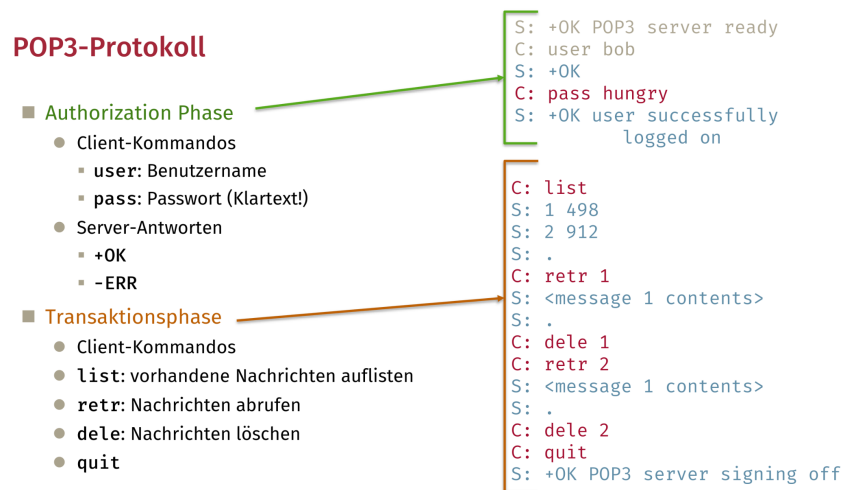


Abbildung 4: POP3 Protokoll

2.7.3 IMAP-Protokoll

- Nachrichten verbleiben üblicherweise auf Server
- Organisation von Nachrichten in Foldern durch User
- Zustände: Zuordnung zu Folder, Metadaten (z.B. gelesen, markiert)

2.8 Domain Name System (DNS)

- Menschenlesbare Identifikation eines Hosts, zum Routing wird IP-Adresse verwendet
- Verteilte Datenbank mit hierarchischer Topologie, Strukturierung durch viele DNS-Server

- Anwendungsprotokoll zur Kommunikation zwischen DNS Nameserver und DNS-Clients

2.8.1 Dienste und Struktur

- **IP Address Translation** ↔ **Reverse Translation**
- Host-Alias (alternativer Name)
- Zuständiger Mail-Server
- Lastverteilung
- **Dezentral**, da zentralisierte Lösungen in verteilten Systemen nicht skalieren (**Single Point of Failure**, Datenvolumen, Wartung und Zuverlässigkeit, verteiltes Management von Daten)

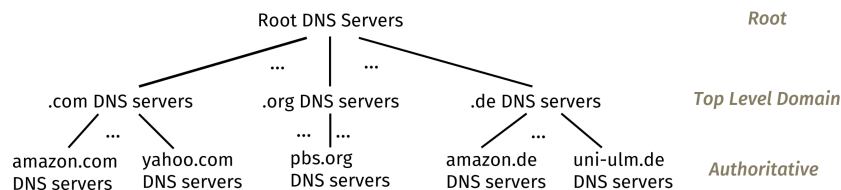


Abbildung 5: DNS - eine verteilte, hierarchische Datenbank

Ablauf

- Client erhält von **Root** DNS-Server **TLD** DNS-Server
- Client erhält von **TLD** DNS Server den **Authoritative** DNS-Server
- Client frag **Authoritative** DNS-Server nach IP-Adresse

2.8.2 DNS Nameserver

- **Root Nameserver**: Kontaktierung durch andere DNS-Server (selten, da Caching), Verwaltung durch Internet Corporation for Assigned Names and Numbers (ICANN)
- **Top-level Domain Server**: Betrieb durch Firmen oder Organisationen
 - **Country Code Domains**: de, uk, fr, ca, jp
 - **Generic TLDs**: com, org, net, edu

- **Authoritative DNS Server:** Verwaltung einer Zone im Bereich des DNS Namensraums, Primary und Secondary Nameserver für Redundanz
- **Local Resolving DNS Nameserver**
 - nicht zwingend Teil der Hierarchie, kann jedoch autorativ sein für bestimmte
 - verwendet von ISPs zur Bereitstellung eines DNS-Servers für Kunden

2.8.3 DNS Namensauflösung

- **Rekursiv**
 - angefragter Nameserver löst Namen vollständig auf
 - **Nachteil:** hohe Last für Root- und TLD-Nameserver
 - In Praxis daher Auflösung nur durch local resolver
- **Iterativ:** Jeder Server liefert Verweis auf nächsten Server, bis autoritativer Nameserver erreicht ist

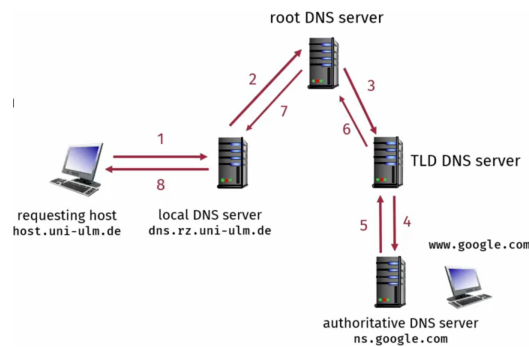


Abbildung 6: DNS Namensauflösung rekursiv

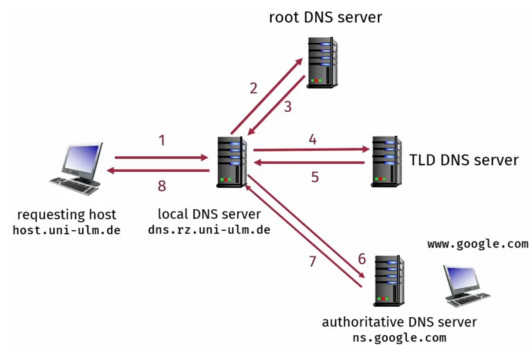


Abbildung 7: DNS Namensauflösung iterativ

2.8.4 DNS Caching

- Nameserver speichern frühere DNS-Anfragen, um Root- und TLD-Nameserver zu entlasten
- **Expiration:** nach TTL (Time-To-Live) werden ältere Einträge automatisch gelöscht, vor wichtigen Änderungen werden TTL-Werte reduziert

2.8.5 DNS Records

- **Resource Records (RRs):** DNS als verteilte Datenbank für RRs im Format (name, value, type, ttl)
- **A-Record**
 - **Name:** Hostname
 - **Value:** IP-Adresse
- **NS-Record**
 - **Name:** Domain
 - **Value:** Hostname des autoritativen Nameservers
 - mehrere Records für gleichen Namen möglich
- **CNAME-Record**
 - **Name:** Alias
 - **Value:** echter ("canonical") Name
- **MX-Record**

- **Name:** Host- oder Domainname
- **Value:** Name des zuständigen Mailservers

2.8.6 DNS Protokoll

- Binäres Protokoll, verwendet TCP und UDP
- Gleiches Nachrichtenformat für **Queries** und **Replies**
- **Identification:** 16 Bit für Zuordnung von Query und Reply

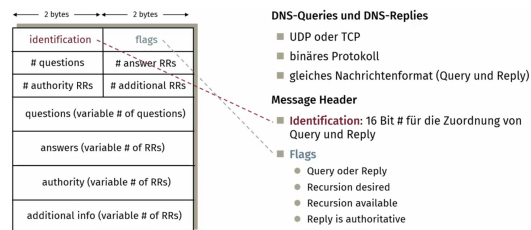


Abbildung 8: DNS Nachricht Aufbau

2.8.7 Angriffe auf DNS

- **DDoS:** kaum erfolgreich durch Traffic Filter, Caching von Root-/ TLD-Server
- **DNS Redirection**
 - **Man-in-the-Middle:** Angreifer fängt Datenverkehr ab und gibt sich als DNS-Server aus
 - **Cache Poisoning:** Caches in Middleboxes werden mit gefälschten Informationen "vergiftet"
- **DNS Amplification**
 - DNS-Server werden für DDoS Angriffe missbraucht
 - Angreifer sendet Queries mit gefälschten Absender-Adressen (dem Angriffsziel) an DNS-Server
 - DNS-Server schickt viele Antworten an Angriffsziel

2.9 Peer-To-Peer (P2P)

2.9.1 P2P-Architektur

- Kein ständig verfügbarer Server, Peers sind nur sporadisch online
- Direkte Kommunikation zwischen Hosts (Peers)
- Häufig verwendet für File Sharing, Streaming, VoIP

2.9.2 Vergleich zu Client-Server

- u_s : Upload-Geschwindigkeit des Servers
- d_{min} : minimale Client Download Rate
- **P2P**
 - Zeit für Übertragung einer Kopie: $\frac{F}{u_s}$
 - jeder Client muss eine Kopie übertragen
 - minimale Download-Zeit eines Clients: $\frac{F}{d_{min}}$
 - alle Clients zusammen müssen NF Bit übertragen
 - Zeit zur Übertragung von F an N Clients: $D_{P2P} \geq \max\{\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum u_i}\}$
- **Client-Server**
 - sequentielles Senden von N File-Kopien
 - Zeit für eine Kopie: $\frac{F}{u_s}$
 - Zeit für N Kopien: $\frac{NF}{u_s}$
 - minimale Client Download Zeit: $\frac{F}{d_{min}}$
 - Zeit zur Übertragung von F an N Clients: $D_{C-S} \geq \max\{\frac{NF}{u_s}, \frac{F}{d_{min}}\}$

Beispiel: Client Upload Rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

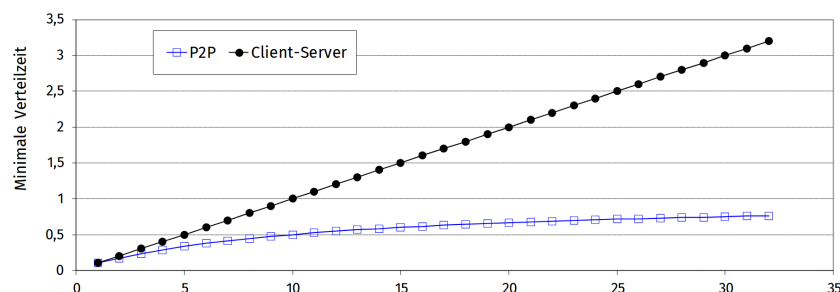


Abbildung 9: Verteilungszeit P2P vs. C-S

3 C Transportschicht

3.1 Dienste und Protokolle

- logische Kommunikation zwischen Anwendungen auf verschiedenen Hosts
- Protokolle laufen nur auf Endsystemen: Sender teilt Nachrichten in Segmente, Empfänger baut diese wieder zusammen
- **TCP**: zuverlässig und geordnet
 - **Überlastkontrolle** (congestion Control)
 - **Flusskontrolle** (flow control)
 - **Verbindungsaufbau** (connection setup)
 - **Fehlerbehandlung** durch wiederholte Übertragung (retransmission)
- **UDP**: unzuverlässig und ungeordnet
 - reicht den **”best-effort”** IP-Dienst an Prozesse weiter
- Beide Transportprotokolle unterstützen keine Garantie maximaler Verzögerung / Bandbreite

3.1.1 Multiplexing und Demultiplexing

- Multiplexing beim Sender: Verwaltung von Daten mehrerer Sockets, Hinzufügen von Transport headers
- Demultiplexing beim Empfänger: benutzt Header Informationen um Segmente in richtiger Reihenfolge an richtigen Socket auszuliefern
 - **IP-Datagramme**: Source & Destination IP
 - **UDP/TCP Segmente**: Source & Destination Port

3.2 User Datagram Protocol (UDP)

- ”Bare Bones” Internet Transport Protokoll
- ”Best Effort” Dienst: UDP Segmente können verloren gehen oder in falscher Reihenfolge ankommen
- Verbindungslos: kein Handshake zwischen Sender und Empfänger, jedes Segment (auch Datagram) wird unabhängig von anderen verarbeitet

3.2.1 Segmente

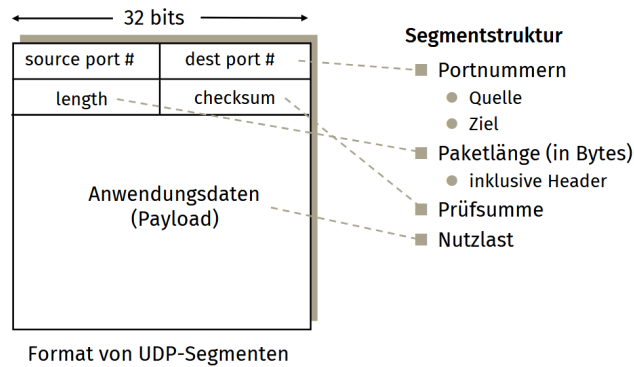


Abbildung 10: Aufbau eines UDP Segment bzw. Datagram

Vorteile von UDP

- Kein Verbindungsaufbau → weniger Delay
- Einfaches Protokoll, weniger Zustände als bei TCP
- Weniger Overhead durch kleinere Header
- Keine **Überlastkontrolle**: maximale Sendeleistung

3.2.2 Checksummen

- Erkennung von Fehlern im Segment
- **Sender** sendet Segmentinhalt als Sequenz von 16-Bit Integers durch Addition der Werte im Einerkomplement
- **Empfänger** berechnet ebenfalls Checksumme und überprüft diese mit der gesendeten auf Gleichheit

```

Sender:
Word1:  1001 1011 0100 0111
Word2:  1010 1011 1101 0111
Word3:  1111 0101 0011 1001
Sum:    0011 1100 0101 1001
Chksum: 1100 0011 1010 0110

Receiver:
Chksum: 1100 0011 1010 0110
Word1:  1001 1011 0100 0111
Word2:  1010 1011 1101 0111
Word3:  1111 0101 0011 1001
Sum:    1111 1111 1111 1111
Inv.:   0000 0000 0000 0000

```

Abbildung 11: Beispielhafte Überprüfung eines UDP Segments durch Checksumme

Checksummenberechnung

- **Sender:** Addition aller 16-Bit Wörter, danach Invertierung aller Bits
- **Empfänger:** Addition aller 16-Bit Wörter inkl. Checksumme des Senders und Invertierung des Ergebnisses, ist dieses gleich 0, stimmt die Checksumme überein
- **Vorteile:** nur ein Register notwendig, einfache HW-Umsetzung, ob Register gleich 0 ist

3.3 Zuverlässiger Datentransport

3.3.1 Reliable Data Transfer (RDT)

- Abstraktion eines Protokolls zur zuverlässigen Datenübertragung über einen unzuverlässigen Kanal
- **Sender**
 - `rdt_send()`: Aufruf aus Anwendungsschicht
 - `udt_send()`: Aufruf durch RDT, versendet Paket über unzuverlässigen Kanal
- **Empfänger**
 - `rdt_rcv()`: wird von unterer Schicht zum Paketempfang aufgerufen

- **deliver_data()**: Aufruf durch RDT um Daten an höhere Schicht zu liefern
- **Unidirektionale** Verbindung für zuverlässigen Kanal: Nutzlast fließt nur in eine Richtung
- **Bidirektionale** Verbindung für unzuverlässigen Kanal: Kontrollinformationen fließen in beide Richtungen

RDT 1.0

- Zuverlässiger Transport über zuverlässigen Kanal: keine Bitfehler / Paketverluste
- Darstellung durch **endliche Zustandsautomaten** (Finite State Machines / FSM): separate FSM für Sender & Receiver
- **Sender**: warten auf Aufruf aus oberer Schicht (`rdt_send()`), dann Aufteilung in Pakete und hinzufügen von Headern und `udt_send()` Aufruf
- **Empfänger**: ebenfalls warten auf Aufruf (`rdt_rcv()`), dann Daten aus Paket extrahieren und `deliver_data()` Aufruf



Abbildung 12: RDT 1.0 Zustandsautomat

RDT 2.0: Kanal mit Bitfehlern

- Zugrunde liegender Kanal ist fehlerhaft (flipped bits): **Checksumme** zur Erkennung von Fehlern
- Bestätigungen (Acknowledgement / **ACKs**): Empfänger teilt Sender mit, dass Paket fehlerfrei ist
- Negative Bestätigungen (negative acknowledgement / **NAKs**): Empfänger teilt Sender mit, dass empfangenes Paket fehlerhaft ist → Sender schickt Paket erneut (**retransmit**)

- **Problem:** Verlust von Paketen über unzuverlässigen Kanal möglich: Retransmissions sind nicht mehr ausreichend
- **Lösung:** Timeout für ACK
- Niedrige Auslastung (Utilization): $Util_{sender} = \frac{L/R}{RTT+L/R}$, Transportschichtprotokoll limitiert Auslastung des Netzwerks!

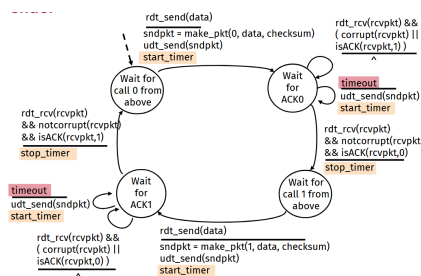


Abbildung 16: RDT 3.0 Zustandsautomat

3.3.2 Pipelining

Definition 3.1: Pipelining

Pipelining ermöglicht es dem Sender, mehrere Pakete gleichzeitig zu schicken, bevor ein ACK vom Empfänger erwartet wird, um die Auslastung zu erhöhen. Dazu sind mehrere Sequenznummern und das Puffern von nicht bestätigten Paketen beim Sender erforderlich. Bei einem nicht saturierten Link steigt die Auslastung der Bandbreite mit N Paketen um den Faktor N.

Go-Back-N

- **Sender**
 - Kann bis zu N nicht bestätigte Pakete in der Pipeline haben
 - Hält Timer von letztem nicht-bestätigtem Paket
- **Empfänger**
 - Sendet kumulative ACKs: Bestätigung immer von letztem Paket, von dem keine Unterbrechung, zuvor empfangene Pakete werden dadurch automatisch auch acknowledged

- Verwerfen von Paketen in falscher Reihenfolge, Empfänger benötigt keinen Puffer in Transportschicht



Abbildung 17: Go-Back-N

Selective Repeat

- **Sender**
 - Kann bis zu N nicht bestätigte Pakete in der Pipeline haben
 - Hält separate Timer für alle nicht bestätigten Pakete
- **Empfänger**
 - Sendet individuelle ACKs für jedes Paket
 - Pufferung von Paketen, um diese in korrekter Reihenfolge an obere Schicht zu geben

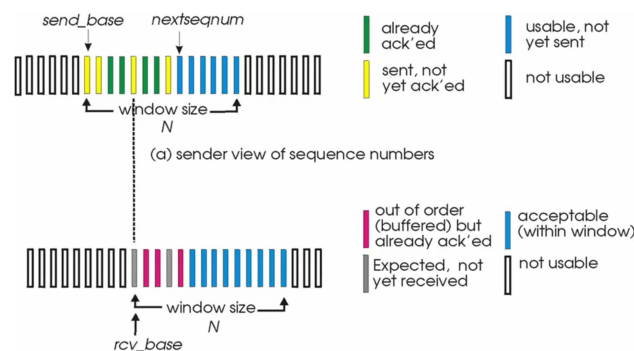


Abbildung 18: Selective Repeat

3.4 Transmission Control Protocol (TCP)

- **Full-duplex Datenverbindung:** bi-direktionaler Datenfluss, Zerlegung von Daten in Segmente (MSS: Maximum Segment Size)
- **Verbindungsorientiert** - Handshake: Kontrollnachrichten zur Initialisierung beim Verbindungsaufbau

- **Punkt-zu-Punkt Verbindung:** unicast (Sender & Empfänger)
- **Flusskontrolle:** Empfänger kann Datenrate des Senders drosseln
- Zuverlässiger & geordneter Bytestream: keine Nachrichtengrenze & kein explizites Senden von Paketen, darunter liegende Netzwerkschichten verantwortlich
- **Pipelined:** Überlast- und Flusskontrolle setzen Window Größe

3.4.1 Segment-Struktur

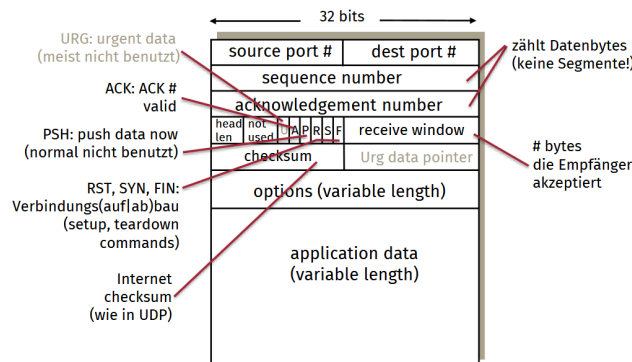


Abbildung 19: TCP Segmentstruktur

3.4.2 Sequenznummer und ACKs

- **Sequenznummer:** Nummer des ersten Bytes im Segment im Byte-stream
- **Acknowledgements:** Sequenznummer des nächsten Bytes, welches die empfangende Seite erwartet, Unterstützung von **Cumulative ACK**
- Separate Zähler für beide Richtungen!

3.4.3 Round Trip Time und Timeouts

- Abschätzung der **Round Trip Time** (RTT) durch exponentiell gewichtetem Mittelwert, wodurch der Einfluss vergangener Messungen exponentiell abnimmt
- $\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$, wobei typischerweise $\alpha = 0.125$ (Gewicht)

- Sicherheitsmarge $\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$, wobei meist $\beta = 0.25$ gilt
- $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$

3.4.4 Zuverlässiger Datentransport

- Implementierung eines RDT-Dienst auf Basis des unzuverlässigen "best-effort" IP-Protokolls
- **Single Retransmission Timer**, diese werden bei Timeouts oder Duplicate ACKs ausgelöst
- Grundlegendes Verhalten zu RDT mit GBN, verwendet jedoch Byte-zähler statt Segmenten
- **Fast Retransmit**: empfängt Sender 4 ACKs mit gleicher Sequenznummer (Triple Duplicate ACKs), sendet dieser das Paket mit der niedrigsten Sequenznummer noch vor Timeout

3.4.5 Flusskontrolle

- Empfangsprozess kontrolliert sendenden Prozess, um Überlaufen des Puffers auf Empfängerseite zu verhindern
- **Receive Window** im Segment: Anzahl an Bytes, die der Empfänger akzeptiert
- Empfänger teilt freien Puffer im **Receive Window** mit, Sender schickt dann niemals mehr Daten als die Größe des freien Puffers

3.4.6 Connection Management

Verbindungsaufbau

- **Handshake**: Einigung auf Verbindungsaufbau, Verhandlung von Verbindungsparametern

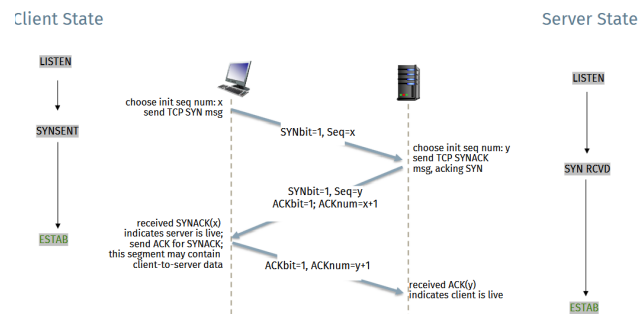


Abbildung 20: TCP 3-Way Handshake

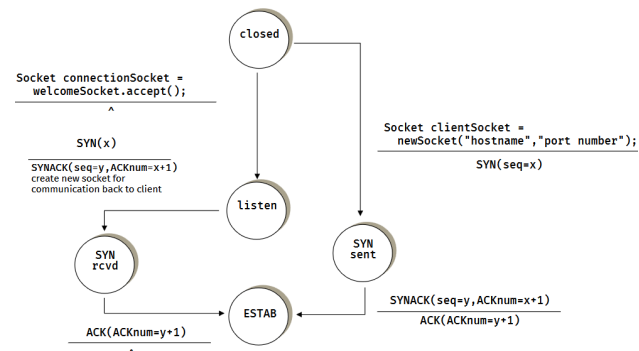


Abbildung 21: TCP Handshake als FSM

Verbindungsabbau

- Client & Server schließen Verbindung getrennt voneinander: senden von TCP-Segment mit FIN-Bit=1
- Empfänger bestätigt FIN mit ACK

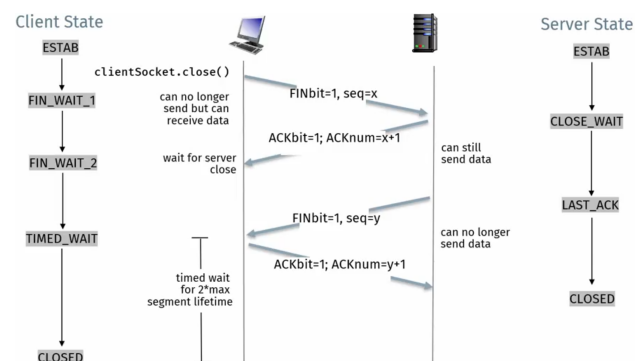


Abbildung 22: TCP Verbindungsabbau

3.5 Überlastkontrolle

Definition 3.2: Überlastkontrolle

- Überlast tritt auf, wenn zu viele Sender zu viele Daten schnell über ein Netzwerk senden → Paketverlust durch Überlaufen der Puffer in Routern
- Überlastkontrolle dient zur Verhinderung der Überlastung des **Netzwerks**
- Nicht identisch zu Flusskontrolle

3.5.1 Ansätze der Überlastkontrolle

End-to-End Congestion Control

- Kein explizites Feedback vom Netzwerk
- Überlast wird von Endsystemen durch Verlust und Verzögerungen abgeschätzt
- ursprünglicher TCP-Ansatz

Network-assisted Congestion Control

- Router liefern Feedback an Endsysteme
- einzelnes **Congestion Bit**
- explizite maximale Rate an Sender

3.5.2 Überlastkontrolle bei TCP

Additive Increase Multiplicative Decrease (AIMD)

- Sender erhöht Übertragungsrate durch Erhöhung des **Congestion Window** (cwnd), um sich an die verfügbare Bandbreite heranzutasten, bis Verluste auftreten
- **Additive Increase**: Erhöhung des cwnd um 1 MSS in jeder RTT, bis Verluste auftreten
- **Multiplicative Decrease**: halbiere cwnd nach Verlust

TCP Congestion Control

- Sender limitiert Bandbreite dynamisch: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$
- Sendefenster = $\min(\text{rwnd}, \text{cwnd})$ wobei rwnd das Receive-Window des Empfängers ist
- TCP-Rate $\approx \frac{\text{cwnd}}{\text{RTT}}$

TCP Slow Start & Congestion Avoidance

- **Slow Start:** exponentielle Erhöhung des cwnd, bis 3 duplicate ACKs auftreten (TCP Reno)
- **Congestion Avoidance:** lineare Erhöhung des cwnd nach Verlust

Explicit Congestion Notification (ECN)

- Zwei Bits (ToS-Feld) im IP-Header werden von Router gesetzt, um Congestion zu markieren
- Empfänger setzt **ECE-Bit** (ECN Echo Flag) in ACK Segmenten, um Sender zu benachrichtigen

Durchsatz

- **Durchschnittlicher Durchsatz:** $\frac{3}{4} \frac{W}{RTT} \frac{\text{bytes}}{\text{s}}$
- **Problem:** Long Fat Pipes TCP-Throughput = $\frac{1,22 \cdot \text{MSS}}{RTT \cdot \sqrt{L}}$, für 10Gbps Durchsatz benötigt man eine unrealistische Verlustrate von $L = 2 \cdot 10^{-10}$

Fairness

- Bei mehreren Verbindungen nähern sich die Clients durch AIMD automatisch einer fairen Aufteilung der Bandbreite an
- **Problem:** Anwendungen haben oft mehrere parallele TCP-Verbindungen

3.5.3 Multipath TCP

Definition 3.3: Multipath TCP (MPTCP)

TCP Variante mit Unterstützung von

- **Multi-homing:** Host hat mehrere Endpunkte / Interfaces
- **Multipath:** zwischen zwei Hosts werden mehrere Pfade für eine Verbindung genutzt

diese Variante ist rückwärtskompatibel mit anderen TCP Versionen

Smartphone Verbindung über Mobilfunknetz und WIFI

- SYN (Client) mit TCP Option **MP_CAPABLE**, SYN-ACKs (Server) sendet auch **MP_CAPABLE**
- Schlüsselaustausch durch **Three-way Handshake**
- SYN mit **MP_JOIN** und Key über andere Verbindung
- Smartphone kann beide Verbindungen nutzen und eine beenden, ohne die TCP-Verbindung zu beenden

Resource Pooling

- Server in Datenzentren haben häufig redundante Pfade
- Pooling der Bandbreite zwischen Knoten
- Bei Ausfall einer Verbindung: TCP-Verbindung bleibt erhalten

3.6 HTTP/2 und HTTP/3

3.6.1 Probleme bei HTTP/1.1

- **Head-of-Line Blocking** (sequenzielle Requests): langsame Bereitstellung einer Ressource verzögert Übertragung aller nachfolgenden Ressourcen
- **Notwendigkeit paralleler Verbindung:** Nutzung mehrerer paralleler Verbindungen hat negative Effekte auf Transportschicht
- **Fehlende Priorisierung:** Antwort-Sequenz folgt strikt der Request-Sequenz, laden von weniger wichtigen Ressourcen verzögert laden von wichtigeren Ressourcen

- **Redundanz** durch ASCII-codierte Header: Redundanz von langen Feldern im Header

3.6.2 HTTP/2

- **Message**: vollständige HTTP-Nachricht
- **Stream**: bidirektionale Verbindung
- **Frame**: kleinste Einheit für Nachrichtendaten
- **Stream-basierte Kommunikation**
 - Logische Streams je Request
 - Parallele Übertragung der Streams über einzige TCP-Verbindung
 - Priorisierung einzelner Frames
- **Flusskontrolle** auf Anwendungsebene
- **Server Push**: Server sendet bisher nicht angefragte Ressourcen über bestehende Verbindung

Header Compression (HPACK) Statische und dynamische Komprimierung von Headerzeilen

- **Statisches Wörterbuch** für vorab bekannte Header Feldnamen
- **Dynamisches Wörterbuch** begrenzte Liste von tatsächlich verwendeten Headern in Verbindung
- **Huffman-Kodierung** (client- und serverseitig) für sonstige Felder und Werte

HTTP/2: Header Compression (HPACK)

Redundanz bei HTTP/1.1

- lange Header-Feldnamen
- wiederholende Informationen bei konsekutiven Requests/Responses

Neues Syntax-Format bei HTTP/2

- Einschränkung der Redundanz der HTTP-Nachrichten (hier: HEADER Frames)
- statische und dynamische Komprimierung von Headerzeilen
 - statisches Wörterbuch: z.B. für vorab bekannte Header-Feldnamen
 - dynamisches Wörterbuch: begrenzte Liste von tatsächlich verwendeten Headern in Verbindung
 - Verwendung einer Huffman-Kodierung (client- und serverseitig) für sonstige Felder und Werte

:method	GET
:scheme	HTTP
:author	uni-ulm.de
:path	/in/vs
user-agent	Mozilla/5.0
x-custom	value

Request Headers

Static Table

1	:authority	
2	method	GET
...	...	...
51	Referer	
...	...	...
62	user-agent	Mozilla/5.0
63	:host	uni-ulm.de
...	...	...

Dynamic Table

2	
7	
63	
19	compress("/in/vs")
62	
	compress("x-custom")
	compress("value")

Encoded Headers

Abbildung 23: HTTP/2 Header Compression

Probleme bei HTTP/2

- **Head-of-Line-Blocking** von TCP-Segmenten statt HTTP-Requests
- **HTTP/2 in Kombination mit TLS**: wie bei HTTP/1.1 sind mehrere Handshakes notwendig (TCP-/ & TLS-Handshake)

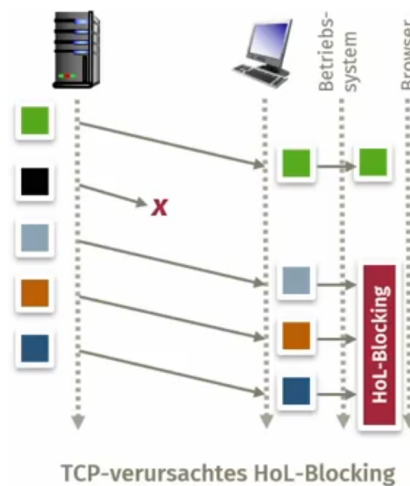


Abbildung 24: HTTP/2 HoL-Blocking

3.6.3 HTTP/3 & QUIC

- Auf UDP aufbauendes Transportschichtprotokoll als Teil der Anwendungsschicht

- **Stream-Multiplexing** mit Flusskontrolle und Überlastkontrolle
- Sicherheit durch **TLS 1.3**
- Einzelner QUIC-Handshake
- **HTTP/3** baut auf HTTP/2 auf, verwendet jedoch QUIC mit verpflichtendem TLS

3.6.4 Entwicklung des HTTP-Protokoll-Stacks

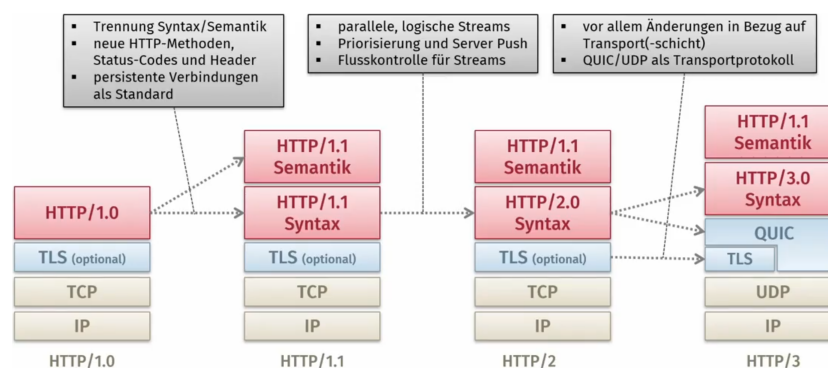


Abbildung 25: HTTP Entwicklung

4 D Netzwerkschicht

4.1 Aufgaben

- Transport von Segmenten vom Sender zum Empfänger
- **Sender:** Einkapseln von Daten in **Datagramme**
- **Empfänger:** Weiterleitung der empfangenen Segmente an Transportschicht
- Netzwerkschichtprotokolle in jedem Router und Host
- Router parsen Netzwerkheader aller IP Datagramme

4.1.1 Forwarding und Routing

Zwei wesentliche Aufgaben

- **Forwarding:** Weiterleitung von Paketen vom Eingangsport zum richtigen Ausgangsport des Routers
- **Routing:** Bestimmung der korrekten Route eines Pakets von der Quelle zum Ziel (in jedem Router: "next hop")

4.1.2 Data Plane und Control Plane

- **Data Plane:** lokale Funktionalität im Router
 - Weiterleitung eines Datagramms von Eingang zu Ausgang
 - Forwarding
- **Control Plane:** netzwerkweite Funktion
 - Weiterleitung eines Datagramms durch Router auf einem Ende-zu-Ende Pfad
 - herkömmliches Routing in Routern
 - **Software-defined Networking (SDN):** implementiert in (zentralen Servern)

4.1.3 Control Planes

- **Per-Router Control Plane:** Interaktion individueller Routing-Algorithmen in jedem einzelnen Router
- **Logisch zentralisierte Control Plane** (z.B. SDN): dedizierter Controller interagiert mit lokalen Kontrollagenten (CAs)

4.1.4 Datagramm-basierte Netzwerke und Virtual Circuits

Internet (Datagramm-basiert)

- Kein Verbindungsaufbau auf Netzwerkschichtebene, zustandslos für Router
- Weiterleitung von Paketen auf Basis von Zieladressen
- Einfach und leistungsfähig, Komplexität im "Edge"

ATM (virtual circuit)

- Entwicklung durch Telefonie
- Hohe Anforderungen an Timing und Zuverlässigkeit, garantierte Dienstgüte

4.1.5 Service-Modelle

Spezifikation des "Kanals" zwischen Sender und Empfänger auf Netzwerkschichtebene

Individuelle Datagramme

- garantierte Zustellung
- garantierte Zustellung mit weniger als 40ms Verzögerung

"Flow" von Datagrammen

- geordnete Zustellung von Datagrammen
- garantierter minimale Datenrate pro Fluss
- Beschränkung des **Jitters** (Varianz des Abstands zwischen Paketen)

Netzwerk-Architektur	Service-Modell	Garantien				Überlast-Information
		Datenrate	Zuverlässigkeit	Reihenfolge	Latenz	
Internet	best effort	✗	✗	✗	✗	✗ (Ableitung aus Verlust)
ATM	CBR <i>constant bit rate</i>	konstante Rate	✓	✓	✓	keine Congestion
ATM	VBR <i>variable bit rate</i>	garantierte Rate	✓	✓	✓	keine Congestion
ATM	ABR <i>available bit rate</i>	garantierte minimale Rate	✗	✓	✗	✓
ATM	UBR <i>unspecified bit rate</i>	✗	✗	✓	✗	✗

Abbildung 26: Service-Modelle der Netzwerkschicht

4.2 Router

4.2.1 Router-Architektur

Wesentliche Funktionen

- Routing-Algorithmus ausführen (RIP, OSPF, BGP)
- Weiterleitung von Datagrammen von Eingangs- auf Ausgangsport
- Aufteilung in **Control Plane** und **Data Plane**

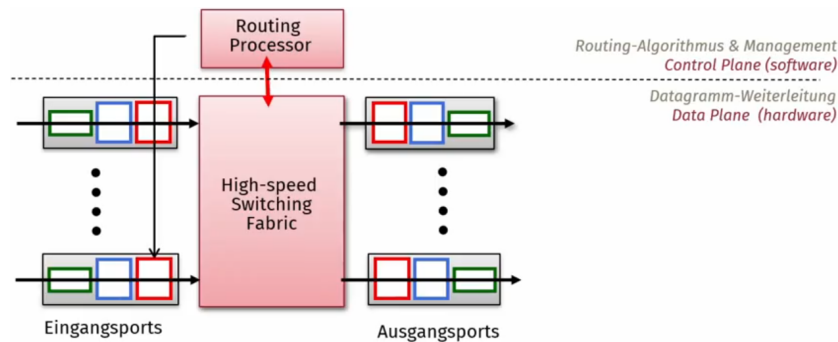


Abbildung 27: Router Architektur

4.2.2 Eingangsports

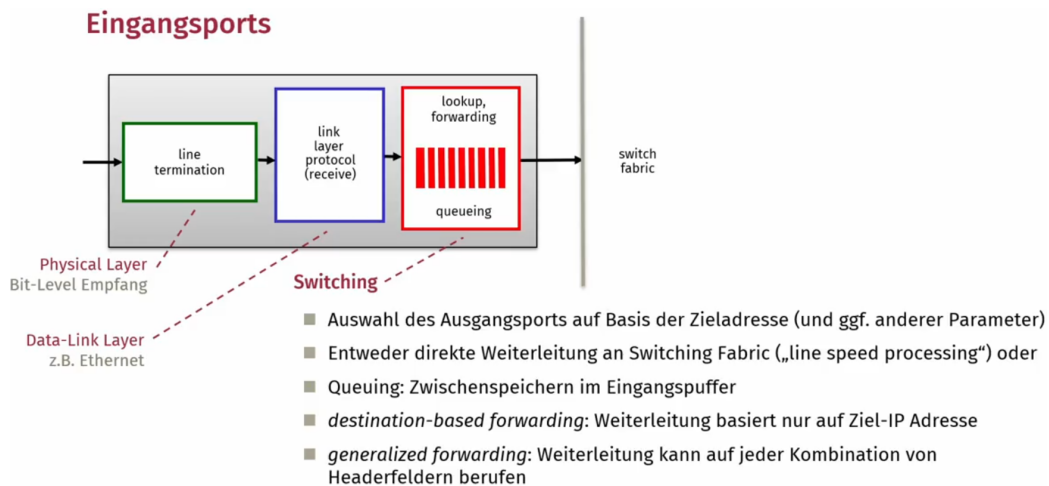


Abbildung 28: Eingangsports

4.2.3 Routing-Tabellen

Verwendung aggregierter Einträge für Adressbereiche

Destination-based Forwarding

- Weiterleitung basiert auf Aufteilung in verschiedene Adressbereiche
- **Problem:** Glatte Aufteilung schwer umsetzbar

Longest Prefix Matching Durchsuchen der Routintabelle nach längstem Adresspräfix, der mit Zieladresse übereinstimmt

4.2.4 Switching Fabrics

- **Switching Rate:** Rate mit der Pakete vom Eingang zum Ausgang weitergeleitet werden können, bei N Eingängen mit Rate R : Switching Rate $N \cdot R$ non-blocking
- Generelle Typen: Speicherbasiert, Bus-basiert, Interconnection Network (z.B. Crossbar Switch)

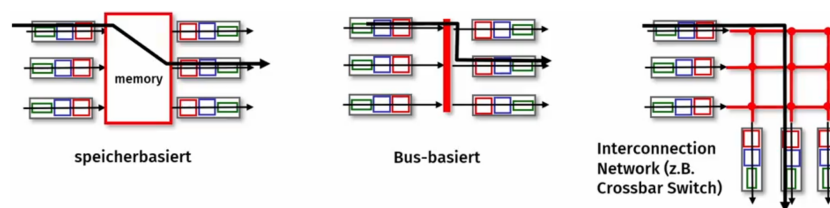


Abbildung 29: Arten von Switching Fabrics

4.2.5 Queueing an Router-Ports

- **Input port queueing:** Switching Fabric langsamer als Summe aller Input-Ports
- **Output port queueing:** Datenrate vom Switching Fabric an bestimmten Zielport zu hoch
 - Puffern wenn Paketankunftsrate die Geschwindigkeit des Ausgangs-ports übersteigt
 - Queueing (Verzögerung) und Verlust bei erschöpftem Ausgangspuffer
 - **Priority Queueing:** Pakete können andere in der Queue überholen

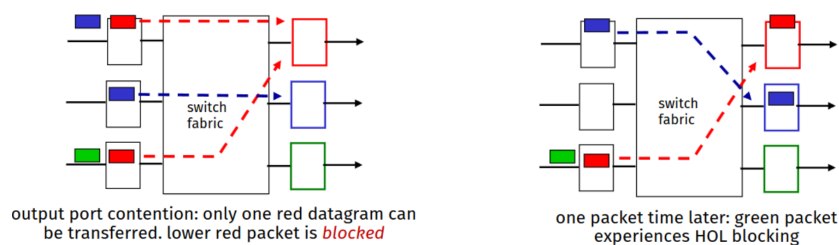


Abbildung 30: Port queueing in switch fabric

4.2.6 Output ports

- Pufferung notwendig, wenn Datagramme schneller ankommen als Datenrate des Ausgangsports eine Weiterleitung zulässt
- Datagramme können potenziell immer aufgrund von Überlast oder fehlendem Puffer verloren gehen!
- **Scheduling-Strategie** bestimmt, welche Pakete verschickt bzw. gedroppt werden

4.2.7 Pufferspeicher

- **RFC 3439** (Daumenregel): Pufferspeicher = "typischer" RTT·Linkkapazität C
- Aktuelle Empfehlung: $\frac{RTT \cdot C}{\sqrt{N}}$, wobei N = Anzahl flows

4.2.8 Scheduling-Strategien

Wahl des nächsten zu sendenden Pakets, implizit auch Entscheidung über zu verwerfende Pakete

First-Come First-Serve (FCFS) / First-In First-Out (FIFO) Reihenfolge der Ankunft = Sendereihenfolge

Priority Scheduling

- Klassifizierung der eingehenden Flows: verschiedene Queues für erkannte Klassen, Klassifizierung durch Header
- Sendeprozess: wähle Queue mit höchster Priorität mit ausstehenden Paketen zum Senden

Round Robin (RR)

- Klassifizierung der eingehenden Flows in separate Queues
- Sendeprozess: rundenweise Iteration über alle Queues

Weighted Fair Queueing (WFQ)

- generalisierte Variante von RR
- Gewichtung von Klassen (i) bzw. Queues (w_i)
- Sendeanteil je Klasse und Runde: $\frac{w_i}{\sum_i w_i}$
- ermöglicht minimale garantierte Bandbreite je Klasse

4.3 Internet Protokoll IPv4

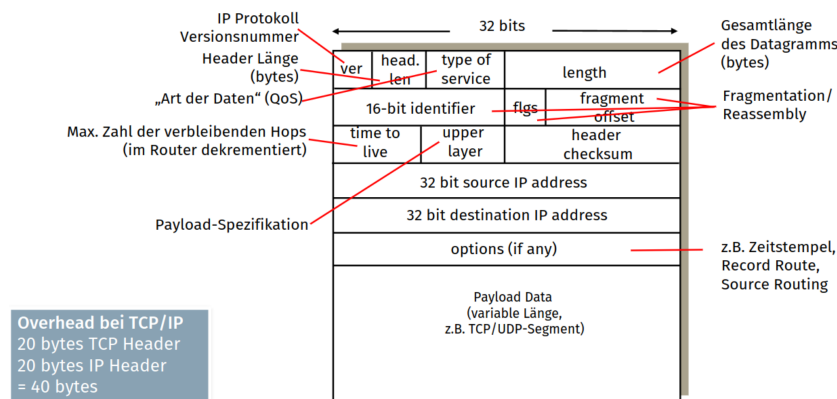


Abbildung 31: IPv4 Datagram format

4.3.1 Fragmentierung und Reassemblierung

- Netzwerk-Links besitzen **Maximum Transfer Unit (MTU)** - größtmöglicher Link-layer Frame
- unterschiedliche Netze mit unterschiedlichen MTUs
- Datagramm muss ggf. im Netz aufgeteilt (fragmentiert) werden
- **Reassemblierung** (wieder zusammensetzen) beim Empfänger
- Felder im IP-Header identifizieren Fragmentierung und Ordnung der Fragmente

4.3.2 Adressierung

Definition 4.1: IPv4-Adressen

32-bit Identifier für ein Host/Router-Interface

Interface

- Verbindung eines Hosts/Routers mit einem physikalischen Link
- Router typischerweise mehrere Interfaces, Hosts ein oder wenige (z.B. Ethernet & WLAN)
- IP-Adressen mit Interface assoziiert
- Ethernet-Interfaces via Switch
- Drahtlose WiFi-Interfaces via WiFi-Access Point

4.3.3 Subnetze

- IPv4 Adresse Subnetz Teil: high order bits, Host Teil: low order Bits
- Aufteilung angegeben durch
 - Netzmaske (z.B. 255.255.255.0)
 - Subnetz Präfix (z.B. 223.1.1.0/24)
- Geräteinterfaces mit gleichem Subnetz-Teil in der IP-Adresse
- Direkter Paketversand ("direct delivery") ohne Beteiligung von Routern

Früher: Einteilung in Klassen

- **Class A:** /8 (0-127)
- **Class B:** /16 (128.1-191.255)
- **Class C:** /24 (192.0.0-223.255.255)

4.3.4 Classless InterDomain Routing (CIDR)

- erlaubt Subnetze mit Netzwerkteil beliebiger Länge
- Adressformat: a.b.c.d/x, wobei x = Anzahl der Bits im Subnetzteil

4.3.5 Dynamic Host Configuration Protocol (DHCP)

- Feste IP-Adresse durch Systemeinstellungen (Windows) bzw. /etc/netplan (Linux)
- DHCP: dynamische Zuweisung von IP-Adressen an Interfaces eines Hosts durch DHCP-Server zum Verbindungszeitpunkt
- Lease-Konzept: nur temporäre Zuweisung mit "Verfallsdatum"
 - Verlängerung / Erneuerung der Lease
 - Wiederverwendung von Adressen für andere Hosts

Ablauf

- **DHCP discover** Nachricht auf Broadcast durch Host
- **DHCP offer** Nachricht von DHCP-Server
- **DHCP request**: Host fragt IP Adresse an
- **DHCP ack**: Server bestätigt Adresse

Weitere Konfigurationsoptionen

- Adresse des Default Routers
- Name und IP-Adresse von DNS-Servern
- Netzwerkmaske/Präfix
- Zeitserver

4.3.6 Hierarchische Adressierung

Route Aggregation effiziente Annoncierung von Adressblöcken im Internet

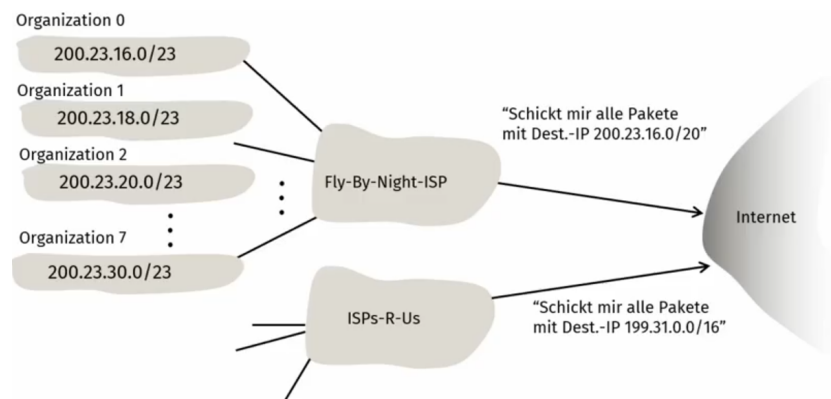


Abbildung 32: Route Aggregation

- Organisation 1 zieht zu anderem ISP (behält jedoch IP-Adressbereich)
- neuer ISP advertised nun spezifischere Route zu Organisation 1

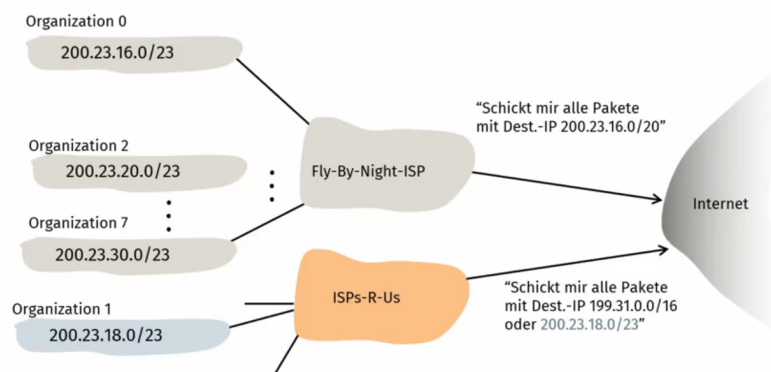


Abbildung 33: Spezifischere Routen

Spezifischere Routen

4.3.7 Adressblöcke

- Internet Corporation for Assigned Names and Numbers (ICANN) weist adressen zu
- Verwaltung von DNS-Einträgen
- weist Domainnamen zu und klärt Streitfälle

4.3.8 Network Address Translation (NAT)

- Alle Datagramme, die das lokale Netzwerk verlassen, haben gleiche Absende-IP-Adresse, jedoch verschiedene Source (TCP/UDP) Ports
- Datagramme im lokalen Netz Adressen als Ziel/Absender
- vom ISP kann (dynamisch) eine einzelne IP-Adresse zugewiesen werden
- Geräte im internen Netz sind nicht direkt von außen adressierbar (Security)
- **NAT-Router** ändert Datagramm Adressen bei Kommunikation nach außen
- Limitiert durch Anzahl an Portnummern

Probleme

- Router sollten nur Header der Netzwerkschicht bearbeiten
- Veränderung des Ende-zu-Ende-Verhalten
- NAT Traversal Problem
- Fazit: keine nachhaltige Lösung für Knappheit von IPv4-Adressen

4.3.9 NAT Traversal Problem

- Server-Adresse nur lokal im LAN erreichbar, kann vom Client nicht als Destination Adresse gesetzt werden
- **Lösungen**
 - **Port-Forwarding**: statische Konfiguration im NAT-Router
 - **Universal Plug and Play (UPnP)**: Internet Gateway Device (IDG) Protocol ermöglicht Hosts hinter NAT Gateway
 - * ihre öffentliche IP Adresse zu erfahren
 - * Port Mapping hinzuzufügen oder zu entfernen (mit lease times)
 - * Sicherheitsrisiko
 - **Relaying**
 - * NATed Client erstellt Verbindung zu Relay Server
 - * externe Clients verbinden sich ebenfalls zum Relay
 - * Relay bridged Pakete zwischen Verbindungen

4.3.10 Internet Control Message Protocol (ICMP)

- Kommunikation von Netzwerkschicht-Informationen zwischen Hosts & Routern
- Fehlermeldungen & Debugging

Traceroute

- Quelle sendet Serie von UDP-Datagrammen an Ziel
 - erster 3er Block Datagramm mit TTL=1, zweiter mit TTL=2 usw.
 - zufällige Port Nummern (odder UDP echo Port)
- bei Ankunft von Datagramm mit TTL=n am n. Router: Router verwirft Datagramm und sendet ICMP-Nachricht an Quelle, diese enthält IP-Adresse des Routers

4.3.11 Adressknappheit

Relevante Ansätze

- Rückholung unbenutzter IP Adressräumen
- CIDR und flexibles Sub-/ Supernetting: feingranulare Zuweisung von Adressen ohne Überlastung der Routing-Tabellen
- Verwendung von NAT
- kein Aufhalten der Verknappung von IPv4-Adressen, sondern Verzögerung

4.4 Internet Protokoll IPv6

4.4.1 Motivation und Verbreitung

- Adressknappheit von IPv4
- Header mit fixer Länge, keine Fragmentierung in Routern
- bessere Unterstützung von QoS-Verarbeitung
- Viele ISPs verwenden Dual-Stack (IPv4 + IPv6)
- IPv4 häufig nur noch mit carrier-grade NAT

4.4.2 Header

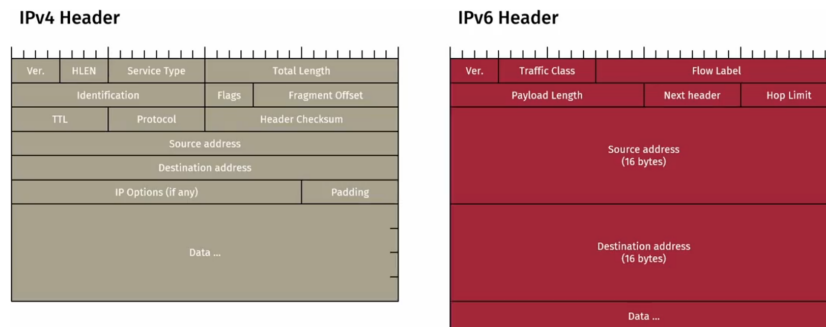


Abbildung 34: IPv4 vs. IPv6 Header

Format von IPv6

- **Traffic Class:** Typ des Verkehrs (QoS), auch für ECN verwendet
- **Flow Label:** identifiziert Datagramme im selben Flow (Sender)
- **Next Header:** identifiziert weitere Header oder Protokoll des Payloads
- kein Checksummenfeld oder Options-Feld
- keine Fragmentierung in Routern, stattdessen Ende-zu-Ende
- Anpassung des ICMP Protokolls: **ICMPv6** für Nachrichten wie "Packet too big"
- Neighbor Discovery Protocol ersetzt ARP

Vorteile

- Größerer Adressbereich
- Vereinfachtes Header Format erleichtert vereinfachte Verarbeitung in Routern
- flexible Integration von Optionen durch Extension Headers
- verpflichtende Unterstützung von IPSEC zur Absicherung der Netzwerkschicht
- bessere QoS-Unterstützung: Traffic Class & Flow Label erleichtern Klassifikation von Datagrammen

4.4.3 Adressformat

- 128 bit: $8 \cdot 16$ bit Worte
- **Kurzschreibweise**
 - Weglassen führender Nullen
 - Abkürzung eines Nullblock zu `::`
- **Adresstypen**
 - Unicast (1:1)
 - Multicast (1:n)
 - Anycast (1:[1 aus N])
 - Kein Broadcast

Adress Type	Binary Prefix	IPv6 Notation
Unspecified	00..0 (128 bit)	::/128
Loopback	00..1 (128 bit)	::1/128
Multicast	11111111	FF00::/8
Link-local unicast	1111111010	FE80::/10
Global unicast	everything else	

Adress Space Layout

Unicast-Adressen

- Variable Unterteilung in Subnet Prefix und Interface ID
- Global route-bare Adressen: keine Feste Struktur, generelle Form: globaler Routing Prefix + Subnet ID + Interface ID
- **IPv4 Mapping**
 - `::124.60.77.1` zwei IPv6 Geräte kommunizieren über IPv4 Netz
 - `::ffff:124.60.77.1` IPv6 Geräte wollen sich mit IPv4 Gerät verbinden
- **Link-local Unicast** `fe80::/64` + 64 bit Interface ID
- **Unique-local Unicast** `fc00::/7` + 40 bit Site ID + 16 bit Subnet + 64 bit Interface ID
- Anycast: kein Unterschied zu Unicast



Abbildung 35: IPv6 Multicast

Multicast

- **FLG**: flags für Multicast Routing
- **SCP**: Scope (interface, link, admin, site, organization, global)
- **Group ID**: kann Unicast-Netzwerkpräfix beinhalten

4.4.4 Transition von IPv4 zu IPv6

- Umstellung an fest definiertem Stichtag (flag day) nicht praktikabel, stattdessen gemischter Betrieb zwischen IPv4 und IPv6
- **Dual-stack Implementierung**: hybride Implementierung, Repräsentation von IPv4-Endpunkten durch mapped Address
- **Tunneling**: IPv6 Datagramm als IPv4 Payload

4.5 Routing Algorithmen

4.5.1 Abstraktion

- Darstellung eines Netzwerks als Graph $G = (N, E)$ mit N Routern und E Links
- **Kantengewichte**
 - Direkte Kosten für die Verbindung von a nach b , bei Unerreichbarkeit ∞
 - Implizit: Länge eines Links
 - Explizit: Kosten für Bandbreite
- Ziel: finde Pfad zwischen Start und Ende mit geringsten Kosten

4.5.2 Klassifikation

Global vs lokal/dezentral

- **Global:** alle Router kennen komplette Topologie inkl. Link-Kosten (Link state Algorithmen)
- **Lokal/Dezentral:** Router sehen nur direkt verbundene Nachbarn und deren Link-Kosten (Distance vector Algorithmen)

Statisch vs. dynamisch

- **Statisch:** Routen ändern sich nur sehr langsam und werden manuell verwaltet
- **Dynamisch:** Routen ändern sich häufig, schnelle Reaktion erforderlich → periodische Anpassungen als Reaktion auf Änderung der Link-Kosten

4.5.3 Dijkstra's Link-State Routing-Algorithmus

- "Link State Broadcast: Fluten der link state Informationen im Netz
- Berechnet "least cost paths" von source zu anderen Knoten im Netz → Routing-Tabelle
- Iterativer Algorithmus mit $\mathcal{O}(n^2)$ Laufzeit, bzw. $\mathcal{O}(n * \log(n))$ bei effizienter Implementierung
- Gefahr von oszillierenden Effekten bei dynamischen Kosten

4.5.4 Distance-Vector-Routing

- basiert auf **Bellman-Ford-Gleichung:** $D_x(y) = \min\{c(x, v) + D_v(y)\}$
- **Verteilt**
 - jeder Knoten berechnet eigene lokale Routing-Tabelle
 - bei Update der eigenen Tabelle: Benachrichtigung der Nachbarn über Updates
 - Distanz-Vektoren (Liste an Ziel-Knoten und derzeitige Kosten)
- **Asynchron & iterativ**
 - eingehende Update-Nachrichten von Nachbarn stoßen nächste lokale Berechnung an

- Konvergenz der geschätzten Distanz-Vektoren zu tatsächlichen least-cost Pfaden

4.6 Routing im Internet

Hierarchisches Routing

- Zusammenfassung von Routern zu Regionen (Autonomous Systems)
- **Intra-AS Routing**: Router im selben AS verwenden das gleiche (Intra-AS) Routing-Protokoll
- **Inter-AS Routing**: Routing zwischen ASs, **Border-Router** am Rand eines AS hat Link mit Routern in anderem AS

Vernetzung von AS Routing Tabelle wird sowohl vom Intra- und Inter-AS Routing-Algorithmus gefüllt

- Intra-AS setzt Einträge für AS-interne Ziele
- Inter-AS & intra-AS setzen Einträge für AS-externe Ziele zusammen

Aufgaben von Gateway-Routern

- müssen lernen, welche Ziele durch benachbarte AS erreichbar sind (Inter-AS Routing)
- diese Information an andere AS weitergeben (Inter-AS Routing)
- diese Information an alle Router im zugehörigen AS weitergeben (Intra-AS Routing)

4.6.1 Intra-AS Routing

Gängige Protokolle

- auch als "interior gateway protocol" (IGP) bezeichnet
- **RIP**: Routing Information Protocol (veraltet)
- **OSPF**: Open Shortest Path First
- **IGRP**: Interior Gateway Routing Protocol (Cisco; proprietär)

Open Shortest Path First (OSPF)

- **Link-State Algorithmus:** LS Pakete fluten Link State Informationen
- Routenberechnung über Dijkstra-Algorithmus
- Advertisements werden im gesamten AS direkt in IP (nicht TCP/UDP) geflutet
- IS-IS-Routing Protokoll: ISO-standardisiert

Fortgeschrittene Funktionen von OSPF

- **Security:** alle Nachrichten werden authentisiert
- Uni- und Multicast Unterstützung
- Hierarchisches OSPF für große Netzwerke

Hierarchisches OSPF

- Zwei Ebenen: Local Area & Backbone
- Begrenzung der Flutung von link-state Advertisements auf jeweiligen Areas
- Knoten kennen exakte Topologie der eigenen Area sowie Richtung für andere Ziele

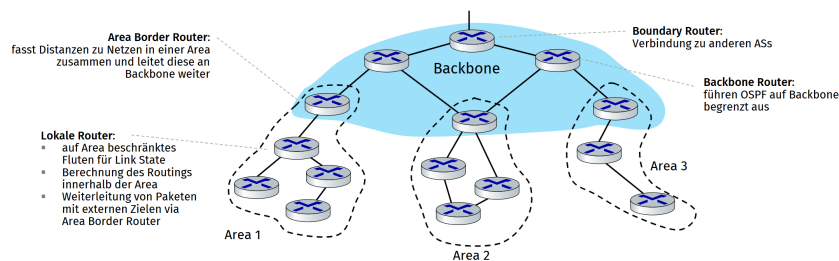


Abbildung 36: Hierarchisches OSPF

4.6.2 Inter-AS Routing

Border Gateway Protocol (BGP) eBGP

- Informationen über Erreichbarkeit von Netzwerk-Präfixen aus benachbarten AS
- Annoncieren von Subnetz-Präfixen an benachbarte AS

iBGP

- Propagierung von BGP-Erreichbarkeitsinformationen an andere BGP-Router im eigenen AS
- OSPF o.Ä. für internes Routing im AS
- Bestimmung der von Routen zu anderen Netzwerken auf Basis der Erreichbarkeitsinformationen und Policies

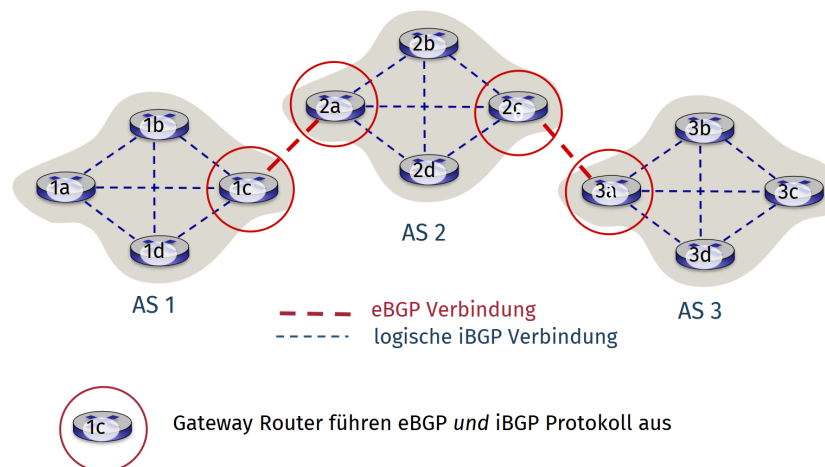


Abbildung 37: eBGP und iBGP Verbindungen

BGP-Session

- zwei BGP-Router ("peers") tauschen BGP-Nachrichten über semi-permanente TCP Verbindungen aus
- Advertisement von Pfaden zu Netzwerkpräfix ("path vectors")

BGP-Routen

- Route = Prefix + Attributes
- **Prefix:** angekündigtes Ziel
- **Attributes:**
 - **AS-PATH:** Pfad beinhaltet alle AS durch die das prefix advertisement gelaufen ist
 - **NEXT-HOP:** bezeichnet den internal-AS Gateway Router zum next-hop AS
- Gateway-Router, welcher Route Advertisement erhält, bestimmt mit Import Policy, ob Route akzeptiert wird

BGP-Routenwahl

- Router empfangen evtl. mehr als eine Route zum gleichen Destination AS
- Routenauswahl basiert auf:
 1. Local preference value Attribut: Policy-basierte Entscheidung
 2. kürzester AS-PATH
 3. nächster NEXT-HOP Router: hot potato routing
 4. weiteren Kriterien
- Bei Bekanntwerden eines neuen Prefix: Router tragen diese in Routing Tabellen ein und verbreiten diese im AS via Intra-AS Routing Protocol

Zusammenfassung

- **Policies**
 - **Inter-AS:** Administration will kontrollieren, wie Traffic geroutet wird und wer durch das eigene Netzwerk routet
 - **Intra-AS:** unter Kontrolle einer einzelnen Administration, meist keine Policy-Entscheidungen notwendig, oft nur kürzeste Routen
- **Skalierbarkeit:** hierarchisches Routing reduziert Tabellengrößen, reduziert Datenverkehr für Updates

- **Performance**

- **Intra-AS:** Schwerpunkt auf Effizienz und Performance
- **Inter-AS:** Policy evtl. wichtiger als Performance

4.7 Software Defined Networking

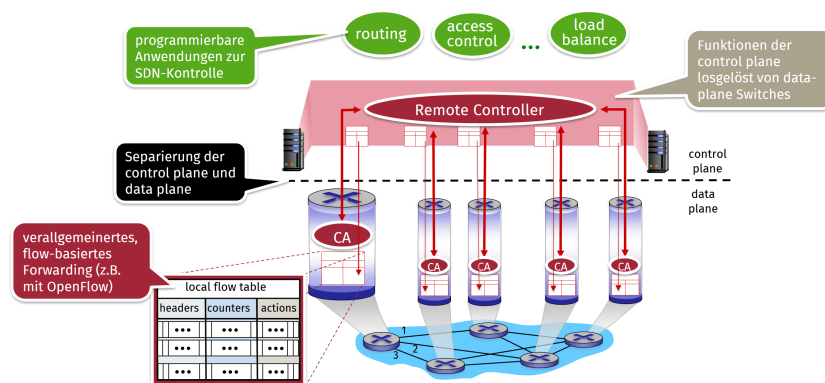


Abbildung 38: Software Defined Networking Überblick

4.7.1 Logisch zentralisierte Control Plane

- Einfacheres Netzwerkmanagement:
 - flexibleres Traffic Engineering
 - einfacheres Debugging von Fehlkonfigurationen
- SDN ermöglicht die Programmierbarkeit von Routern
 - spezifische Netzwerk-Anwendungen für bestimmte Aufgaben/Traffic-Arten
 - Programmierung zentralisierter Anwendungen einfacher als bei verteilten Anwendungen

4.7.2 Forwarding

OpenFlow Data Plane

- Pro Router/Switch: Flow-Tabelle mit Weiterleitungsverhalten
 - Flow: zusammengehörige Datenpakete (durch Headerfelder definiert)

- Zuweisung der Flow-Tabelle durch zentralen Controller
- **Generalized Forwarding:** einfache Regeln zur Paketbehandlung
 - **Pattern:** Muster im Paket Header
 - **Actions:** für zutreffende Pakete (Match)
 - * drop/forward/modify von Paketen
 - * Weiterleitung zum Controller
 - Prioritätsregeln bei mehreren Matches
 - **Counters:** Zähler für #bytes und #packets
 - **Flow Table:** speichert Regeln im Router

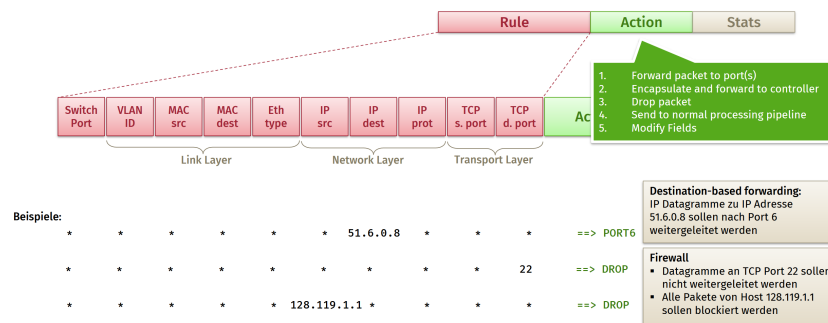


Abbildung 39: Flow Table Einträge bei Openflow

Flow Table Einträge bei OpenFlow

4.7.3 Software Defined Networking (SDN)

Data-plane Switches

- einfache Switches implementieren lediglich generalisiertes data-plane forwarding in hardware
 - kein komplexes OS mehr notwendig
 - Switch flow table wird vom Controller berechnet und an Switches verteilt
 - API / Regelformat definiert Switch-Verhalten (z.B. OpenFlow)
- definiert, was gesteuert werden kann und was nicht
- Protokoll zur Kommunikation zwischen Controller und Switch

Controller

- verwaltet Netzwerkszustand
 - interagiert mit Netzwerkanwendungen (Apps) über Northbound API
 - interagiert mit Switches über Southbound API
 - kann bei Bedarf auch verteilt realisiert werden
- Performance, Skalierbarkeit, Fehlertoleranz, Ausfallsicherheit

Anwendungen

- logisches „Gehirn“ des Netzwerks
- implementieren Kontrollfunktionen via API und Programmiermodell des SDN-Controllers
- SDN-Controller als Ausführungsumgebung

4.7.4 OpenFlow Protokoll

- Protokoll zwischen OpenFlow-fähigem Controller und OpenFlow-fähigen Switches
- TCP / TLS Verbindung zum Nachrichtenaustausch
- OpenFlow Nachrichten
 - **Controller-to-switch**: Konfiguration, Tabellen, packet-out
 - **Switch-to-controller** (asynchronous): packet-in, Status updates
 - **Symmetric** (weitere Operationen)

4.7.5 Herausforderungen bei SDN

Sicherheit und Zuverlässigkeit

- Robustheit bei Fehlern
- Entwicklungsunterstützung und Debugging
- Schutz gegen Angriffe auf Controller oder SDN-fähige Geräte

Performance und Skalierbarkeit

- Realisierung AS-übergreifender SDN-Deployments
- Unterstützung von anwendungsspezifischen und netzspezifischen Anforderungen
- SDN als wichtige Komponente in 5G-Netzen

5 E Verbindungsschicht

Definition 5.1: Verbindungsschicht

zuständig für Transport von Datagrammen von einem Knoten zu einem physikalisch benachbarten Knoten über einen Link.

5.1 Dienste der Verbindungsschicht

- **Framing:** Datagramme in Frames einpacken
- **Medienzugriff:** bei geteiltem Medium
- MAC-Adresse zur Adressierung von Sender & Empfänger
- Zuverlässige Zustellung zwischen benachbarten Knoten
- **Flusskontrolle:** Empfänger kontrolliert Sendegeschwindigkeit
- **Fehlererkennung:** Empfänger erkennt Fehler und löst Retransmission aus oder verwirft Frame
- **Fehlerkorrektur:** Empfänger erkennt und korrigiert Fehler ohne Retransmission
- **Half-duplex und Full-duplex**
 - **Half-duplex:** nur ein Knoten kann an einem Punkt-zu-Punkt Link senden
 - **Full-duplex:** beide können gleichzeitig senden

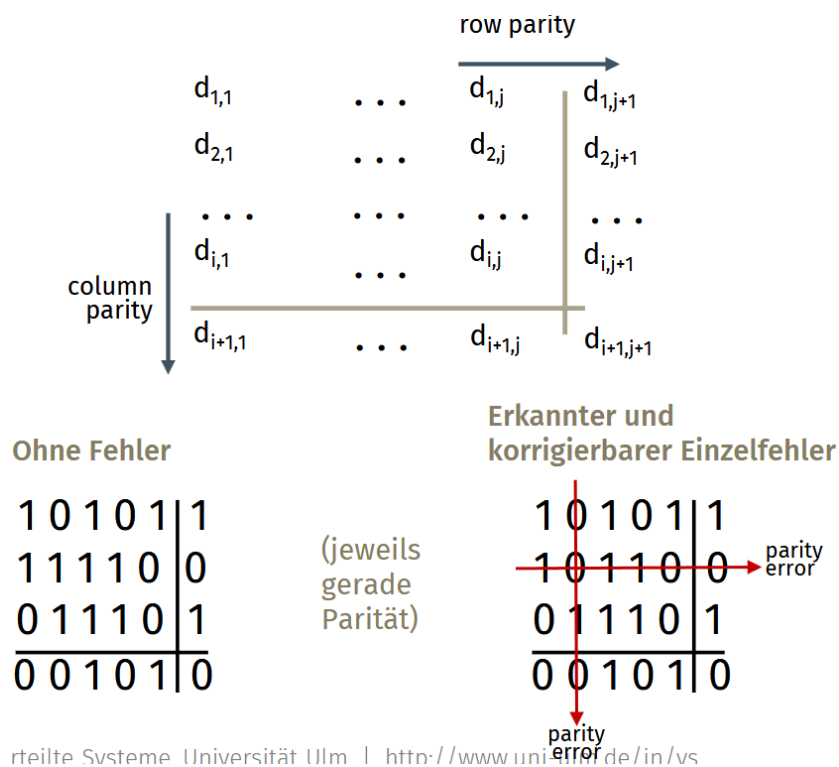
5.2 Technische Realisierung

- in jedem Host
- Umsetzung im Netzwerkadapter
- an Systembus angeschlossen (oft DMA)
- Kombination von Hardware und Software

5.3 Fehlererkennung

5.3.1 Paritätsprüfung (Parity Check)

- **Einzelbitparität:** Erkennung von Einzelbitfehler
- **Zweidimensionale Parität:** Erkennung und Korrektur von Einzelbitfehler



reile Systeme Universität Ulm | <http://www.uni-ulm.de/in/vs>

Abbildung 40: Zweidimensionale Parität

5.3.2 Cyclic Redundancy Check (CRC)

- D: Datenbits (Interpretation als Binärzahl)
- G: vorgegebenes Bit-Muster (Generator) der Länge $r+1$
- Erkennt zuverlässig Bitfehler mit weniger als $r+1$ Bits

Berechnung

- Sequenz an Datenbits (D) als binäres Polynom
- mod(2)-Division von Polynom durch Generatorpolynom G
- Rest R: CRC-Wert

5.4 Protokolle für Mehrfachzugriff

5.4.1 Link-Arten

- **Punkt-zu-Punkt**: full-duplex point-to-point Link zwischen Host und Ethernet-Switch
- **Broadcast** (geteiltes Medium oder Kanal): klassisches Ethernet, WLAN

5.4.2 Medium Access Control (MAC)

- verteilter Algorithmus, der gemeinsamen Zugriff regelt
- bestimmt, wann ein Knoten senden darf
- verhindert Kollisionen oder reduziert die Wahrscheinlichkeit

Wesentliche Hauptklasse

- **Kanalpartitionierung** (channel partitioning): teilt Kanal in kleinere Teile (Zeitslots, Frequenzen, etc.) ein
- **Wahlfreier Zugriff** (random access): Kollisionen verringern und Maßnahmen zur Recovery
- **Abwechseln** (taking turns): Knoten wechseln sich mit Kanal ab

5.4.3 Kanalpartitionierung

Time Division Multiple Access (TDMA)

- Kanalzugriff in zeitlichen Runden
- jede Station erhält in jeder Runde (Rahmen/Frame) Zeitslot fester Länge (Pakettransferzeit)

Frequency Division Multiple Access (FDMA)

- Kanal in Frequenzbänder aufgeteilt
- jede Station erhält festes Frequenzband zugeteilt
- nicht genutzte Zeit in Frequenzband bleibt ungenutzt

Vor- und Nachteile

- Faire und effiziente Nutzung bei vielen Teilnehmern und hoher Last
- Ineffizient bei niedriger Last: bei einem aktiven Sender wird nur $\frac{1}{N}$ der Bandbreite genutzt!

5.4.4 Wahlfreier Zugriff (Random Access)

Random access MAC-Protokoll spezifiziert:

- Wann gesendet werden darf
- Wie Kollisionen erkannt werden
- Wie auf Kollisionen reagiert wird

Slotted Aloha

- Annahmen
 - Alle Frames gleiche Größe
 - Knoten sind (Zeit-) synchronisiert
 - Jeder Sender kann Kollisionen erkennen
- Ablauf
 - Wenn Knoten neue Frames senden will, sende im nächsten Slot
 - Bei Kollision: erneute Übertragung (Retransmission) in jedem folgenden Slot mit Wahrscheinlichkeit p
- **Effizienz:** max. 37%

(Unslotted) Aloha

- einfacher, da keine Synchronisation, da keine Slots
- wenn Frame bereit zum Senden → direkte & sofortige Übertragung
- **Effizienz:** max. 18%

Vor- und Nachteile

- Effizient bei niedriger Last; ein Sender kann Kanal voll auslasten
- Vielzahl konkurrierender Übertragungen: hoher Overhead durch Kollisionen

5.4.5 Carrier Sense Multiple Access (CSMA)

- CSMA: "Carrier Sense"
 - Kanal vor Übertragung abhören, ob dieser frei ist
 - Bei belegtem Kanal (busy) wird die Übertragung verzögert
- CSMA/CD: CSMA mit zusätzlicher Kollisionserkennung
 - Bei Kollision: Abbruch laufender Übertragungen
 - **Problem** bei drahtlosen Netzwerken, da das selbst gesendete Signal stärker ist als das empfangene

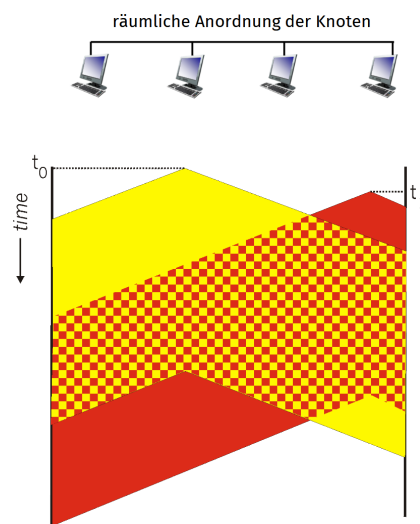


Abbildung 41: Kollisionen bei CSMA

Umgang mit Kollisionen

- Kollisionen können noch immer auftreten aufgrund Ausbreitungsgeschwindigkeit bzw. der unterschiedlichen räumlichen Anordnung der Empfänger
- CSMA/CD reduziert die durch Kollisionen vergeudete Zeit, indem die laufende Übertragung abgebrochen wird

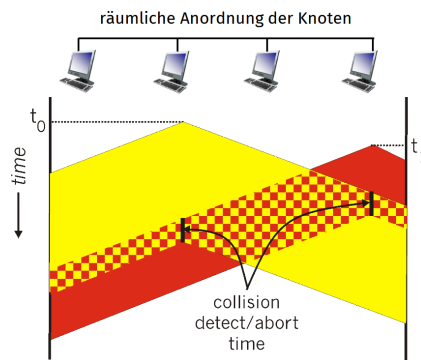


Abbildung 42: Kollisionen bei CSMA/CD

CSMA/CD bei Ethernet

1. NIC empfängt Datagramm von Netzwerkschicht, erzeugt Frame
2. NIC prüft Kanal
3. NIC prüft während Übertragung auf Kollision. Keine Kollision → Übertragung beendet
4. NIC erkennt Kollision: Abbruch der Frameübertragung und Senden eines „Jam“-Signals
5. nach Abbruch führt NIC „Retransmission with binary (exponential) backoff“ durch

Effizienz von CSMA/CD

$$\text{Effizienz} = \frac{1}{1 + 5 \cdot \frac{t_{prop}}{t_{trans}}}$$

5.4.6 Taking-Turns-Mac-Protokolle

Polling

- Leader-Knoten fordert andere zum Senden auf
- Verwendet bei einfachen Geräten
- **Probleme:** Polling-Overhead, Latenz, Single-Point-of-Failure
- **Beispiel:** Bluetooth

Token Passing

- Control Token wird von einem Knoten zum nächsten weitergereicht
- Bei Erhalt des Tokens hat der jeweilige Knoten Senderecht
- **Probleme:** Token Overhead, Latenz, Single-Point-of-Failure (Token)
- **Beispiel:** Token Ring

5.5 LANs

5.5.1 MAC-Adressen

- Auslieferung im lokalen Segment von einem Interface zu anderem Interface im gleichen lokalen Netz (gleiches IP-Subnetz)
- IEEE 48-bit MAC-Adressen fest im NIC ROM eingestellt
- **Beispiel:** 1A-2F-BB-76-09-AD

MAC-Adressraum

- NIC-Hersteller kaufen Adressbereich von IEEE und vergeben daraus Adressen für NICs
- MAC: flacher Adressraum → Portabilität
 - LAN-Karte kann in jedem beliebigen LAN ohne Konfiguration genutzt werden
 - nur lokal erreichbar
- IP: hierarchischer Adressraum → keine Portabilität
 - Adresse hängt vom IP-Subnetz ab, an den Rechner angeschlossen ist
 - weltweite Erreichbarkeit bei rout-barer Adresse

5.5.2 Address Resolution Protocol (ARP)

- **ARP-Tabelle:** von jedem Knoten verwaltet
- Abbildung IP/MAC Adressen für benachbarte Knoten mit TTL (typischerweise 20min)
- **”Plug-and-play”:** kein Management notwendig

Adressauflösungen bei IPv4

- A möchte Datagram an B senden, hat jedoch nicht den ARP-Eintrag
- A schickt ARP-Query mit IPv4-Adresse von B als Broadcast mit Ziel-MAC-Adresse: FF-FF-FF-FF-FF-FF

5.5.3 Neighbor Discovery Protocol bei IPv6

- ersetzt ARP und (teilweise) DHCP
- **Hauptaufgaben**
 - **Address Resolution** (Adressauflösung): Ermittelt die MAC-Adresse eines Nachbarn, wenn nur dessen IPv6-Adresse bekannt ist
 - **Router Discovery:** Geräte finden automatisch Router im Netzwerk
 - **Prefix Discovery:** Router teilen den Geräten mit, welche IP-Präfixe (Subnetze) im lokalen Netzwerk verwendet werden
 - **Parameter Discovery:** Übermittlung von Netzwerkeinstellungen wie der MTU
 - Duplicate Address Detection (DAD): Prüft, ob eine IPv6-Adresse bereits von einem anderen Gerät im Netzwerk verwendet wird, um Konflikte zu vermeiden
- Wichtige ICMPv6 Nachrichten: Router Advertisement/Solicitation, Neighbor Advertisement/Solicitation, Redirect
- **Lokale Datenstrukturen**
 - Neighbor Cache: speichert link local Adressen aller bekannten Nachbarn
 - Destination Cache: speichert next hops für andere Knoten

- Prefix List: speichert alle lokal gültigen Präfixe, die von Routern annonciert werden
- Default Router List: speichert alle bekannten Router

5.5.4 IPv6: Stateless Address Autoconfiguration (SLAAC)

- Automatische Konfiguration von IPv6-Adressen am Netzwerk-Interface
- **Duplicate Address Detection:** Abfrage per Multicast, ob Adresse bereits existiert
- IPv6 Privacy Extensions: zufälligerer Adresse zur Verhinderung der Nachverfolgbarkeit von Hosts
- Alternative zu SLAAC: Stateful Address Autoconfiguration mit DHCPv6

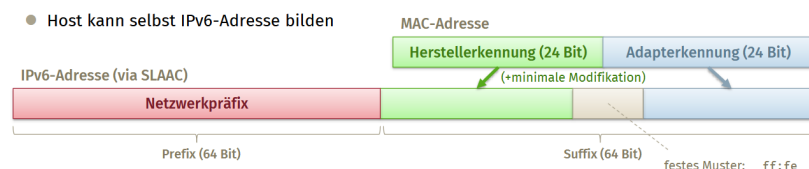


Abbildung 43: Stateless Address Autoconfiguration

5.5.5 Ethernet

Eigenschaften

- **Verbindungslos:** kein Handshake zwischen Sender und Empfänger
- **Unzuverlässig:** Daten können verloren gehen
- **MAC-Protokoll:** Unslotted CSMA/CD mit binary backoff

Physikalische Topologie

- **Bus**
 - populär bis Mitte der 90er
 - alle Knoten in der gleichen Kollisionsdomäne
- **Stern** ("geswitched")
 - heute vorherrschende Topologie
 - aktiver Switch im Zentrum

Ethernet-Frames

- **Preamble**
 - 7 Byte mit Bitmuster 10101010 gefolgt von einem Byte mit Bitmuster 10101011
 - Synchronisation Empfänger Clock & Frequenz
- **Destination & Source MAC-Adressen** (je 48 Bit): wenn NIC einen Frame mit eigener Zieladresse (oder Broadcastadresse) empfängt, wird die Payload an Netzwerkschicht weitergegeben, ansonsten wird Frame verworfen
- **Type**: gibt higher-layer Protokoll an (z.B. IP)
- **CRC**: Cyclic-Redundancy-Check

Ethernet-Switches

- Speichern & Weiterleiten von Ethernet-Frames
- **Transparent**: Hosts sehen Switches nicht (z.B. bei Traceroute)
- **”Plug-and-play”**: selbstlernend, müssen nicht konfiguriert werden
- **Parallele Übertragungen**: möglich, wenn nicht zwei Geräte über den selben Port kommunizieren
- **Forwarding-Tabellen**: ähnlich zu Routern

5.5.6 Spanning Tree Protocol

- **Initialzustand**: Ports leiten keine Daten weiter (*Listening* und *Learning*), um Schleifen vorab zu verhindern.
- **Kommunikation**: Switches tauschen **BPDUs** über die Multicast-Adresse 01:80:C2:00:00:00 aus.
- **Root Bridge Wahl**: Der Switch mit der **niedrigsten Bridge ID** (Priorität + MAC) wird zentraler Bezugspunkt.
- **Pfadberechnung**: Ermittlung der **Least-Cost Pfade** zur Root Bridge basierend auf der Datenrate (Bandbreite) der Links.
- **Port-Blockierung**: Deaktivierung redundanter Pfade (*Blocking State*), während Ports zu Endgeräten (*Hosts*) aktiv bleiben.

- **Wartung:** Regelmäßige BPDUs der Root Bridge prüfen die Topologie; bei Änderungen erfolgt eine **TCN** (Topology Change Notification).
- **Optimierung: RSTP** (802.1w) verbessert die Konvergenzzeit deutlich gegenüber dem klassischen 802.1D.

5.5.7 Virtual Local Area Network (VLAN)

Herausforderungen in großen LANs Probleme einer einzelnen Broadcast-Domäne

- **Skalierungsprobleme:** kompletter Verkehr auf Layer-2 (ARP, DHCP, etc.) läuft über gesamtes LAN
- Sicherheit, Privacy, Effizienz

Administrative Herausforderungen

- Person wechselt in ein Büro einer anderen Abteilung
- Person soll weiterhin im logischen Netz der selben Abteilung sein

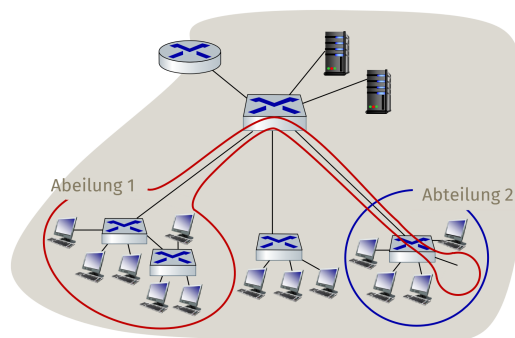


Abbildung 44: LAN Herausforderungen

Lösungsansatz Switches mit VLAN-Unterstützung

- eine physikalische LAN-Umgebung
- mehrere virtuelle logische LAN-Umgebungen

IEEE 802.1Q VLAN Frame-Struktur

- Tag Protocol Identifier (Wert 81-00)
- Ethernet-Frame mit VLAN

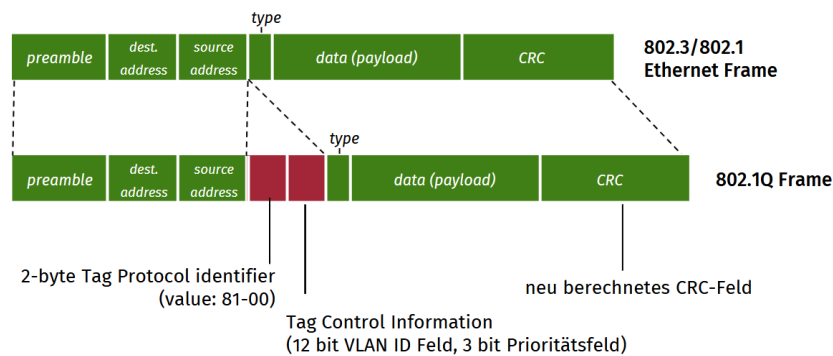


Abbildung 45: VLAN Frame

6 F Physikalische Schicht

6.1 Signale

6.1.1 Periodisches Signal

$$g(t) = A_t \sin(2\pi f_t t + \phi_t)$$

- A_t : Amplitude (Amplitudenmodulation)
- f_t Frequenz (Frequenzmodulation)
- ϕ_t : Phasenverschiebung (Phasenmodulation)

6.1.2 Digitale Signale

- **Schritt(dauer)**: minimales konstantes Zeitintervall zwischen möglichen aufeinanderfolgenden Änderungen des Information-tragenden Signalparameters
- **Schrittgeschwindigkeit**: Kehrwert der Schrittdauer (Einheit: Baud)

Begriffe

- **Binärsignal** bzw. Digitalsignal: Signal mit genau zwei Zuständen
→ Baudrate = Bitrate
- Bei mehrwertigen Signal mehr als zwei unterschiedliche Zustände möglich

- **Quellenkodierung:** Umsetzung eines analogen Signals in eine binäre Zeichenfolge durch zeitgesteuerte **Abtastung** und wertmäßige **Quantisierung**
- **Kanalkodierung:** hinzufügen redundanter Informationen für Fehlererkennung- / korrektur
- **Leitungskodierung:** Anpassung der zu übertragenden Binärwerte an die physikalischen Eigenschaften des Kanals zur bestmöglichen Übertragbarkeit

6.1.3 Digitale Modulation

Begriffe

- **Bandbreite** (Hz): verwendetes Frequenzband eines Kommunikationskanals
- **Baud** (Symbole/s): Anzahl der pro Sekunde gesendeten Symbole
- **Symbol:** Abtastung des Signals; abhängig von den zu übertragenden Daten
- **Datenrate** (Bit/s): $\text{Baud} * \text{Bits/Symbol}$

Amplitudenmodulation / Amplitude-Shift-Keying (ASK)

- Einfach
- Geringer Bandbreitenbedarf
- Störanfällig

Frequenzmodulation / Frequency-Shift-Keying

- Benötigt größere Bandbreite

Phasenmodulation / Phase-Shift-Keying (PSK)

- Komplexe Implementierung
- Robust gegen Interferenzen

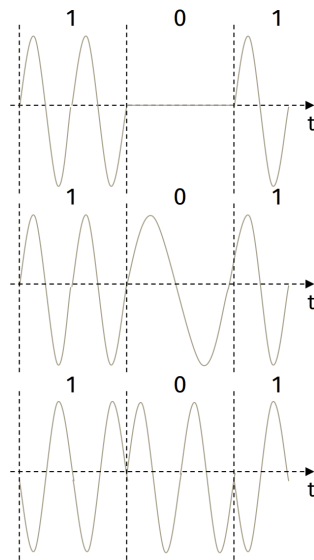


Abbildung 46: Digitale Modulation

6.1.4 Bandbreitenbegrenzung

- Vorschriften
- Eigenschaften der Antenne
- Dämpfung in Kabeln

6.1.5 Nyquist-Rate

$$\text{Max. Datarate} = 2H \log_2 V \frac{\text{Bit}}{s}$$

6.1.6 Shannon-Grenze

$$\text{Max. Datarate} = H \log_2 \left(1 + \frac{S}{N}\right) \frac{\text{Bit}}{s}$$

6.1.7 Signal-Noise-Ration (SNR)

- S: Signalstärke
- N: Geräuschpegel
- $SNR = 10 \log_{10} \frac{S}{N} \text{ dB}$

6.2 Kabelgebundene Zugangsnetze

6.2.1 Asymmetric Digital Subscriber Line (ADSL)

- Nutzung von herkömmlichen Telefonleitungen zur Bereitstellung von Breitband-Internet
- **Asymmetrisch**: Downloadrate ist deutlich höher als Upload-Rate
- Geschwindigkeit abhängig von der Entfernung zum Hauptverteiler (DSLAM)

Orthogonal Frequency Division Multiplex (OFDM)

- Parallele Datenübertragung eines Senders auf mehreren orthogonalen Unterträgern (Subcarrier) mit geringerer Baudrate
- Maximum einer Unterträgerfrequenz liegt genau bei einer Frequenz, bei der alle anderen Unterträger Nulldurchgänge haben

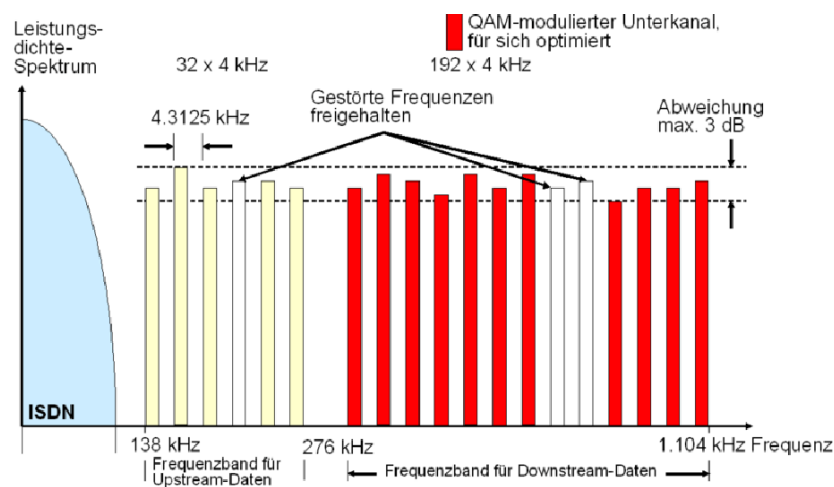


Abbildung 47: OFDM in ADSL

ADSL Standards

- **Annex A**: ADSL über POTS, Internetdaten werden oberhalb des analogen Telefonsignals übertragen
- **Annex B**: ADSL über ISDN, Internetdaten werden überhalb des Frequenzspektrums von ISDN übertragen

- **Annex I/J:** Telefonie ald VoIP (Voice over IP)

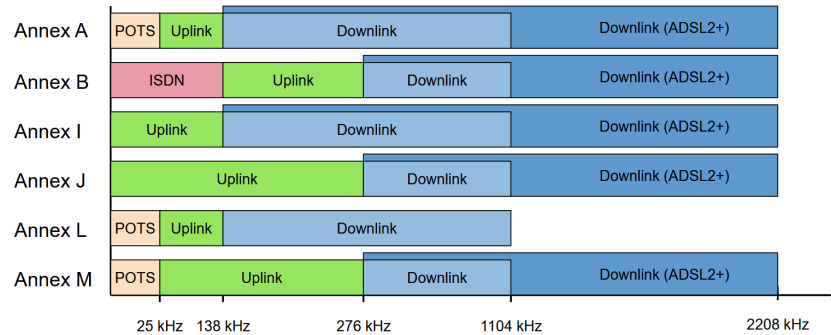


Abbildung 48: ADSL Standards

ADSL2(+)

- **ADSL:** nutzt Shannon-Limit bereits weitgehend aus
- **ADSL2:** verbesserte Modulation und Signalverarbeitung
- **ADSL2+:** höhere Leistung durch Erweiterung des Spektrums

6.2.2 VDSL & VDSL 2

- Erfordert kurze Teilnehmeranschlüsse
- Outdoor-DSLAM mit "Fiber-to-the-Curb" (FTTC)
- Datenrate bis zu 300 Mbit/s

6.2.3 Alternative Broadband Access Technologies

- N: Node (Verteilknoten pro Stadtviertel)
- C: Curb (Verteilschränke am Straßenrand)
- B: Basement (Keller von Mehrparteienhaus)
- H: Home (bis in jede einzelne Wohnung)

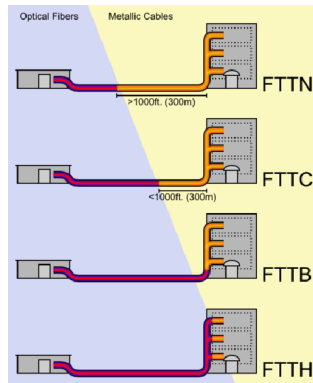


Abbildung 49: Alternative Broadband Access Technologies

6.2.4 Glasfaser

- **Singlemode-Faser:** dünner Kern, sehr hohe Reichweiten
- **Multimode-Faser:** dickerer Kern, Licht breitet sich in verschiedenen Winkeln aus, die an den Kernwänden reflektiert werden, günstiger

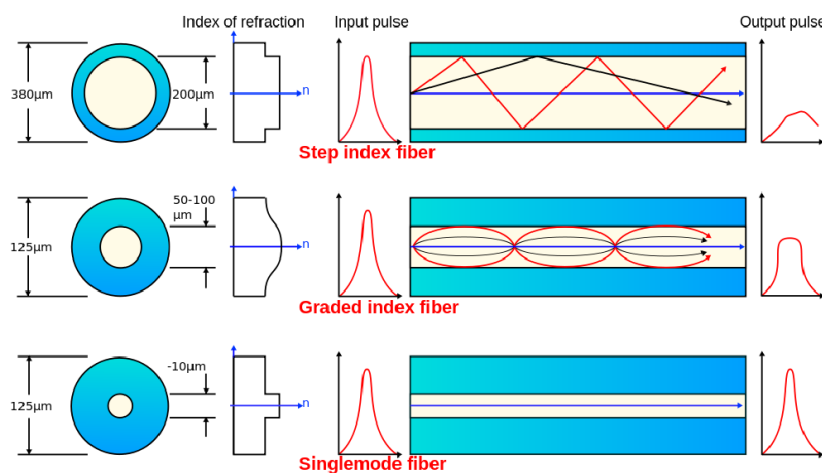
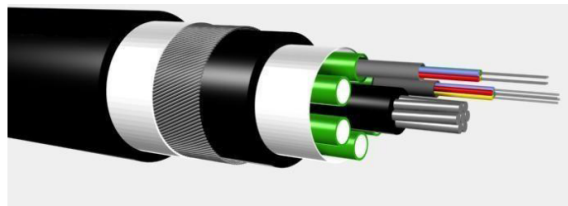


Abbildung 50: Glasfaser

6.2.5 Ethernet

Kodierung bei 100BASE* (Fast Ethernet)

- **4B5B-Kodierung:** ermöglicht Wiederherstellung des Taktes sowie zusätzliche Symbole für die Synchronisierung
- **MLT-3:** weniger elektromagnetische Interferenzen und geringerer Bandbreitenbedarf
 - Wenn 1: Durchlaufen von [0, +, 0, -]
 - Wenn 0: Ebene halten

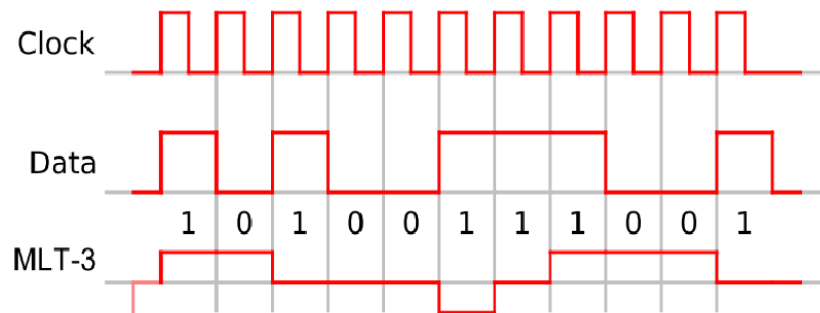


Abbildung 51: MLT-3 Kodierung

6.3 Drahtlose Kommunikation

6.3.1 Bestandteile eines drahtlosen Netz

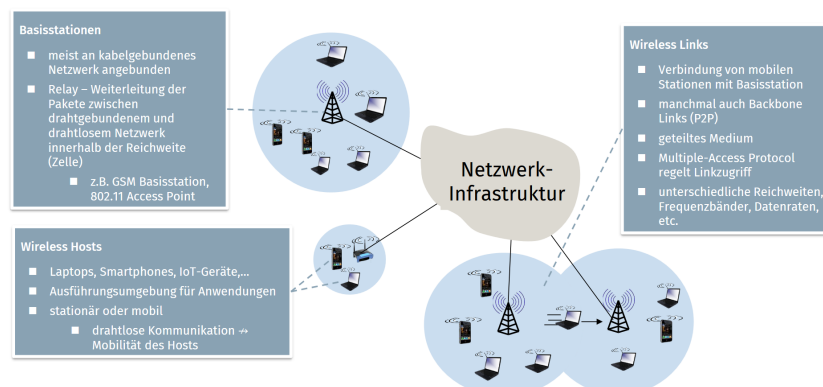


Abbildung 52: Bestandteile eines drahtlosen Netz

Infrastruktur-Netz

- Basisstationen verbinden mobile Stationen mit Netzwerk
- Handoff/Handover/Roaming: mobile Stationen wechseln die Basisstation

Ad-hoc Netz

- Keine Basistationen
- Knoten können innerhalb Funkreichweite direkt miteinander kommunizieren
- Selbstständige Organisation von Knoten in größeren Netzwerken
 - multi-hop ad-hoc Netzwerke
 - Routing

6.3.2 Herausforderungen drahtloser Links

- **Path Loss:** Dämpfung schwächt Signal während Signalausbreitung ab
- Interferenz mit anderen Sendern
- **Multipath Propagation:** Signale werden reflektiert, gestreut, gebeugt usw.
- **SNR** (Signal Noise Ration): bei höherem Wert kann das Signal einfacher aus dem Rauschen herausgefiltert werden
- **Bitfehlerrate** (Bit-Error-Rate): höhere Sendeleistung = besserer SNR = geringere BER
- **Fazit:** Kommunikation über drahtlose Links deutlich umständlicher als in drahtgebundenem Netz

Hidden Terminal Problem

- B, A in Reichweite
- B, C in Reichweite
- A, C nicht in Reichweite: beide Stationen wissen nicht von möglicher Interferenz bei B

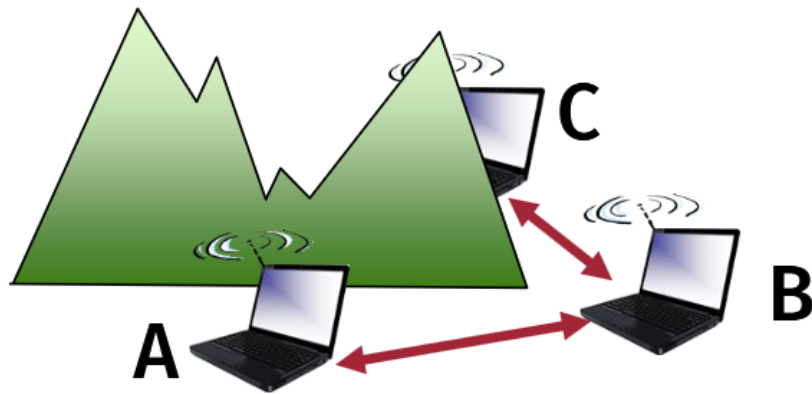


Abbildung 53: Hidden Terminal Problem

Signal-Dämpfung

- B, A in Reichweite
- B, C in Reichweite
- A, C wissen nicht von Interferenz bei B

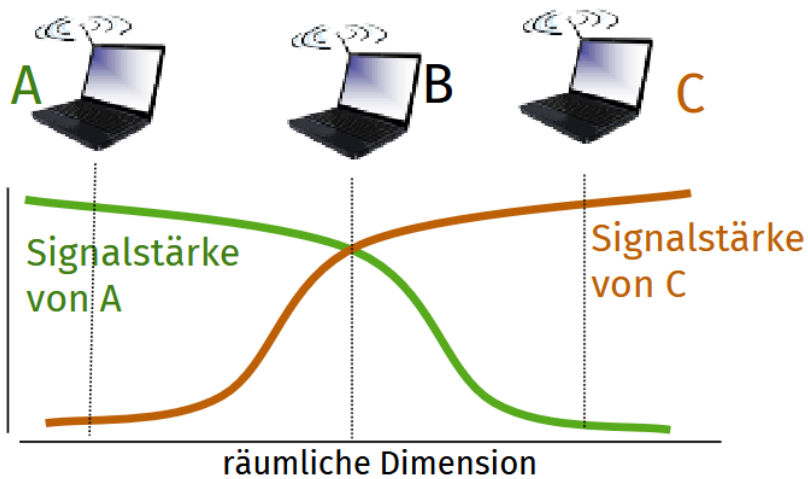


Abbildung 54: Signal Dämpfung

6.4 IEEE 802.11 Wireless LAN (WLAN)

6.4.1 Architektur

- Kommunikation drahtloser Stationen mit Access Point (AP)

- BSS: Basic Service Set (Zelle)
 - Wireless Hosts
 - Access Points: "Basisstationen"
 - Ad-hoc Modus: kein AP, nur Hosts

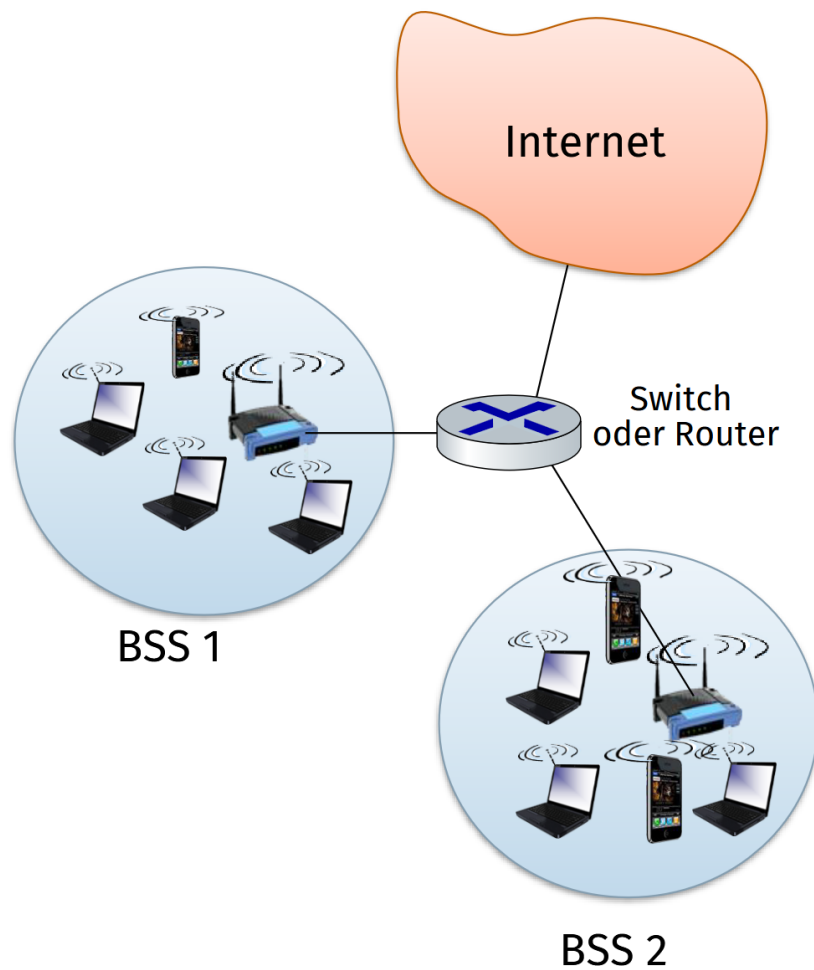


Abbildung 55: WLAN Architektur

6.4.2 Kanäle & Assoziierung

Frequenz-Kanäle

- 11 Frequenz-Kanäle zwischen 2.4-2.485 GHz
- Interferenz möglich, besonders wenn benachbarter AP denselben Kanal verwendet

Assoziierung

- **Assoziierung:** Hosts müssen sich bei AP anmelden
- AP sendet periodisch Beacon-Frame mit AP-Name (SSID) & MAC-Adresse
- Authentisierung (optional)

6.4.3 Mehrfachzugriff

- **CSMA:** vermeidet Kollisionen mit bereits sendenden Knoten in Kommunikationsreichweite
- **Keine Kollisionserkennung** (CSMA/CD): Erkennung von Kollisionen schwierig
- **Kollisionsvermeidung:** CSMA/CA

MAC-Protokoll

- **Sender**
 1. Sende gesamten Frame, wenn Kanal für DIFS Zeiteinheiten un belegt ist
 2. Wenn Kanal belegt ist
 - Starte "random backoff timer"
 - Timer zählt herunter, während Kanal unbelegt
 - Übertragung, sobald Timer auf 0
- **Empfänger**
 - Wenn empfangener Frame Ok ist (CRC Checksumme), schicke ACK nach SIFS Intervall
 - RTS/CTS (Request-to-send/Clear-to-send) lösen Hidden Terminal Problem weiter

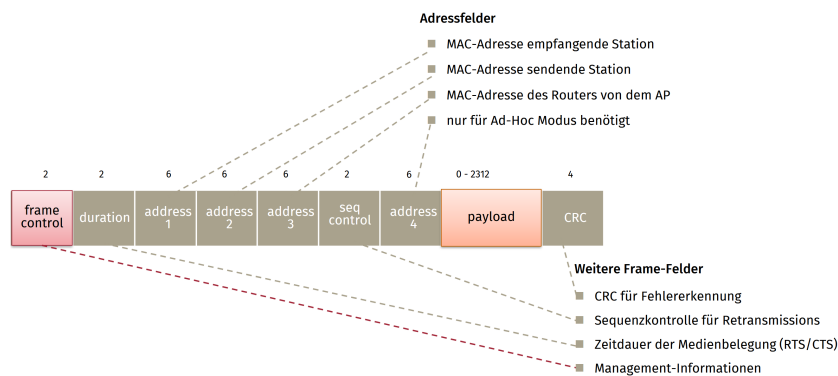


Abbildung 56: WLAN Frame Adressierung

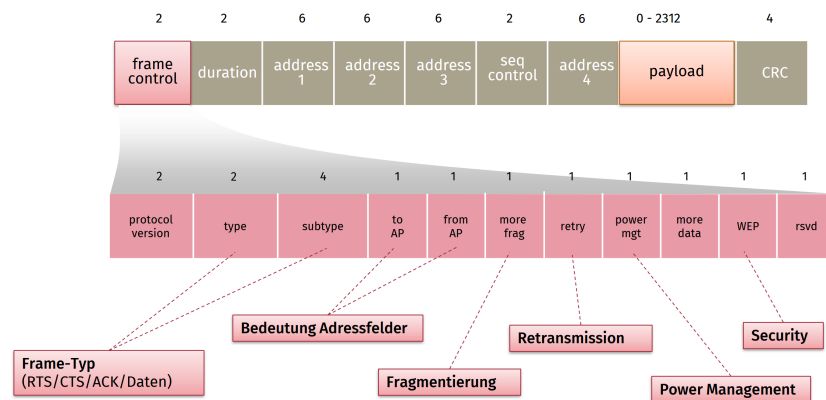


Abbildung 57: WLAN Frame Control

6.4.4 Personal Area Networks (PANs): Bluetooth

- Betrieb im Ad-Hoc-Modus
- Master-Controller & Client-Geräte bilden Piconetz
 - Master pollt Clients und verteilte Übertragungserlaubnis
 - Parked-Modus: kurzzeitiger Übertragungs-Standby von Geräten
- 2.4-2.5 GHz ISM Band mit bis zu 3 Mbps
 - Frequency Hopping Spread Spectrum: pseudo-zufälliges Wechseln der Kanäle je Piconetz

7 G Wiederholende Zusammenfassung

7.1 Aufbau der Internetverbindung

- Laptop benötigt eigene IP Adresse, Adresse des first-hop-routers, Adresse des DNS Servers: DHCP
- DHCP Request in UDP Datagramm, in IP Paket, in 802.3 Frame
- Ethernet Frame Broadcast (dest: FFFFFFFFFFFFFFFF) im LAN, empfangen von Router mit DHCP Server (Proxy)
- Ethernet demuxed zu IP demuxed zu UDP demuxed zu DHCP Server

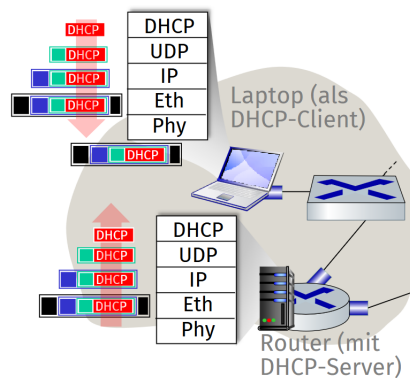


Abbildung 58: Aufbau Internetverbindung

- DHCP-Server schickt DHCP ACK mit Client IP-Adresse, IP-Adresse first-hop Router, Name & IP-Adresse DNS-Server
- DHCP-Server schickt Frame via Switch (switch learning) durch LAN an Client
- DHCP-Client empfängt DHCP ACK Antwort
- Client hat jetzt IP-Adresse, kennt Name & Adresse des DNS-Servers sowie IP-Adresse des first-hop Routers.

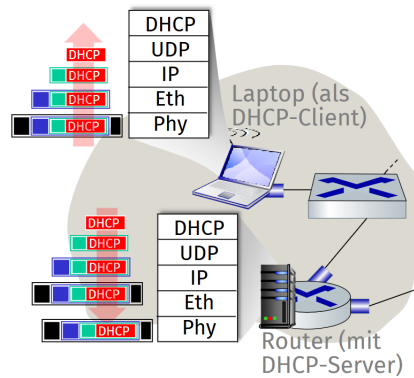


Abbildung 59: Aufbau Internetverbindung (2)

7.2 ARP-Auflösung (noch vor DNS und HTTP)

- DNS-Abfrage für Zielhost notwendig
- Erstellung von DNS Query, Kapselung in UDP/IP/Ethernet Paket
- Versand von Ethernet-Frame an Router
- MAC-Adresse des Routers?: ARP Query Broadcast
- Router antwortet mit ARP Reply
- MAC-Adresse des Routers nun für Ethernet-Frame (mit ursprünglicher DNS Query) bekannt

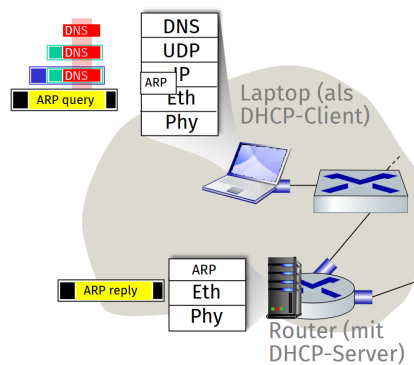


Abbildung 60: ARP Auflösung

7.3 DNS-Auflösung

- IP-Datagram mit DNS-Anfrage wird über Switch zum First- Hop-Router weitergeleitet
- IP-Datagram wird weiter in das Netzwerk des ISPs geleitet und zum DNS-Server geroutet (auf Basis von RIP, OSPF, IS-IS und/oder BGP)
- Demux zu DNS & DNS Reply mit IP-Adresse

7.4 TCP-Verbindungsaufbau

- TCP SYN Segment
- Antwort von Server mit SYN/ACK
- nach Bestätigung (ACK): TCP-Verbindung aufgebaut

7.5 HTTP-Kommunikation

- Versand von HTTP-Request über TCP
- Routing von IP-Datagram mit Request zu Webserver
- HTTP-Response als Antwort des Webserver auf Request
- Routing des IP-Datagrams mit Response zurück zum Client

8 H Einführung Verteilte Systeme

8.1 H1 Motivation: Warum verteilte Systeme?

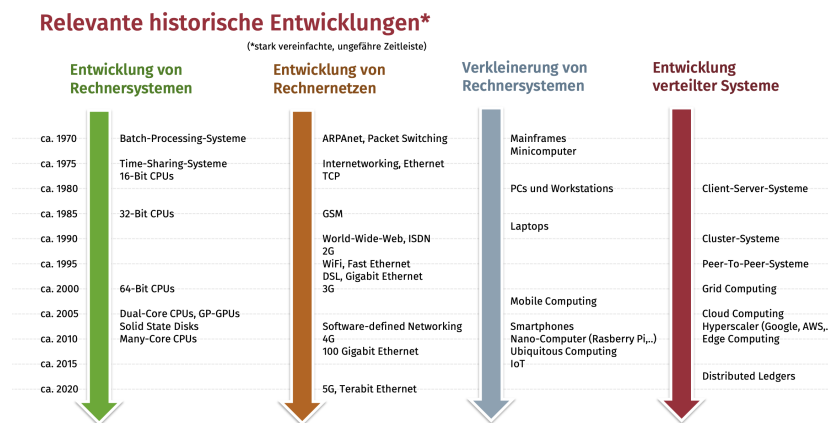


Abbildung 61: Relevante Historische Entwicklungen

8.1.1 Definition verteilter Systeme

Definition nach Lamport

- „Ein verteiltes System ist ein System, bei dem der Ausfall eines Rechners, von dem man vorher noch nie gehört hat, zum Ausfall des Gesamtsystems führt.“
- **Merkmale dieser Definition:**
 - eher zynische/ ironische Beschreibung, für praktische Betrachtung nicht hilfreich
 - mittlerweile auch inhaltlich oft nicht mehr zeitgemäß

Definition nach Tanenbaum & van Steen

- „Ein verteiltes System ist eine Ansammlung von unabhängigen Rechnern, die für seine Benutzer wie ein einzelnes Computersystem aussieht.“
- **Merkmale dieser Definition:**
 - Fokus auf Innensicht/ Außensicht (mehrere Systeme bilden nach außen hin ein gemeinsames System)

- Fokus auf Rechner - Definition damit teilweise etwas einschränkend

Definition nach Coulouris et al.

- „Ein verteiltes System ist ein System, in dem Komponenten, die sich auf vernetzten Computern befinden, miteinander kommunizieren und ihre Aktionen nur durch die Weitergabe von Nachrichten koordinieren.“
- **Merkmale dieser Definition:**
 - Komponenten auf Rechnern: allgemeiner als vorherige Definition von Tanenbaum/ Steen
 - Komponenten agieren nebenläufig
 - Komponenten können unabhängig voneinander ausfallen
 - Komponenten haben lokalen Zustand, können jedoch mit anderen kommunizieren
 - Netzwerknachrichten als alleiniger Interaktions- und Koordinationsmechanismus

Verschiedene Ansichten:

- **Hardware Sicht:**
 - mehrere Rechner
 - Kommunikation zwischen den Rechner über Computernetzwerke
- **Software Sicht:**
 - verteilte Anwendungsprogramme
 - spezialisierte Systemsoftware

8.1.2 Verteilungsbegriffe

Definition 8.1: verteilte Algorithmen

- Software-Anwendungen, deren Komponenten auf mehreren Rechnern ausgeführt werden
- Anwendungsarchitektur profitiert von den Eigenschaften verteilter Systeme

- Beispiel: verteiltes Datenbanksystem, das auf drei Rechnern läuft

Definition 8.2: verteilte Algorithmen

- Algorithmen, die für verteilte Systemen entworfen wurden und verteilt ausgeführt werden können
- Durchführung lokaler Berechnungsschritte, Nachrichtenaustausch für Datenaustausch/Koordinierung
- Beispiel: Distance-Vector-Algorithmus (Folie D-77) zur verteilten Berechnung von Routing-Tabellen

Definition 8.3: verteiltes Rechnen

- rechen- oder datenintensive Berechnungsaufgaben, die auf verteiltem System ausgeführt werden
- Nutzung einer verteilten Anwendung oder Systemsoftware zur Ausführung
- Beispiel: Folding@home zur verteilten Simulation von Proteinfaltung

8.1.3 Anwendungsszenarien verteilter Systeme

Eine dedizierte Anwendung

- **hoch-parallelisierte Anwendungen**
 - große Datenmengen, kurze Berechnungsdauer
 - effizientes Rechnen auf vielen Rechnern gleichzeitig
- Beispiele:
 - Simulationen, naturwissenschaftliche Berechnungen: CMS Cern, MDPI Aerospace & Folding@home

Ein User an mehreren Orten

- Verwendung verschiedener Rechnersysteme (PC, Laptop, Smartphone)
- Mobilität der Benutzer
- Wunsch nach homogener Umgebung

- Beispiele:
 - zentrale Dateihaltung (File-Server)
 - zentraler Terminkalender
 - E-Mail-Postfach mit IMAP

Viele User an vielen Orten

- Effizienz, Lastverteilung, Verfügbarkeit & Überwindung der Ortsgrenzen
- Beispiele:
 - Multiplayer Online Games, virtuelle Welten, Chat und Video Conferencing, E-Commerce

Viele User an vielen Orten (fortges.)

- dezentrale mobile Anwendungen
 - Corona-Warn-App, AirDrop, Quick Share & Vehicle-to-X Kommunikation

8.2 Definitionen und Grundlagen

8.2.1 Flynn'sche Taxonomie

Klassifikation von Rechnerarchitekturen nach Flynn (1972)

- **SSID** - Single Instruction Stream, Single Data Stream
 - alle Einprozessor-Maschinen (PC, Mainframe etc.)
- **SIMD** - Single Instruction Stream, Multiple Data Streams
 - Hochleistungsrechner: Vektorprozessor
- **MISD** - Multiple Instruction Streams, Single Data Stream
 - keine Rechner bekannt, nie gebaut, weil keine verlässliche Deterministische Anwendungssemantik
- **MIMD** - Multiple Instruction Streams, Multiple Data Streams
 - parallele und verteilte Systeme mit unabhängigen Prozessoren
 - verteilte Systeme sind im Bereich MIMD angesiedelt

8.2.2 Nebenläufigkeit, Parallelität & Verteilung

Definition 8.4: Nebenläufigkeit

- gleichzeitige Ausführung mehrerer Aktivitätsstränge
- möglicherweise auch durch abwechselnde Ausführung
- bereits mit einer CPU möglich (Interrupt/Kontextwechsel)



Abbildung 62: Nebenläufigkeit

Definition 8.5: Parallelität

- echt-gleichzeitige Ausführung mehrerer Aktivitätsstränge
- erfordert zwingend mehrere CPUs oder CPU-Kerne



Abbildung 63: Parallelität

Definition 8.6: Verteilte Ausführung

- Ausführung auf mehreren Rechnern ist implizit parallel
- jeder Rechner kann echt-gleichzeitig zu anderen Rechnern den lokalen Aktivitätsstrang ausführen



Abbildung 64: Verteilte Ausführung

Merkmal	Nebenläufigkeit (Concurrency)	Parallelität (Parallelism)
Konzept	Aktivitäten überlappen zeitlich	Aktivitäten passieren echt gleichzeitig (simultan)
Implikation	Impliziert keine echte Gleichzeitigkeit	Impliziert immer Nebenläufigkeit
Hardware	Bereits mit einer CPU möglich (Scheduling)	Erfordert zwingend mehrere CPUs/Kerne
Beispiel (Herd)	Eine Platte: Abwechselnde Nutzung mit zwei Töpfen	Zwei Platten: Simultanes Kochen mit zwei Töpfen
Verteilung	Prozesse können lokal nebenläufig sein	Mehrere Rechner sind implizit parallel

Tabelle 1: Gegenüberstellung von Nebenläufigkeit und Parallelität

Nebenläufigkeit vs. Parallelität

8.2.3 Zentralisierte vs. dezentralisierte Topologien

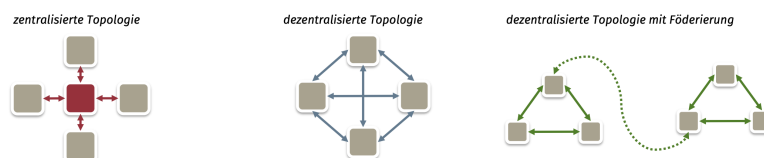


Abbildung 65: Zentralisierte vs. dezentralisierte Topologien

Zentrale und dezentralisierte Systeme

- wichtige Unterscheidung: logische Sicht vs. physikalische/ geografische Sicht
- Beispiel: Domain Name System
 - logische Sicht: zentralisiertes, globales System mit fester Hierarchie
 - physikalische/ geografische Sicht: dezentralisierte Topologie mit unabhängigen Instanzen & kopien
- Unterschied "Zentral" vs. "dezentrale" bei verteilten Systemen nicht immer eindeutig/ hilfreich

8.2.4 Enge kopplung vs. lose Kopplung von verteilten Anwendungen

Kopplung von Anwendungskomponenten

- verteilte Anwendungen laufen auf einem verteilten System
 - mehrere Komponenten der Anwendungen verteilt auf mehrere Rechner
 - interne, anwendungsabhängige Kommunikation zwischen Komponenten
- eng gekoppelte Anwendungskomponenten
 - starke Abhängigkeit zwischen den Komponenten
 - starke Verzahnung, simultanes Arbeiten
 - Beispiele: parallele Berechnungen, Multiplayer-Spiel
- lose gekoppelte Anwendungskomponenten
 - gelegentliche Kommunikation. kaum Abhängigkeit
 - Beispiele: WWW, verteiltes Dateisystem
- trennscharfe Unterscheidung in der Praxis oft nicht möglich

8.3 H.3 Design-Ziele und Merkmale verteilter Systeme

8.3.1 Gemeinsame Nutzung von Ressourcen

Ressourcen in verteilten Systemen

- Bereitstellung von Ressourcen über das Netzwerk
 - oft in Form eines Netzwerkdiensts bzw. als Service
 - typischerweise Server-Perspektive einer verteilten Anwendung
 - Beispiel: Service für Datei-Hosting (Dropbox, Nextcloud, Amazon S3)
- Nutzung von Ressourcen über das Netzwerk
 - meist durch Zugriff auf entfernten Dienst/Service
 - typischerweise Client-Perspektive einer verteilten Anwendung
 - Beispiel: Client-App für Video-Streaming-Dienst (z.B. Netflix)
- auch gemeinsame Verwendung der Ressourcen des verteilten Systems möglich
 - teilnehmende Knoten fügen lokale Ressourcen dem verteilten System hinzu
 - entspricht oft der Peer-to-Peer-Perspektive
 - Beispiel: folding@home (Berechnungsressourcen), BitTorrent (Speicherkapazität und Bandbreite)

8.3.2 Offenheit

Offenheit verteilter Systeme

- Verwendung von (offen) spezifizierten Protokollen, APIs oder Interfaces (RFCs, Protokoll-Standards etc.)
- Separierung von Software-Implementierung und spezifiziertem Standard
 - trennt Schnittstelle nach außen und tatsächliche, interne Implementierung
 - stellt Kompatibilität verschiedener Implementierungen untereinander sicher

- vereinfacht Erweiterbarkeit von Schnittstellen und Implementierungen
- Beispiel offener verteilter Systeme: WWW
 - HTTP als Protokoll zwischen verschiedensten Browser- und Server-Implementierungen
 - Nutzung des Webs unabhängig möglich, sofern Software HTTP-Standard korrekt implementiert
- Gegenbeispiel: Skype, Apple iMessage, uvm.
 - kommerzielle Dienstleister (Geschäftsmodell?) haben oft kein Interesse an Third-Party-Software
 - Geheimhaltung des Protokolls, Präventionsmaßnahmen gegen fremde Software

8.3.3 Heterogenität

Heterogenität in verteilten Systemen

- Nutzung verschiedener Netzwerkinfrastrukturen/ Netzwerkprotokolle/ Hardwarearchitekturen/ Betriebssysteme/ Software ...
- verteiltes System muss dennoch Interoperabilität zwischen Komponenten ermöglichen
- insbesondere relevant bei offenen, verteilten Systemen

8.3.4 Verteilungstransparenz

Abstraktion von Komplexität in verteilten Systemen

- verteilte Systeme sind inhärent komplex(er)
 - unzuverlässige Kommunikation, (Teil-)Ausfälle, möglicherweise viele teilnehmenden Knoten
- Abstraktion bestimmter Merkmale erleichtert Umgang mit verteilten Systemen
 - Reduktion der Komplexität für Entwickler*innen und Benutzer*innen
 - System-Software oder Software-Komponenten lösen Teilproblem und bieten verschiedene Transparenz-Eigenschaften

- vgl. Netzwerkstack: verschiedene Schichten lösen unterschiedliche Teilprobleme
- Beispiele: Zugriffe auf eine Datei via Datei-Hoster (z.B. Dropbox)
 - relevant für User: Datei ist aufrufbar
 - im Idealfall nicht relevant für diese Funktion:
 - * Auf welchem server speichern?
 - * Befinden ihre Dateien auf anderen internen Server?
 - * gibt es evtl. Kopien der Datei

8.3.5 Auswahl an Verteilungstransparenz

Zugriffstransparenz

- verdeckt die Unterschiede zwischen eines lokalen vs. eines entfernten Zugriffs auf eine Ressource

Ortstransparenz

- verdeckt den konkreten Ort, also die Instanz, auf der sich eine geteilte Ressource befindet

Migrationstransparenz

- erlaubt das Verschieben einer Ressource von einer instanz zu einer anderen

Relokationstransparenz

- erlaubt das Verschieben einer Ressource selbst wenn diese aktuell verwendet wird

Replikationstransparenz

- verdeckt die Tatsache, dass eine Ressource möglicherweise mehrmals im verteilten System existiert

Nebenläufigkeitstransparenz

- erlaubt gleichzeitige Zugriffe auf ressourcen im verteilten System

Fehlertransparenz

- verdeckt das Auftreten oder Beheben von Fehlern

Persistenztransparenz

- versteckt Implementierungsdetails, wie Zustand im System gespeichert wird

Zu viel Abstraktion von Komplexität?

- nicht immer alle Transparenz in der Praxis wünschenswert
 - verstecken von Komplexität vergrößert die interne Komplexität
 - auswirkung auf Performance, Wartbarkeit, Erweiterbarkeit
- abhängig von konkreter Anwendung und Perspektive
- Beispiel:
 - Benutzer eines Datei-Hosters: erwartet z.B. Orts- und Fehlertransparenz

8.3.6 Sicherheit

Sicherheit in verteilten Systemen

- Computernetzwerke oft inhärent unsicher
 - Netzwerksicherheit in der Anfangsphase von Ethernet, IP und Internet noch kaum relevant gewesen
 - massive Auswirkungen auf heutige Rechnernetze und Netzwerkanwendungen
- verschiedenste Angriffe möglich
 - Angriffe auf Vertraulichkeit, z.B. Mitlesen von Netzwerknachrichten
 - Angriffe auf Integrität, z.B. Manipulation von Netzwerknachrichten
 - Angriffe auf Verfügbarkeit, z.B. Überfluten eines Netzwerkdiensts bis zum Ausfall
- (pro-)aktive Maßnahmen zur Absicherung, u.a.:
 - Verwendung von kryptografischen Mechanismen wie Verschlüsselung
 - Authentifizierung von Benutzer*innen oder anderen Systemen
 - Erkennung und Abwehr von Angriffen

8.3.7 Skalierbarkeit

Anpassung eines verteilten Systems an sich ändernde Systemlast

- steigender (fallender) Ressourcenbedarf eines verteilten Systems
- Hinzunahme oder Entfernung von Ressourcen
 - **scale-up**: Aufrüsten der bestehenden Hardware
 - **scale-out**: Hinzufügen von Rechnern

8.3.8 Fehlertoleranz

Verfügbarkeit und Zuverlässigkeit

- redundante Auslegung von Komponenten möglich
 - mehrere Komponenten des System erfüllen gleiche Funktionalität oder bieten gleiche Ressourcen an
- System bleibt auch bei Ausfall von (Teil-)Komponenten verfügbar
 - erfordert Erkennung und geeignete Behandlung des Ausfalls
- Maskierung von Fehlern von Hardware- und Softwarekomponenten
 - Fail-Stop-Fehler
 - byzantinische Fehler

8.4 H.4 Fallstricke verteilter Systeme & Anwendungen

8.4.1 Fallstricke verteilter Systeme

Inhärent Komplexität verteilter Systeme

- verteilte Systeme quasi immer komplexer als nicht verteilter Systeme
- nicht verteiltes System
 - während der Ausführung möglich lokale Hardware- oder Software-Probleme oder Fehler
- verteiltes System
 - Hardware- oder Software-Ausfälle bei irgendeine, oder mehreren Knoten des verteilten Systems z.B.: **partiell laufendes System**: manche Knoten verfügbar, andere ausgefallen

- Ausfall des Netwerks oder Teile des Netzwerks z.B.: **split-brain**: die Kommunikationstopologie eines verteilten Systems zerfällt in zwei disjunkte Teile, die aufgrund eines Netzwerksausfalls nicht mehr miteinander kommunizieren können
- Beispiele weiter Fehlerquellen
 - * Heterogenität von Knoten (unterschiedliche Hardware-Architekturen)
 - * Ausfall zentraler Komponenten stört Gesamtsystem (Single point of failure)

Weitere Phänomene in verteilten Systemen

- **unsichere**, nachrichtenbasierte **Datenübertragung** über das Netzwerk
 - Nachrichtenverlust, fehlerhafte Übertragungen
- **unbestimmte Nachrichtenlaufzeit**
 - unterschiedliche routen von Paketen im Netzwerk
 - unterschiedliche Übertragungsleistungen (Bandbreite)
 - Überholung von Nachrichten (Paket-Reihen)
- keine gemeinsame Zeitbasis
 - Ereignisse ohne zeitliche Ordnung im Gesamtsystem
- **verteilter Systemzustand**
 - Inkonsistenz von redundant gespeichert Informationen

Sammlung von Fallstricken in vernetzten Systemen

- Katalog von Fehlannahmen und Stolperfallen in Netzwerk und Netzwerkanwendungen
- relevant sowohl für die Netzwerkadministration als auch für den Entwurf verteilter Systeme
- Verwendung als zu vermeidende Entwurfsmuster (Anti-Patterns)
 - Überprüfung, ob eigene Systementwürfe implizit Fehlannahmen treffen

8.4.2 Fallcies of Distributes Computing

Die Irrtümer des verteilten Computing

1. The network is reliable
2. Latency is zero
3. Bandwith is infinite
4. The network is secure
5. topology doesnt change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous

8.4.3 The Twelve Networking Truths

1. It Has To Work.
2. No matter how hard you push and no matter what the priority, you can't increase the speed of light.
 - a) (corollary). No matter how hard you try, you can't make a baby in much less than 9 months. Trying to speed this up *might* make it slower, but it won't make it happen any quicker.
3. With sufficient thrust, pigs fly just fine. However, this is not necessarily a good idea. It is hard to be sure where they are going to land, and it could be dangerous sitting under them as they fly overhead.
4. Some things in life can never be fully appreciated nor understood unless experienced firsthand. Some things in networking can never be fully understood by someone who neither builds commercial networking equipment nor runs an operational network.
5. It is always possible to aglutenate multiple separate problems into a single complex interdependent solution. In most cases this is a bad idea.
6. It is easier to move a problem around (for example, by moving the problem to a different part of the overall network architecture) than it is to solve it.

- a. (corollary). It is always possible to add another level of indirection.
- 7. It is always something
 - a. (corollary). Good, Fast, Cheap: Pick any two (you can't have all three).
- 8. It is more complicated than you think.
- 9. For all resources, whatever it is, you need more.
 - a. (corollary) Every networking problem always takes longer to solve than it seems like it should.
- 10. One size never fits all.
- 11. Every old idea will be proposed again with a different name and a different presentation, regardless of whether it works.
 - a. (corollary). See rule 6a.
- 12. In protocol design, perfection has been reached not when there is nothing left to add, but when there is nothing left to take away.

9 I Konzepte und Paradigmen verteilter Systeme

9.1 Konzepte und Paradigmen

9.1.1 Betriebssysteme für verteilte Anwendungen

Netzwerkbetriebssysteme

- netzwerkfähiges OS
- anwendungsspezifische Verteilung und Kommunikation

Verteiltes Betriebssystem

- OS ist selbst verteilt
- transparente Ausführung einer Anwendung durch OS auf verteilter Infrastruktur

Middleware

- Bereitstellung einer Laufzeitumgebung mit Verteilungsfunktionen
- Baut auf klassischen netzwerkfähigen Betriebssystemen auf
- Middleware bietet Laufzeitumgebung und Bibliothek für verteilte Anwendungen
- Verteilte Anwendung baut wiederum auf Middleware auf

9.1.2 Lokale vs. globale Perspektive

- **Gesamtsystemsicht:** Globale Sicht des Systems über die Zeit
- **Lokale Sicht der beteiligten Knoten**
 - Tatsächliche Sicht der beteiligten Knoten
 - Wissen über lokalen Zustand, sowie lokal gesendete und empfangene Nachrichten

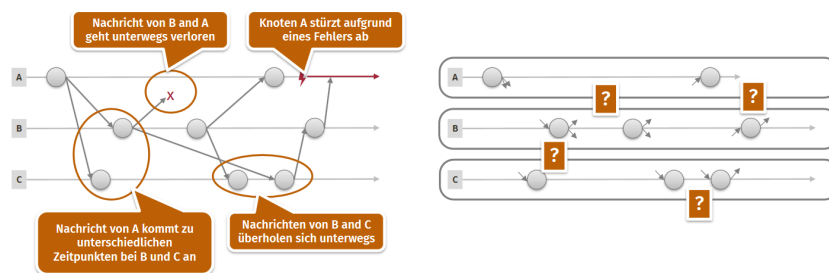


Abbildung 66: Globale vs lokale Perspektive

9.1.3 Zeit in verteilten Systemen

- Uhren laufen nie perfekt gleichmäßig
- Exakte Synchronisierung zwischen Uhren unmöglich

9.2 Kommunikationsparadigmen

9.2.1 Nachrichtenbasierte Kommunikation

Aufrufbasierte Kommunikation

- Client stellt Anfrage

- Server schickt Antwort
- **One-way Aufrufe:** Anfragen ohne Antworten

Strombasierte Kommunikation

- Geordneter Strom von Anwendungsnachrichten
- **Application-level framing:** Anwendung erstellt Nachrichtengrenzen für die Konstruktion von Nachrichten aus Datenstrom

Umsetzung nachrichtenbasierter Kommunikation

- Direkte Verwendung von Sockets
- Nachrichtenbasierte Middleware, die Nachrichtenaustausch übernimmt und Komplexität versteckt

Message-oriented Middleware

- Warteschlangen für Nachrichten
- Pufferung von Nachrichten
- Priorisierung bestimmter Nachrichten
- Zuverlässige Zustellung
- Als dedizierte Komponente: **Message Broker**
 - zentralisierte Lösung
 - stärkere Entkopplung Sender / Empfänger

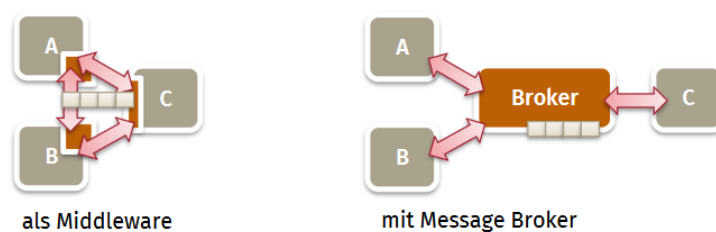


Abbildung 67: Middleware vs. Message Broker

9.2.2 Remote Procedure Call (RPC)

- Knoten stellt Funktionen im Netzwerk bereit, die von anderen Knoten aufgerufen werden können
- Client- und serverseitige Komponenten zur Unterstützung (meist auto-generiert)
 - **Stub**: clientseitiger Stellvertreter der entfernten Funktion; leitet Aufrufe weiter
 - **Skeleton**: serverseitiger Wrapper um lokale Funktion, um diese entfernt aufrufbar zu machen

Java RMI: Remote Method Invocation

- Marker-Interface (Remote) definiert Methoden als entfernt aufrufbar
- Server-seitiges Objekt implementiert Interface und erbt von Unicast-RemoteObject
- **Registry**: verknüpft benannte Objekte zu deren Endpunkten bzw. Objektreferenzen

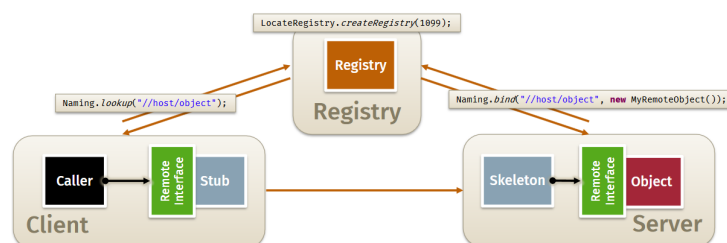


Abbildung 68: Java RMI Registry

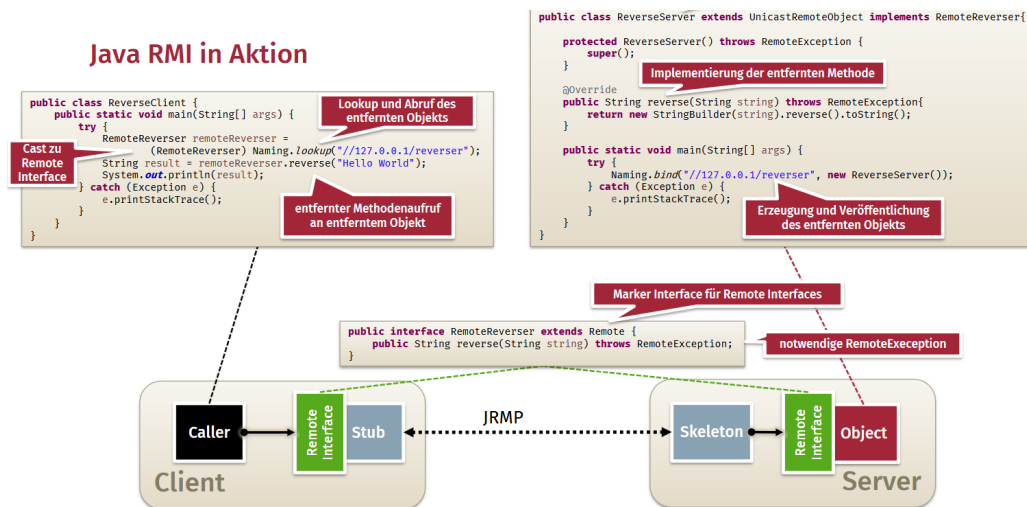


Abbildung 69: Java RMI in Aktion

9.3 Architekturstile verteilter Systeme

9.3.1 Client/Server-Architekturen

- **Server:** stellen Dienste/Ressourcen für Clients bereit
- **Clients:** nutzen Dienste/Ressourcen von Servern
- Push-basierte und pull-basierte Protokolle
- Komplexere serverseitige Architekturen möglich: Aufteilung der Komplexität durch separierte layers oder tiers

Client-Typen

- **Thin client:** Anwendungslogik im Server
- **Thick client:** Anwendungslogik im Client

9.3.2 Mehrschichtige bzw. Multi-Tier Architekturen

- Aufteilung in separate Server
- Mehrschichtige Architekturen: logische Separierung der Anwendung (layers)
- Multi-Tier Architekturen: Aufteilung in separate Einheiten (tiers) z.B. Webserver, Anwendungsserver, Datenbankserver

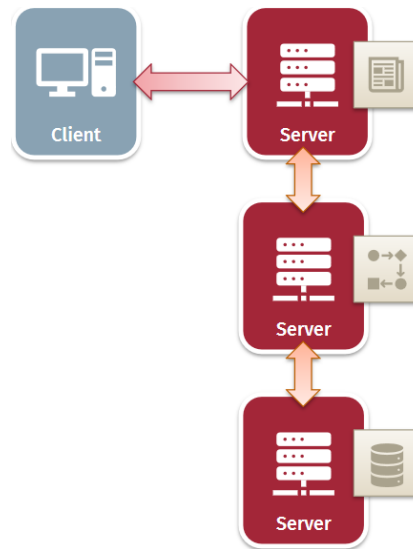


Abbildung 70: Mehrschichtige Architektur

9.3.3 Service-orientierte Architekturen

- Knoten stellen Service bereit und führen Service bei Aufruf durch Clients aus
- Eigenschaften von Services
 - **Self-contained** (in sich abgeschlossen)
 - Unabhängigkeit von anderen Services
 - Auffindbarkeit (z.B. über Service Registry)

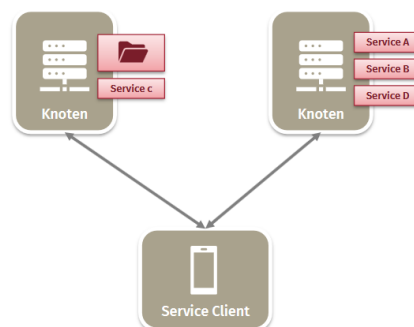


Abbildung 71: Service-orientierte Architektur

Beispiele

- Entfernte Funktionsaufrufe
- „Service-oriented Architectures“ (SOA): grob-granulare (Web-)Services
- Ressourcen-zentrische Architekturen (REST)
- Microservices

9.3.4 Publish/Subscribe-basierte Architekturen

Eigenschaften

- Komponenten interagieren über Nachrichten: Komponenten können Nachrichten **veröffentlichen** oder **subskribieren**
- Lose Kopplung von Komponenten
- Offenheit und Erweiterbarkeit: Einführung neuer Publisher oder Subscriber bzw. neue Nachrichtentypen
- Häufig verwendet in **event-basierten Architekturen**

Aufbau

- **Themenbasierte** Nachrichtenzustellung: separate, logische Kanäle für Nachrichtenthemen
- **Inhaltsbasierte** Nachrichtenverteilung: Filterung von relevanten Nachrichten auf Basis von Attributen der Nachricht je Subscriber
- **Zentrale** Nachrichtenverteilung: Message Broker z.B. Apache Kafka
- **Dezentrale** Nachrichtenverteilung: über Middleware z.B. ZeroMQ

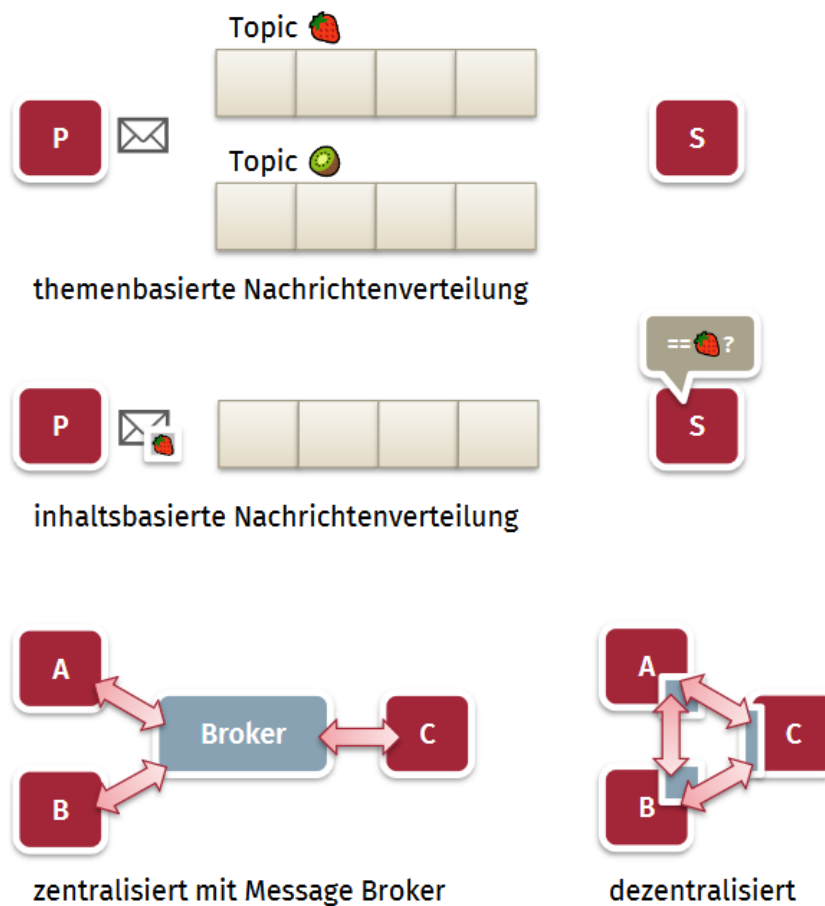


Abbildung 72: Publish/Subscribe Architektur

9.3.5 Peer-to-Peer-Architekturen

Eigenschaften

- Kein ständig verfügbarer Server
- hoher Churn der Peers: können jederzeit beitreten oder System verlassen
- **Selbstskalierbarkeit** (Systemleistung wächst mit mehr Knoten)
- **Dezentralität**
- **Heterogenität** der Peers und Offenheit des Systems

Aufbau

- **Strukturierte** P2P-Systeme: Overlay-Topologie der teilnehmenden Knoten z.B. Ring-Topologie durch Hash der IP
- **Unstrukturierte** P2P-Systeme: Topologie entspricht einem Zufallsgraph
- **Hierarchische** P2P-Systeme
 - **Super peers** übernehmen zentralisierte Aufgaben im System
 - **Weak peers** verwenden super peers, z.B. als Broker

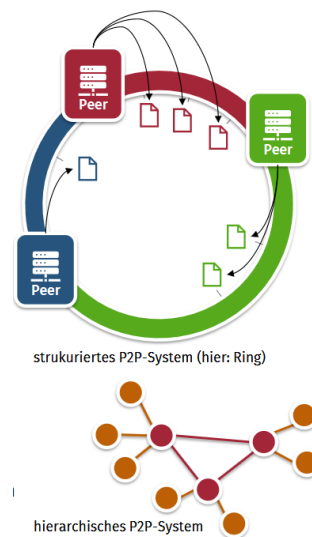


Abbildung 73: P2P-Architektur

9.3.6 Hybride Architekturen

Cloud Computing

- **Pay-as-you-go**: berechnung nach tatsächlicher Nutzung
- **On-demand self service**: einfache Bereitstellung, z.B. via API
- Pooling der Ressourcen beim Anbieter
- Hohe Elastizität: erlaubt Skalierung nach oben oder unten

Grundsätzliche Dienstmodelle

- **Infrastructure as a Service (IaaS):** virtualisierte Computerhardware
- **Platform as a Service (PaaS):** Laufzeitumgebungen für Anwendungen
- **Software as a Service (SaaS):** Anwendungen (z.B. Google Docs)

Edge Computing

- Verteilte Systeme "am Rand" des Netzwerks
- Vernetzung physischer und virtueller Ressourcen, Dienste und Objekte
- Aufwendige Berechnungen in der Cloud

Distributed Ledger Technology

- Vertrauenswürdiger Dienst ohne notwendige trusted third party
- Dezentralisierter Transaktions-Log
- Verschiedene Ansätze
 - P2P-ähnliche Strukturen
 - Consensus-Protokoll für gemeinsame Wahl und Validierung des nächsten Eintrags
 - Replizierte Speicherung des Transaktions-Logs
 - Kryptografische Absicherung des Logs
- Permissioned vs. permissionless blockchains: wenige vs. alle Knoten kontrollieren Log

9.3.7 Monolithische vs. modulare Architekturen

Monolithische Architekturen

- ein einzelnes System
- (oft) eine große Code-Basis
- enge Kopplung der Komponenten
- einfacheres Deployment
- Wartung/Erweiterung eher schwieriger

Modulare Architekturen

- ein System unabhängiger Komponenten
- unabhängige Implementierungen
- lose Kopplung der Komponenten
- komplexeres Deployment
- leichter zu warten und zu erweitern



Abbildung 74: Monolithische vs. Modulare Architekturen

9.4 Koordination

Definition 9.1: Uhrenbedingung

$$a \rightarrow b \implies T(a) < T(b)$$

Ungenauigkeit von Uhren

- **Abweichung** (skew): Differenz von Uhr zur korrekten Realzeit
- **Genauigkeit** (drift): Unterschied im Gang zur Realzeit (schneller oder langsamer)

9.4.1 Ansätze zur Zeitsynchronisation

Christians Verfahren

Ein einfacher Client-Server-Ansatz, bei dem der Client die Serverzeit anfragt.

$$T_{neu} = T_{Server} + \frac{RTT}{2}$$

Problem: Setzt symmetrische Netzwerklaufzeiten voraus ($t_{req} \approx t_{res}$), was im Internet selten gegeben ist.

Network Time Protocol (NTP)

Nutzt ein 4-Zeitstempel-Verfahren zur präziseren Schätzung.

- **Verzögerung (δ):** Misst die reine Netzwerklaufzeit ohne Bearbeitungszeit im Server.

$$\delta = (T_4 - T_1) - (T_3 - T_2)$$

- **Phasenversatz (Θ):** Schätzt den Zeitunterschied zwischen Client- und Server-Uhr.

$$\Theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

9.4.2 Logische Uhren

- Realzeit häufig nicht ausreichend aufgrund von Synchronisierungsproblematik
- Realzeit für Reihenfolge der Ereignisse nicht zwingend notwendig
- Logische Uhren enthalten streng monoton steigende Werte
 - jedes lokale Ereignisse erhöht den Wert der Uhr
 - Durch Nachrichten werden die logischen Uhren der einzelnen Knoten synchronisiert

Lamport-Uhren

- Jeder Knoten hat eigene logische Uhr (LC)
- Inkrementierung der logischen Uhr bei relevantem Ereignis im Knoten
 - **Lokales Ereignis:** $LC = LC + 1$;
 - **Sendeereignis:** $LC = LC + 1$; $\text{send}(\text{message}, LC)$;
 - **Empfangsereignis:** $\text{receive}(\text{message}, LC_s)$; $LC = \max(LC, LC_s) + 1$;
- **Piggybacking:** ausgehende Nachrichten erhalten neben der Message auch den aktuellen Wert der logischen Uhr des Senders
- Erfüllt Uhrenbedingung
- Umkehrung der Uhrenbedingung nicht erfüllt $T(a) < T(b) \not\Rightarrow a \rightarrow b$
- **Totale Ordnung** möglich durch: Tupel aus Uhrzeit und Knoten mit Ordnungsrelation z.B. $(5, B) < (5, C)$

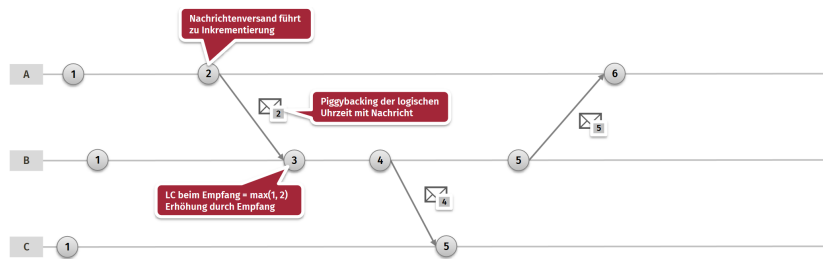


Abbildung 75: Lamport Uhren

Vektor-Uhren

- Ähnlich zu Lamport-Uhren, jedoch mit Vektoren der Größe n (bei einem System mit n Knoten)
- **Lokales Ereignis:** $VC[i] = VC[i] + 1$;
- **Sendeereignis:** $VC[i] = VC[i] + 1$; $\text{send}(\text{message}, VC)$;
- **Empfangsereignis:** $VC[i] = VC[i] + 1$; $\text{receive}(\text{message}, VC_s)$; $\forall j: VC[j] = \max(VC[j], VC_s[j])$

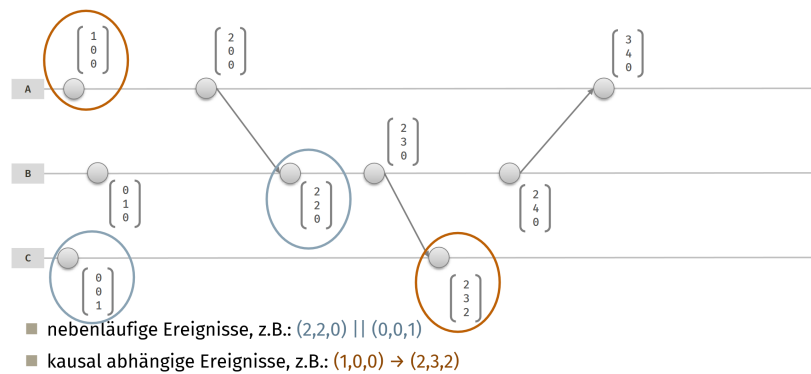


Abbildung 76: Vektor Uhren

9.4.3 Snapshots

Konsistente Schnitte

- **Snapshotting:** Zustandssicherung des verteilten Systems
- globaler Zustand als Menge aller lokaler Zustände
- nicht möglich mit physikalischen Uhren

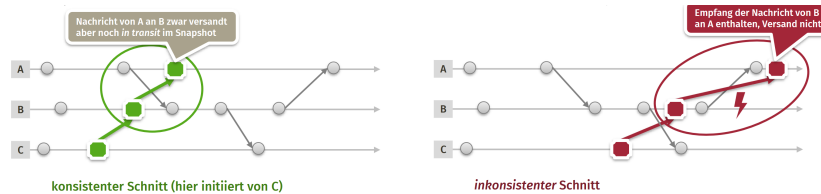


Abbildung 77: Konsistente Schnitte

Bedeutung von Snapshots

- Erkennung globaler Zustände, insbesondere stabile Zustände (Zustände ändern sich nicht mehr)
- Beispiel: **Deadlocks** oder **Terminierungen**

9.4.4 Gegenseitiger Ausschluss (Mutex)

- Kritische Ressourcen dürfen nur von einem Knoten gleichzeitig verwendet werden
- **Zentrale Lösung**: Knoten stellen Anfragen an Koordinator
- **Dezentrale Lösung**:
 - **Token**: Knoten geben Token an andere weiter
 - **Logische Uhren**: Lamport-Uhr mit totaler Ordnung; Warteschlange mit Zeitstempeln

9.4.5 Einigung in verteilten Systemen

Agreement / consensus algorithms

- Koordinierung einer gemeinsamen Entscheidung
- Verschiedene Varianten
 - Ein Knoten schlägt Wert vor, Einigung auf Annahme des Vorschlags
 - Alle Knoten schlagen Werte vor, Einigung auf einen finalen Wert

Election algorithms ((Aus-)Wahlalgorithmen)

- Einigung auf bestimmte Rolle eines teilnehmenden Knotens
- z.B. Wahl eines Koordinators im verteilten System

Umsetzung in fehlerfreien Systemen

- (Total geordnete) Multicasts an alle Knoten: alle Nachrichten kommen bei jedem Knoten in der richtigen Reihenfolge an
- Einfache Lösungen möglich

Umsetzung für Systeme mit komplexeren Fehlern

- Ausfall von Knoten
- Verlust und beliebige Latenz von Nachrichten
- Manipulation von Nachrichten
- Problem allgemein nicht lösbar, jedoch Ansätze mit hoher Erfolgchance

9.5 Replikation und Konsistenz

9.5.1 Konsistenzmodelle

Definition 9.2: Konsistenzmodell

Vertrag zwischen beteiligten Knoten & Prozessen

- Beschreibung des zu erwartenden Verhalten bei Lese- und Schreiboperationen
- Definition von gültigen und ungültigen Zuständen im Modell

Datenzentrierte Konsistenzmodelle

- Verteilter Datenspeicher mit lesenden & schreibenden Prozessen
- Systemweit konsistente Sicht

Client-zentrierte Konsistenzmodelle

- Perspektive der Clients des verteilten Datenspeichers
- Konsistenzgarantien für einzelne Clients

9.6 Fehlertoleranz

9.6.1 Fehlermodelle

- **Crash failure:** Programm-/ Knotenabsturz
- **Omission failure:** trotz Ausführung ist Knoten nicht verfügbar
- **Timing failure:** Antwort wird nicht innerhalb eines spezifizierten Zeitraums erbracht
- **Response failure:** Antwort ist inkorrekt
- **Byzantine failure:** System kann sich beliebig falsch verhalten, schwieriges Fehlermodell

9.6.2 Umgänge mit Fehlern

- **Vermeidung** von Faults
- **Tolerierung** von Faults und angemessener Umgang
 - **Fehlererkennung:** Hearbeats, Timeouts, Checksummen
 - **Fehlereindämmung:** Isolation der fehlerhaften Komponente
 - **Fehlermaskierung:** Dynamische Korrektur des Fehlers
 - **Reparatur:** Wiederherstellung eines funktionsfähigen Zustands
- **Replikation**
 - **Redundanz von Daten** (strukturelle Redundanz): replizierte Haltung von Daten
 - **Redundanz von Anwendungslogik** (funktionale Redundanz): replizierte Bereitstellung eines Dienstes

10 J Anwendungen Verteiler Systeme

10.1 J.1 Verteilte Berechnung mit MapReduce

Definition 10.1: Internet

"Netz von Netzen", Vernetzung von Internet Service Providern (ISPs)

10.1.1 Motivation

Berechnung auf großen Datenmengen

- Cluster bestehend aus einer Vielzahl von Maschinen
- verteilte Speicherung großer Datenmengen
- Verwendung der Rechenleistung vieler Maschinen zur schnellen Berechnung

Historisches Beispiel bei Google: Analyse von Webseiten nach Download durch Crawler

- Extraktion und Zählung von relevanten (key-) Wörtern in (Hypertext-)Dokumenten
 - Wie oft kommen bestimmte Wörter auf einer Website vor?
 - Information Retrieval: Welche Begriffe sind im Dokument relevant
- Extraktion und Analyse ausgehender Hyperlinks
 - Zu welchen anderen Webseiten verlinkt die Website?

10.1.2 Problemstellung: Wie solche Berechnungen durchführen?

Anforderungen

- Lösung muss für verschiedene Fragestellungen anpassbar sein
- Lösung muss parallelisiert & verteilt ausgeführt werden können
- Lösung muss skalieren
 - größerer Cluster => größere Eingabedaten möglich
 - größerer Cluster => kürzere Berechnungen möglich
- Lösung muss Herausforderungen von verteilten Systemen betrachten
 - Anwender sollen möglichst wenig über System wissen müssen
 - * Ort der Daten, Ort der Berechnung, Koordinierung der Berechnung etc.
 - * Verteilungstransparenz
 - Maschinen oder Prozesse im Cluster können ausfallen
 - * Fehlertoleranz

10.1.3 Map & Reduce als verteiltes Programmiermodell

Map & Reduce

- Operatoren auf Datenstrukturen in funktionalen Programmiersprachen
 - z.B. OCaml, Haskell, Lisp
 - sogenannte higher-order functions
 - (Name manchmal Fold anstatt Reduce)
- split-apply-combine Ansatz mit zwei vom User bereit gestellten Funktionen
 - map wird auf alle Input-Elemente angewandt und erzeugt (key, value)-Tupel als Zwischenergebnis
 - reduce berechnet das Endergebnis für jeden Zwischenergebnis-Key und den dazu gruppierten Werten
- Definition der Funktionen
 - Input: $[(k_1, v_1), \dots, (k_n, v_n)]$
 - $\text{map}(): (k, v) \rightarrow [(l_1, x_1), \dots, (l_{r_k}, x_{r_k})]$
 - $\text{reduce}(): (l, [y_1, \dots, y_{s_l}]) \rightarrow [w_1, \dots, w_{m_l}]$
 - Output: $[(l_1, w_1), \dots, (l_z, w_z)]$

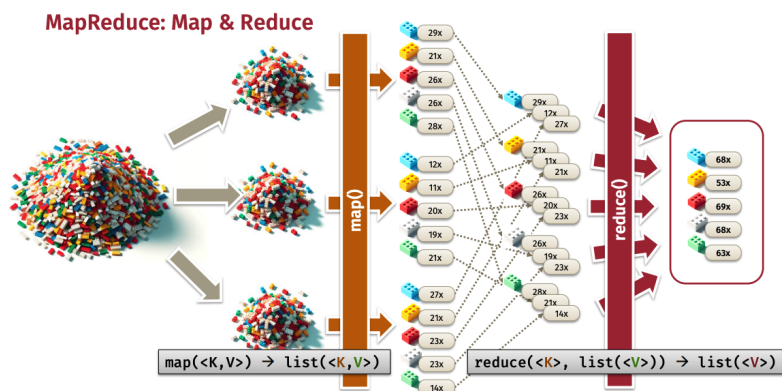


Abbildung 78: Map & Reduce Beispiel

Programmieren mit MapReduce (2)

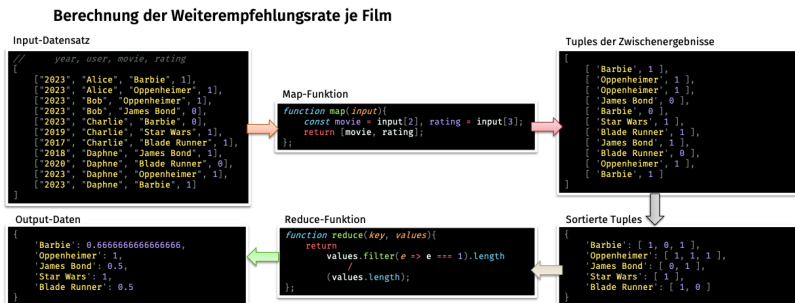


Abbildung 79: Map & Reduce Programmieren Beispiel

10.1.4 MapReduce-Paradigma als datenparalleles Berechnungsmodell

Embarrassingly Parallel Modell

- embarrassingly parallel $\approx$ perfekt parallelisierbar
- Modell benötigt lediglich zwei user-defined functions
 - Funktionen bekommen Eingabedaten und geben Ausgabedaten zurück
 - keinerlei Seiteneffekte (purely functional)
- klare Separierung von Programmiermodell und tatsächlichem Ausführungsmodell
 - keine Synchronisation in den user-defined functions
 - erlaubt hochparallele Ausführung und Verteilung
- Ausführungsumgebung übernimmt Hauptarbeit
 - Partitionierung und Verteilung von Daten
 - Verteilung, Scheduling und Koordinierung von Tasks

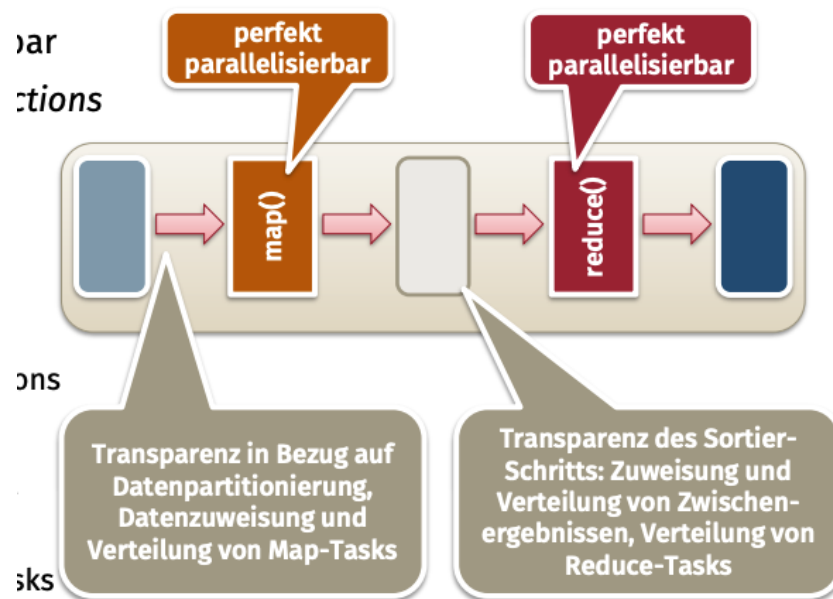


Abbildung 80: Map & Reduce datenparalleles Berechnungsmodell

10.1.5 MapReduce: Simplified Data Processing on Large Clusters

Ursprüngliches Google Paper von Dean & Ghemawat (NSDI '04)

- Programmiermodell und Implementierung (nicht veröffentlicht) für Datenverarbeitung
 - Fokus auf Big Data (Petabyte scale)
 - Fokus auf hoch-parallele und verteilte Ausführung (z.B. Cluster mit 100 – 1000 Maschinen)
- wichtigste Beiträge der wissenschaftlichen Veröffentlichung
 - Trennung von Logik (user-defined functions) und Laufzeitumgebung (Plattform)
 - Architektur des hoch-skalierbaren Ausführungssystems
 - Fähigkeiten der Fehlertoleranz
- Google beendete die interne Nutzung von MapReduce im Jahr 2014

10.1.6 Google MapReduce: Übersicht der Original-Architektur

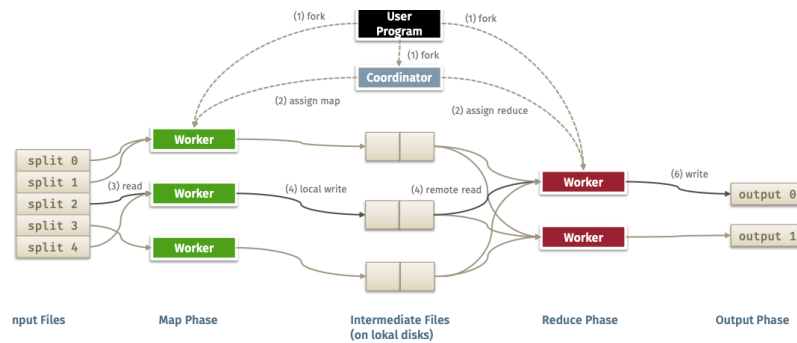


Abbildung 81: Google Map & Reduce Original Architektur

Coordinator & Worker

10.1.7 Google MapReduce: Fehlertoleranz

Unterstützung für den Ausfall von Workern

- periodische Heartbeats um Ausfälle zu erkennen
- bei Worker-Ausfall
 - Neu-Ausführung der Map-Tasks des Workers
 - Neu-Ausführung der nicht beendeten Reduce-Tasks des Workers
- bei Ausfall des Coordinators
 - im ursprünglichen Paper nicht behandelt
 - single-point-of-failure
 - mögliche Lösung: Checkpointing

10.1.8 Einige Optimierungen aus dem MapReduce Paper

Lokalität von Daten

- verteiltes Dateisystem stellt Informationen bereit, wo sich Input-Daten befinden
- Map-Tasks werden wenn möglich direkt dort beauftragt
- somit evtl. keine Netzwerkkommunikation notwendig, um Input-Daten zu laden

Redundante Ausführung von Tasks

- teilweise unterschiedliche Berechnungszeiten zwischen den Rechenknoten
- langsame Worker bremsen Gesamtberechnung aus (z.B. Warten auf letzte Map-Tasks)
- Lösung
 - Verteilung von identischen Jobs an mehrere Worker gegen Ende der Berechnung
 - Ergebnis des ersten Workers, der Ergebnis bereitstellt, „gewinnt“

10.1.9 Ergebnisse der Performance-Evaluierung (aus Originalpaper von 2004)

Setup

- 1800 Maschinen
- 4 GB RAM
- Xeon Dual CPU w/2 GHz
- dual 160 GB HDDs
- 1 GbE

Workload

- Sortierung
- 10^{10} Records mit jeweils 100 Bytes

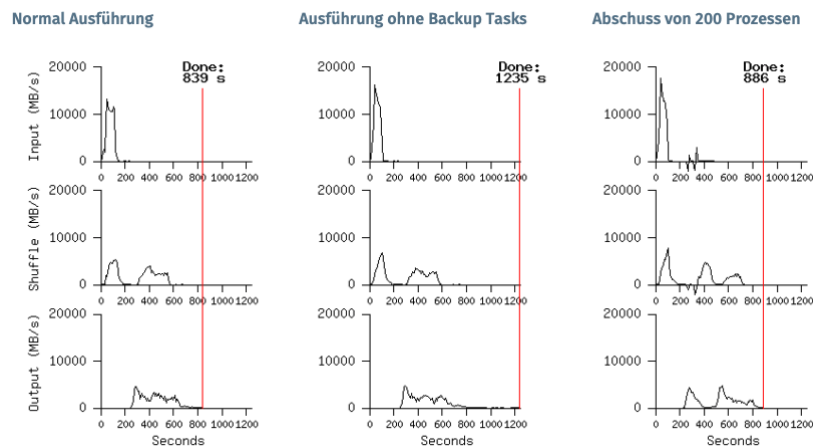


Abbildung 82: Performance Evaluation Originalpaper

10.1.10 Ausblick: Apache Hadoop

Open Source Distributed Processing Stack

- Stack für Big Data Processing in Java
 - zu wesentlichen Teilen durch Google-Publikationen inspiriert (z.B. MapReduce, Google File System)
 - ausgelegt für Cluster aus “normalen” Rechnern anstatt für dedizierte Super-Computer
- wichtige Module
 - Hadoop Common: Hauptbibliothek
 - HDFS: verteiltes Datei-System
 - YARN: Ressourcen-Management & Scheduling
 - MapReduce: ein mögliches Berechnungsmodell

10.1.11 Berechnungen mit MapReduce

Rückverweise auf vorherige Vorlesungsthemen

- Skalierung: scale-out
 - Verteilung der Berechnungen auf viele Maschinen durch Partitionierung der Eingabedaten

- maximale Parallelisierung der map() und reduce() Funktionen auf verfügbaren Maschinen
- Fokus auf Verteilungstransparenzen
 - einfaches Programmiermodell für User
 - MapReduce-System kapselt Großteil der Komplexität
- Verwendung von RPC intern zwischen Workern bzw. Coordinator
- Fehlertoleranz
 - Heartbeats (Erkennung) & erneute Ausführung von Tasks
- Optimierung
 - redundante Ausführung von Tasks
 - Datenlokalität (lokaler Zugriff schneller als entfernter Zugriff über das Netzwerk)

10.2 J.2 Bitcoin als Distributed Ledger Technology

10.2.1 Der Traum vom Digitalen Geld

- David Chaum 1982: Blind Signatures for Untraceable Payments
- David Chaum 1993: DigiCash
- Wie funktioniert DigiCash (Intuition): Analogie Umschlag mit Kohlepapier ausgelegt

10.2.2 Bitcoin: A Technical Perspective

Blockchain

- Auch "Distributed Ledger Technology"
- Append-only Logfile
- Verteilter Konsens-Mechanismus

10.2.5 Die Blockchain

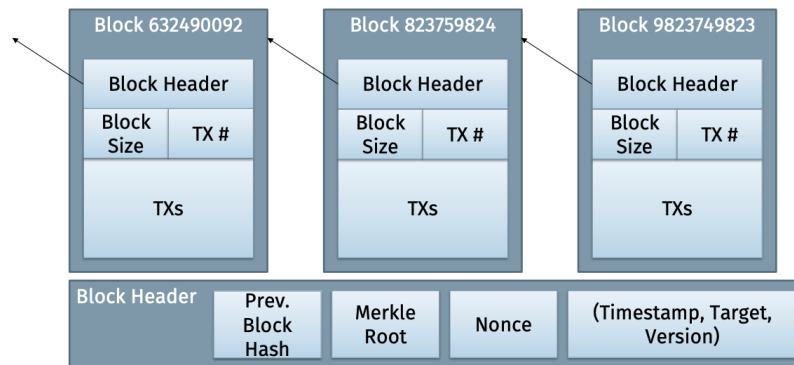


Abbildung 85: Blockchain

10.2.6 Was benötigen wir für Bitcoin (oder andere Kryptowährungen)?

Bitcoin zielt darauf ab, eine vollständige Version von kryptografischem Geld aufzubauen, in der jeder teilnehmen kann.

Dies erfordert einige Bausteine und bringt einige Herausforderungen mit sich:

- **Identität:** Besitzer von Werten
- **Transaktionen:** Beschreibung der Übertragung von Werten
- **Peer-to-Peer Netzwerk:** Kommunikation zwischen Knoten zur Übertragung von Transaktionen und Blöcken
- **Verteilter Konsens:** Vermeidung unterschiedlicher Sichten über den Zustand des Systems

10.2.7 Identität in Bitcoin

Identität:

- Wir müssen in der Lage sein, verschiedene „Nutzer“ von Bitcoin zu unterscheiden und ihnen zu ermöglichen, den Besitz des Wertes nachzuweisen.
- Benutzer sollten in der Lage sein, Identitäten ohne Hilfe zentraler Instanzen zu erstellen und zu verwalten.

- Identitäten werden durch kryptografische Schlüsselpaare dargestellt Verwendung von asymmetrischer Kryptografie

10.2.8 Kategorien von Kryptographie

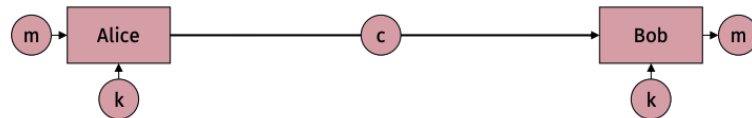


Abbildung 86: Symmetrische Kryptographie (secret key cryptography)

Asymmetrische Kryptographie (public key cryptography)

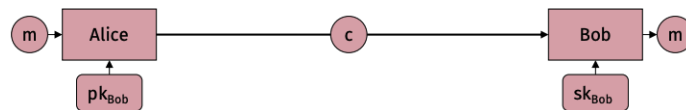


Abbildung 87: Asymmetrische Kryptographie (public key cryptography)

Häufige Begriffe in der Kryptographie

- A, B: Alice, Bob (Kommunikationspartner)
- m: Klartext
- c: Chiffre
- k: Geheimer gemeinsamer Schlüssel
- pk_{Bob} : Öffentlicher Schlüssel von Bob (Public Key)
- sk_{Bob} : Privater Schlüssel von Bob (Private Key)
- E: Eve (passiver Angreifer)
- M: Mallory (aktiver Angreifer)

10.2.9 Bitcoin Identitäten

Bitcoin-Identitäten sind Schlüsselpaare eines asymmetrischen Kryptoverfahrens

- Alice kann selbstständig ein Schlüsselpaar erstellen
- Durch den Nachweis der Kenntnis ihres geheimen Schlüssels kann sie ihre Identität nachweisen (→ digitale Signaturen)
- Identitäten in Bitcoin sind pseudonym
- Alice kann mehrere pseudonyme Identitäten erstellen

10.2.10 Transaktionen

In der einfachsten Form sollte Alice in der Lage sein, eine genau definierte Menge an Kryptowährung an Bob zu überweisen.

- Fragen:
 - Woher weiß man, dass die Transaktion wirklich von Alice stammt? → Digitale Signaturen
 - Woher weiß man, dass Alice tatsächlich 5B besitzt? → UTXO-Modell

10.2.11 Digitale Signaturen

- Reduktion der Signaturgrößen und Verbesserung der Geschwindigkeit:
 - Dokument erst hashen (Prüfsumme des Dokuments erstellen)

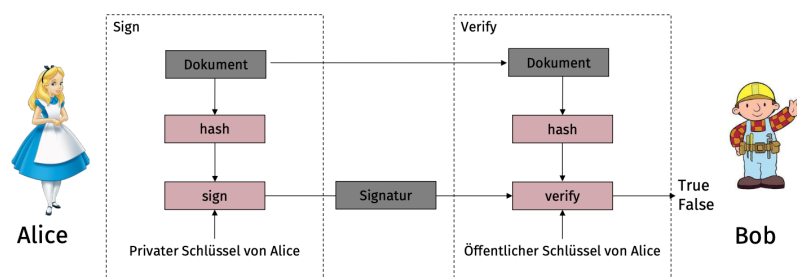


Abbildung 88: Digitale Signaturen

Digitale Signaturen stellen sicher:

- **Authentizität der Nachricht:** Die Signatur wurde vom Besitzer des privaten Schlüssels erstellt (entspricht dem öffentlichen Schlüssel, der für die Verifizierung verwendet wird)
- **Nichtabstreitbarkeit (non-repudiation):** Der Ersteller der Signatur kann später nicht leugnen, sie erstellt zu haben
- **Nachrichtenintegrität:** Wenn die Nachricht geändert wird, wird die Signatur ungültig

Bitcoin verwendet den ECDSA-Algorithmus mit 256-Bit-Schlüsseln

10.2.12 Schlüssel und Adressen in Bitcoin

- **Privater Schlüssel:** $sk_{Alice} = \text{Zufallszahl } n$
- **Öffentlicher Schlüssel** $pk_{Alice} = n \times P \pmod{p}$ (**Operationen auf Punkt P auf elliptischer Kurve**)
- **Computational-discrete Logarithm Problem**
 - Aus pk kann nicht sk (n) berechnet werden
 - Bei bestimmten Kurven ist dies bei ausreichend großen Zahlen praktisch nicht machbar
- $PubKeyHash_{Alice} = \text{RIPEMD160}(\text{SHA256}(pk_{Alice}))$
 - RIPEMD160 und SHA256 zwei kryptographische Hash-Funktionen
- **Bitcoin Address:** $Bitcoin_Address_{Alice} = \text{Base58CheckEncode}(PubKeyHash_{Alice})$
 - **Base58CheckEncode** enthält:
 - * 1 Byte Versionsnummer
 - * Base58Encode: Übersetzt Binärzahlen in Text (Ziffern + A-Z / a-z außer den mehrdeutigen Zeichen 0, O, I, l)
 - * 4-Byte-Prüfsumme

10.2.13 Kryptografische Hashfunktionen

Definition Hash-Funktion (Streuwertfunktion): $h : D \rightarrow S$, mit $|D| \geq |S|$

Gewünschte Eigenschaften:

- **Komprimierung:** $|D| \gg |S|$ (z.B. 1 MB $\rightarrow$ 256 Bit)
- **Chaotisch:** Geringste Änderung von D (1 Bit) führt zu maximaler Änderung von S (Bit-Flip mit 50% Wahrscheinlichkeit)
- **Surjektiv:** $|S|$ ist voll ausgelastet
- **Effizient:** h kann auch bei großen Eingaben schnell berechnet werden

Hinweis: Diese Eigenschaften allein machen noch keine kryptografische Hash-Funktion aus (Beispiel: CRC-32).

Kryptografische Anforderungen:

- **Einwegfunktion (first pre-image resistance):** Gegeben $h(m) = y$, so kann m nicht effizient aus y berechnet werden.
- **Schwache Kollisionsresistenz (second pre-image resistance):** Gegeben m , so kann kein $m' \neq m$ effizient gefunden werden, so dass $h(m') = h(m)$.
- **(Starke) Kollisionsresistenz:** Es können keine zwei verschiedenen m und m' effizient gefunden werden, so dass $h(m) = h(m')$.

Beispiele: SHA-2, SHA-3 (Veraltet/gebrochen: MD5, SHA-1).

Beispiel für kryptografische Eigenschaften:

- **Komprimierung:** 1 MB $\rightarrow$ 256 Bit
- **Chaotisch:** 1 Byte ändern oder hinzufügen $\rightarrow$ völlig anderer Hashwert
- **Effizient:** Berechnung des Hashwerts für eine 1-MB-Datei in 1/10 s
- **Einwegfunktion:** Bei Kenntnis des Hashwerts kann die Datei nicht rekonstruiert werden
- **Schwach kollisionsresistent:** Es kann keine zweite Datei mit demselben spezifischen Hashwert gefunden werden.
- **Stark kollisionsresistent:** Es können keine zwei beliebigen verschiedenen Dateien mit demselben Hashwert gefunden werden.

10.2.14 UTXO Modell (Unspent Transaction Output)

Wie kann Alice Bob beweisen, dass sie 5 Bitcoins besitzt, die sie ihm übertragen möchte?

- **Ansatz 1: Verwalten eines Kontostandes** (Account-based Model, z.B. Ethereum)
 - **Vorteile:** Einfacher zu verstehen, effizientere Smart Contracts, kleinere Transaktionsgröße.
 - **Nachteile:** Skalierbarkeit (Zustand muss sequenziell aktualisiert werden), geringere Privatsphäre.
- **Ansatz 2: UTXO-Modell** (z.B. Bitcoin)
 - Eingaben von Transaktionen verweisen auf noch nicht „verbrauchte“ Ausgaben aus vorherigen Transaktionen (UTXO).
 - **Vorteile:** Höhere Privatsphäre (durch Wechselgeldeadressen), parallele Verarbeitung möglich, deterministisch.
 - **Nachteile:** Komplexeres Wallet-Management, Datenaufwand (UTXO-Set wächst).

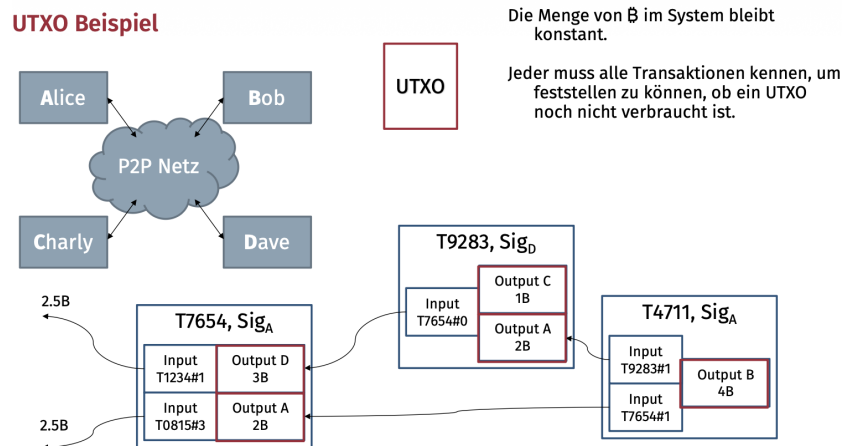


Abbildung 89: UTXO Beispiel

10.2.15 Verteilte, unveränderliche Datenbank (append-only) für Transaktionen

Auch Distributed Ledger genannt

- Distributed-Ledger-Technologien (DLTs)

Transaktionen werden an alle teilnehmenden Miner in einem Peer-to-Peer-Netzwerk übertragen

- Transaktionen erreichen Miner zu unterschiedlichen Zeitpunkten und in unterschiedlicher Reihenfolge
- → P2P-Netzwerk von Bitcoin
- Globale Reihenfolge wichtig
 - andernfalls unterschiedliche Sicht auf UTXO und Gültigkeit von Transaktionen

Globaler Konsens aufwändig und mit Latenz verbunden

- Gruppierung von nicht verketteten Transaktionen in Blöcken
- nur Entscheidung über Reihenfolge der Blöcke
- daher Blockchain

10.2.16 Block-ID

Block-ID = $H(\text{blockHeader}) = H(\text{prevBlockHash} || \text{Merkle-Root} || \text{Nonce} || \dots)$

- Änderung irgendeiner Transaktion → Änderung Merkle-Root → Änderung Blockheader → Änderung ID
- Änderung in vorherigem Block → gesamte Blockchain ab diesem Block ungültig
- Was ist ein Merkle-Tree und eine Merkle-Root?

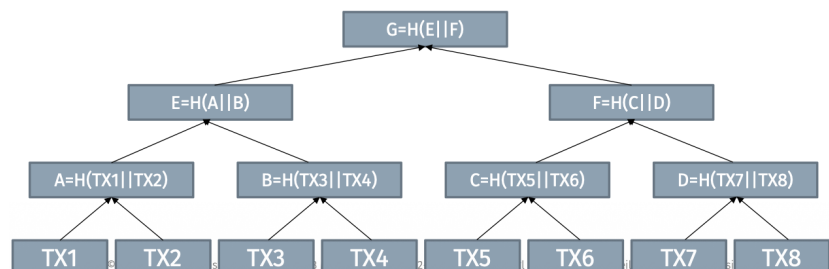


Abbildung 90: MerkleTree Beispiel

10.2.17 Konsens bei Bitcoin

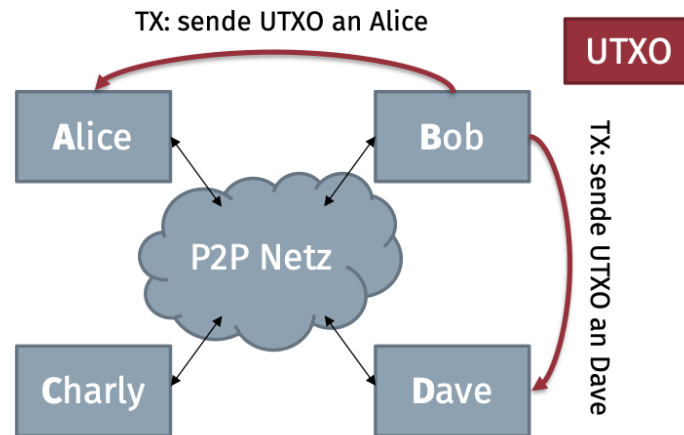


Abbildung 91: Konsens bei Bitcoin

10.2.18 Konsens

Wie können sich also alle auf gültige TXs, Blöcke und die Reihenfolge der Blöcke ohne zentrale Instanz einigen?

- Was ist, wenn wir zwei Transaktionen haben, die denselben UTXO ausgeben?
- Welche ist gültig und welche nicht?

Kein Problem die Informatik hat das bereits gelöst

- Nodes können über korrekte TXs abstimmen und falsche ablehnen
- PBTF-Protokoll (praktisch byzantische Fehlertoleranz)
- Erzielen Sie einen Konsens bei fehlerhaften oder bösartigen Knoten
- Kann bis zu $f < (n - 1) / 3$ fehlerhafte Knoten tolerieren (n: Gesamtzahl der Knoten im System)



Abbildung 92: Proof of Work

10.2.19 Blockchain Mining

$H(\text{BlockHeader}) < \text{target} \rightarrow$ Probieren von neuen Nonces, bis die Bedingung erfüllt ist $\rightarrow$ Block mining

10.2.20 Mining Intuition

Als würde man mit verbundenen Augen Pfeile auf eine Zielscheibe werfen

- Die Größe des inneren Rings bestimmt den Schwierigkeitsgrad (repräsentiert durch die Zahl der führenden „0“en im Hashwert)
- Ebenso bestimmt die Rate (Darts/s), mit der man Darts wirfst die Erfolgsquote (Mining-Rate)
- Mehr Miner sollten nicht zu einer höheren Rate der Anzahl der gefundenen gültigen Blöcke führen
- Schwierigkeit algorithmisch so gewählt, dass im Durchschnitt 2016 Blöcke pro 2 Wochen gefunden werden (oder alle 10 Minuten einer)
- Schwierigkeitsgrad $\ast = \text{two_weeks} / \text{time_to_mine_prev_2016_blocks}$

10.2.21 Bitcoin Script

Bitcoin-Transaktionen werden in einer Stack-basierten Skriptsprache ausgedrückt

- dies ermöglicht es, auch komplexere Transaktionen abzubilden

- aber es fehlen einige Elemente aus klassischen Computersprachen wie
 - Schutz vor Denial-of-Service Angriffen wie Endlosschleifen
- Transaktionen ordnen UTXO-Inputs den Outputs zu

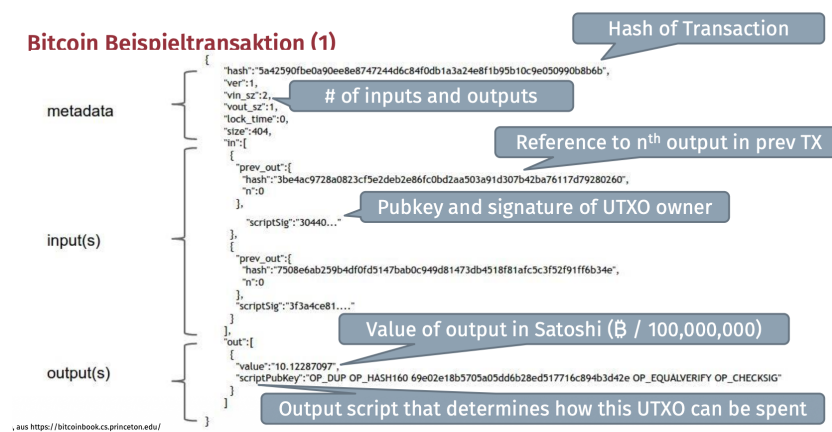


Abbildung 93: Bitcoin Transaktion Beispiel

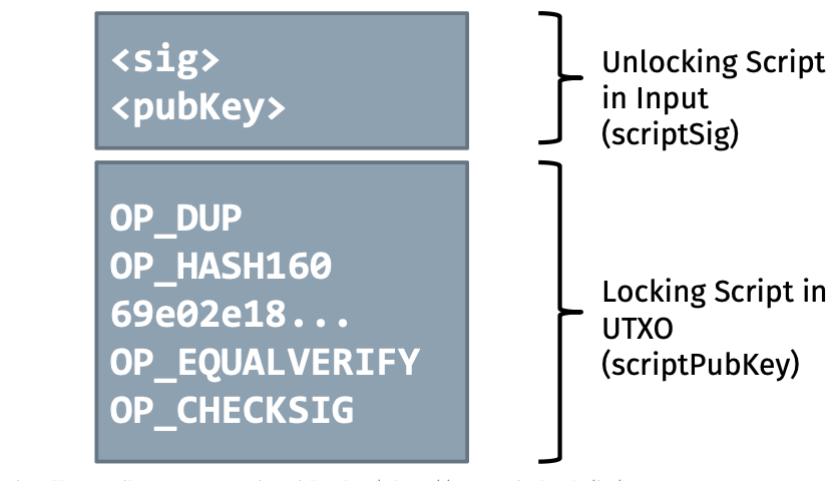


Abbildung 94: Bitcoin Transaktion Beispiel

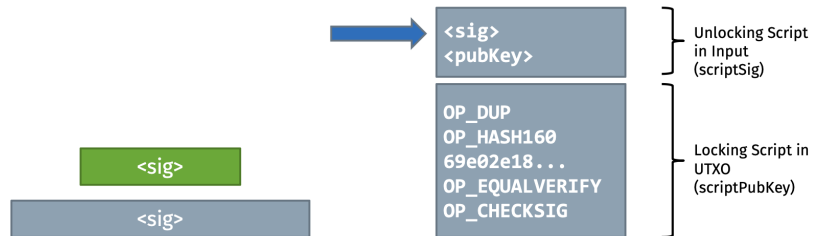


Abbildung 95: Bitcoin Transaktion Beispiel

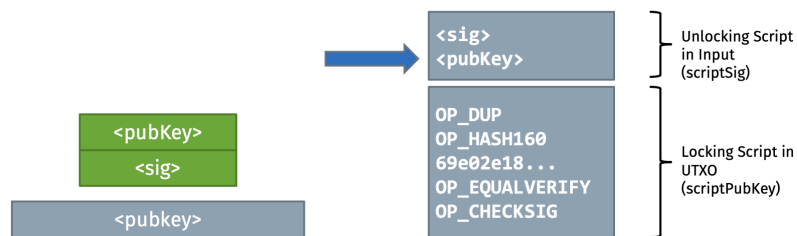


Abbildung 96: Bitcoin Transaktion Beispiel

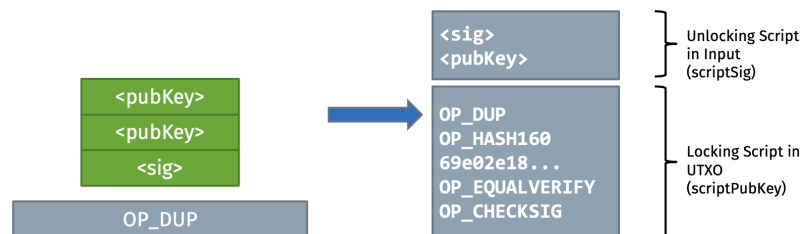


Abbildung 97: Bitcoin Transaktion Beispiel

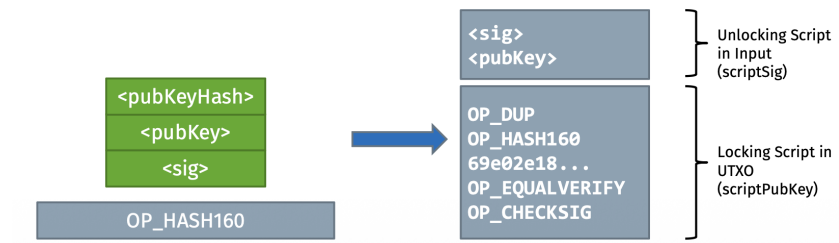


Abbildung 98: Bitcoin Transaktion Beispiel

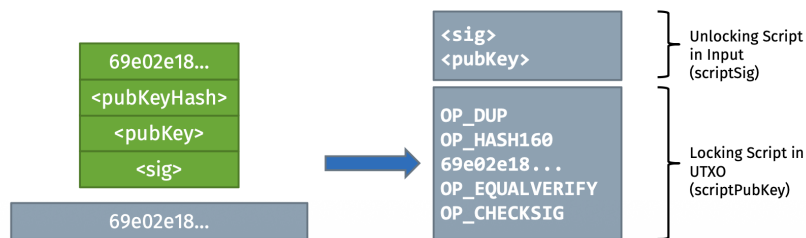


Abbildung 99: Bitcoin Transaktion Beispiel

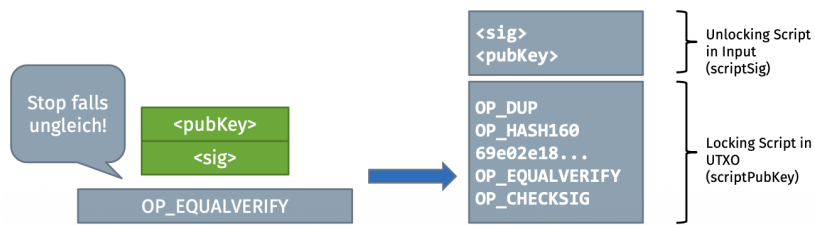


Abbildung 100: Bitcoin Transaktion Beispiel

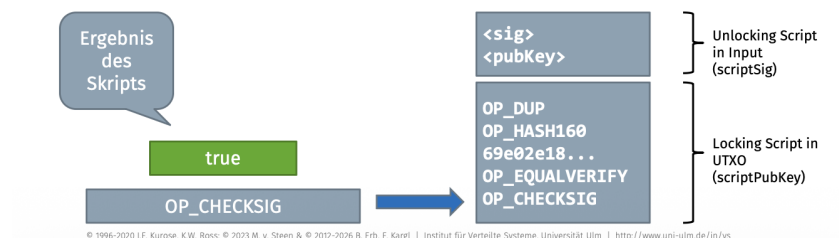


Abbildung 101: Bitcoin Transaktion Beispiel